CXL* Type 3 Memory Device Software Guide

Development Guides

August 2024

Revision 1.1

Document Number: 643805

Notice: This document contains information on products in the design phase of development. The information here is subject to change without notice. Do not finalize a design with this information.

Intel technologies may require enabled hardware, software or service activation.

You may not use or facilitate the use of this document in connection with any infringement or other legal analysis concerning Intel products described herein. You agree to grant Intel a non-exclusive, royalty-free license to any patent claim thereafter drafted which includes subject matter disclosed herein.

No license (express or implied, by estoppel or otherwise) to any intellectual property rights is granted by this document.

The products described may contain design defects or errors known as errata which may cause the product to deviate from published specifications. Current characterized errata are available on request.

Performance results are based on testing as of dates shown in configurations and may not reflect all publicly available updates. See backup for configuration details. No product or component can be absolutely secure.

Performance varies by use, configuration, and other factors. Learn more on the Performance Index site.

Your costs and results may vary.

"Conflict-free" refers to products, suppliers, supply chains, smelters, and refiners that, based on our due diligence, do not contain or source tantalum, tin, tungsten or gold (referred to as "conflict minerals" by the U.S. Securities and Exchange Commission) that directly or indirectly finance or benefit armed groups in the Democratic Republic of the Congo or adjoining countries.

All product plans and roadmaps are subject to change without notice.

Code names are used by Intel to identify products, technologies, or services that are in development and not publicly available. These are not "commercial" names and not intended to function as trademarks.

Intel disclaims all express and implied warranties, including without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement, as well as any warranty arising from course of performance, course of dealing, or usage in trade.

Altering clock frequency or voltage may void any product warranties and reduce stability, security, performance, and life of the processor and other components. Check with system and component manufacturers for details.

ENERGY STAR is a system-level energy specification, defined by the US Environmental Protection Agency, that relies on all system components, such as processor, chipset, power supply, and so forth. For more information, visit Energy Star.

Results have been estimated or simulated.

Intel does not control or audit third-party data. You should consult other sources to evaluate accuracy.

Copies of documents which have an order number and are referenced in this document may be obtained by calling 1-800-548-4725 or visiting the Intel Resource and Document Center.

Contents

Figures

Tables

Revision History

Revision Number	Description	Date
1.1	• Included the recommendation that Compute Express Link* (CXL*) expansion memory should be marked as Specific Purpose • Added section 2.13.24.1. It describes Host Physical Address (HPA) to Device Physical Address (DPA) translation algorithm when Modulo arithmetic combined with Exclusively-OR (XOR) is used • Added section 2.13.25.1. It describes DPA to HPA translation algorithm when Modulo arithmetic combined with XOR is used • Added section 2.13.25.2. It describes DPA to HPA translation algorithm when CXL Fixed Memory Window Structure (CFMWS) references a single Host Bridge instance twice • Updated references to match the latest *CXL Specification* and the terminology	August 2024
1.0	• Initial release of the document	June 2021

1 Document Overview

1.1 Document Goals

This document focuses on Compute Express Link* (CXL*) Memory Expander devices and the responsibilities of the software ecosystem to manage, configure, and enumerate these devices. Much of the content could apply to CXL RCDs that implement the Device Command Interface.

This is considered an informative document and is not meant to prescribe explicit software requirements but to demonstrate how the interfaces provided by CXL, ACPI, UEFI standards and the Engineering Change Notices (ECNs) are to be utilized by Software.

While the document identifies separate volatile and persistent memory architectural components and the flows differentiate volatile versus persistent memory capacity steps, the intent is there is a single CXL type 3 common memory driver that handles both types of devices.

The specific goals of this document include:

- Cleary delineate System Firmware, UEFI and OS driver responsibilities for supporting CXL
- Define informative behaviors for System Firmware, UEFI and OS Drivers for supporting CXL
- Demonstrate a set of high-level sequences for System Firmware, UEFI and OS Drivers to follow for supporting CXL

Non-goals of the document include:

- Specific System Firmware, UEFI and OS driver implementation
- Specific System Firmware, UEFI and OS driver policy
- Exhaustive low-level architecture and sequences
- PCI Express* (PCIe*) details

Target Audience includes:

- CXL software architects and authors
- CXL memory device architects and implementers
- System engineers wanting a deeper understanding of the CXL system software responsibilities, interfaces, and software sequences.

1.2 Document Scope

- CXL Memory Devices (Type 3) focused
- Provide enough information for CXL Memory device driver writers to create a high-level architecture/design

- Provide enough information for CXL Memory device driver writers to create high-level requirements and test plans
- The following are considered out of scope for the document at this time:
  - Hot adding of CXL Host Bridges
  - Hot add of devices in FW first: Requires platform to pass knowledge of the VDM MEFN vector to utilize when OS hot adds the device
  - More than one level of CXL switch is not comprehended in the flows

1.3 Reference Material Dependencies for this Version of the Document

- PCIe 6.2 or later Specification
- CXL 3.1 or later Specification
- ACPI 6.5 or later Specification
- UEFI 2.10 or later Specification
- Coherent Device Attribute Table (CDAT) 1.04 or later Specification
- SNIA Persistent Memory Programming Model
- Pmem.io
  - DSC White Paper
    https://pmem.io/documents/Dirty_Shutdown_Handling-V1.0.pdf
  - PMDK
    https://pmem.io/pmdk/

1.4 Abbreviations

Abbreviations used in this document that may not be found in the CXL, PCIe, ACPI or *UEFI Specifications*:

Table 1-1 Terms and Acronyms

Acronym	Expansion
CFMWS	ACPI CEDT CXL Fixed Memory Window Structure
CXIMS	CXL XOR Interleave Math Structure
DPA	CXL Memory Device Physical Address
HB	CXL Host Bridge. See XHB
HPA	Host Physical Address
Interleave Set	Collection of DPA ranges from one or more devices that make up a single HPA range. See Region
ISP	Interleave set position, the position of a device within an Interleave set. 0-based
LSA	Label Storage Area
OS Drivers	The collection of OS kernel components required to implement CXL

Acronym	Expansion
PMEM	Persistent memory
Region	CXL term for a memory interleave set. See Interleave set
UEFI Drivers	UEFI CXL Bus and Memory device drivers
XHB	Cross CXL Host Bridge interleave set. An interleave set the spans multiple Host Bridges. May or may not cross sockets

1.5 Open Issues

- This CXL SW Implementation Guide
  - Add Cross CXL Host Bridge (XHB) and switch position validation sequences in the creating regions flow
  - Add FW Activation flow: OS should reload CEL after a FW activation
  - In BIOS boot flow, remove checking of Get Part Info. BIOS will determine partitioning based solely on CDAT DSMAS ranges
  - Correct Figure 62 – Calculating HDM decoder settings for HB, Switch, nested switch IW calculation

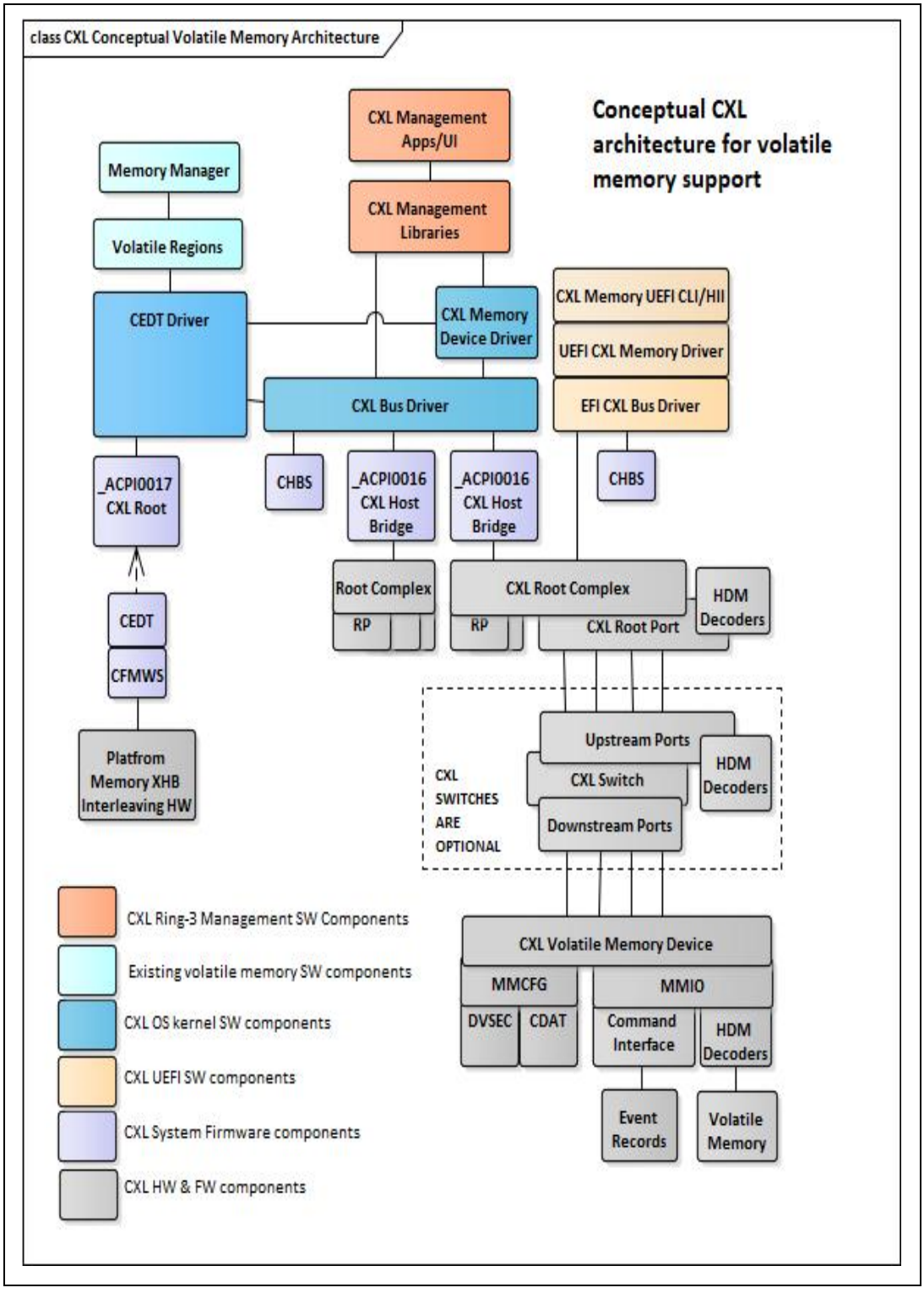
2 High Level Architecture

The following sections outline the high-level architectural components, their responsibilities, and a basic review of System Firmware interfaces.

2.1 Conceptual CXL Architecture for Volatile Memory Support

The section describes the conceptual CXL HW and SW components required for general CXL support with volatile CXL memory device. The architecture shown is not based on a specific OS implementation. See Section 2.3 for more details on the Linux* CXL memory architecture.

Figure 2-1 Conceptual CXL Architecture for Volatile Memory Support



CXL Volatile Memory Device (Type 3)

CXL Memory devices may be connected to the CXL Root Port or a CXL switch downstream port by one or more flexbus lanes. The device is mapped into Memory Map Configuration (MMCFG) and MMIO regions using standard PCIe mechanisms. The Type 3 specific memory device command MMIO mailbox interface is used to manage and configure the device.

HDM Decoders - The MMIO mapped HDM decoders are setup by the system firmware for known CXL volatile capacity, by the UEFI and OS driver for known CXL persistent capacity, and by the OS for hot added CXL volatile and persistent memory capacity. These registers determine what Host Physical Address (HPA) range will be mapped to the Device Physical Address (DPA) range exposed by the device. HDM decoders are found in all up-stream CXL switch ports as well as in each CXL Root Complex. These HDM decoders will also need to be programmed to account for all the downstream device HDM decoder programming. HDM decoders in the CXL Root Complex determine which root port is the target of the memory transaction. Similarly, HDM decoders in an upstream port determine which downstream port is the target.

CDAT - Standardized device registers to report latency, BW and memory type / attribute information. System firmware utilizes this information to build Heterogeneous Memory Attribute Table (HMAT) tables at platform boot time for volatile CXL memory devices. At OS boot and hot add time, the CDAT information is utilized by the OS in combination with the System Resource Affinity Table (SRAT) and HMAT to build a complete BW and latency picture for the persistent memory devices.

Command Interface - The CXL device command interface utilized by system firmware, UEFI and OS drivers to:

- Configure and manage the device
- Retrieve and store persistent region and namespace configuration information
- Configure, retrieve, and clear asynchronous device runtime alerts

The command interface surfaces the following to the system firmware, UEFI and OS drivers:

Device Capabilities, Capacity, and Partition Information

Event Log - Each CXL Memory device is required to support space for at least one event record in each of the informational, warning, failure, or fatal event severity queues. Asynchronous notification of new entries in the list is done using standard PCIe Medium Scale Integration (MSI)/MSI-X (OS First) or Vendor Defined Message (VDM) message interrupts (FW first).

Health Information - The device must maintain consistent health information including life used, device temperature, and persistent dirty shutdown count.

CXL Switch

Provides expansion of CXL Root Ports into many downstream switch ports allowing more CXL memory devices to be connected in a scalable way. Each switch has a set of HDM decoders that govern the upstream switch ports

decoding of the HPA. The system firmware, UEFI and OS drivers are responsible for programming these HDM decoders to cover all the devices and switches connected downstream.

CXL Root Port

CXL HW connection to the CXL memory device or CXL switch port via one or more flexbus lanes. Equivalent to a PCIe root port.

CXL Root Complex

Platform specific CXL root ports and the equivalent to a PCIe root complex. Bus number assignments are the responsibility of the system firmware and exposed through the CXL host bridge ACPI namespace device.

ACPI0016 CXL Host Bridge Object

Virtual SW entity implemented by the system firmware under the _SB (System Bus) of the ACPI tree and consumed by the OS. Each ACPI0016 object represents a single CXL root complex. Since the root of the CXL tree (the CXL root complex) is platform specific and is not presented through a PCI Base Address Register (BAR), the system firmware is responsible for generating an object to represent the collection of CXL root ports that represent a CXL host bridge. Each HB is represented by a unique ACPI0016 object under the top of the ACPI /SB device tree. There are a several ACPI methods attached to this object that the system firmware will implement on behalf of the OS. This is not an exhaustive list of ACPI methods the CXL host bridge device is expected to support. Assume all standard PCI host bridge driver methods for finding and configuring HW will apply:

- _CRS – Same as PCIe host bridge method
- _OSC - Determine FW first/OS first responsibility. Follows standard PCIe host bridge functionality with CXL additions. See the CXL _OSC section in *CXL 3.1 Specification*.
- _REG – Same as PCIe host bridge method
- _BBN – Same as PCIe host bridge method
- _SEG – Same as PCIe host bridge method
- _CBR – Retrieve pointer to the CXL host bridge register block which provides access to the HDM decoders. If the platform supports hot add of CXL host bridges the OS can utilize this method to find the register block after the addition of the new HB. CXL host bridge hot add is considered out of scope for this document. CXL host bridges present at boot will not have a _CBR method and the CEDT CXL Host Bridge Structure (CHBS) must be utilized.
- _PXM – Same as PCIe host bridge method

CXL Bus Driver

A CXL enlightened version of a standard PCIe bus driver that consumes all ACPI0016 CXL host bridge ACPI device instances and initiates CXL bus enumeration, understands the relationship between CXL host bridges, CXL

switches, and CXL end-point devices, and loads device specific CXL memory device driver instances for each supported end point it enumerates.

CXL Memory Device Driver

Each physical CXL memory device surfaced by the bus driver is consumed by a separate instance of the CXL memory device driver. This driver consumes the command interface and typically exports those features through OS specific IOCTLs to allow the OS in-band management stack components to manage the device.

ACPI0017 CXL Root Object

A virtual SW entity implemented by the system firmware under _SB of the ACPI tree that represents the presence of the CXL Early Discovery Table (CEDT). The following methods are attached to this device:

Get Quality of Service (QoS) throttling group DSM – Retrieve the QTG the device should be programmed to:

CEDT

CXL Early Discovery Table – System firmware provides this ACPI table that UEFI and OS drivers utilize to retrieve pointers to all of the CXL CHBS, a Set of Fixed Memory Windows (CFMWS) for each CXL host bridge present at platform boot time and optionally one or more CXIMS. The pointer to the register block allows the system firmware, UEFI and OS drivers to program HDM decoders for the CXL Host Bridges. While the ACPI0017 object will signal the presence of the CEDT, this table is not dependent on the ACPI0017 object since it must be available in early boot, before the ACPI device tree has been created. This is a static table created by the system firmware at platform boot time.

CHBS

ACPI CXL Host Bridge Structure – Each CXL Host Bridge instance will have a corresponding CHBS which identifies what version the CXL host bridge supports and a pointer to the CXL root complex register block that is needed for programming the CXL root complex's HDM decoder.

CFMWS

CXL Fixed Memory Window Structure – A new structure type in CEDT that describe all the platform allocated and programmed HPA based windows where the system firmware, UEFI and OS drivers can map CXL memory.

CEDT Driver

The ACPI0017 CEDT and CFMWS will be consumed by a bus driver that concatenates the HDM decoders for each CXL host bridge in to one or more regions (or interleave sets) that the bus driver surfaces to the existing Persistent Memory (PMEM)/ Security Control Module (SCM) stacks. Since interleave sets in CXL may span multiple CXL host bridges, this driver handles XHB interleaving, and presents other drivers in the stack with a single

consistent set of regions, whether they are contained in a single CXL Host Bridge or span multiple CXL host bridges

Volatile Region

Each volatile region represents an HPA range that utilizes a set collection of CXL memory devices with volatile capacity.

Memory Manager

Volatile regions are typically consumed by the OS memory manager which controls allocation and deallocation of physical memory on behalf of other ring 0 and ring 3 SW components.

In-band OS Management Stack

CXL Management Apps/UI – CXL management applications and user interfaces utilizing CXL management libraries.

CXL Management Libraries – Management libraries covering the standardized CXL interfaces.

CXL based in-band management libraries and UI components that will utilize OS implementation specific IOCTLs and pass-through interfaces surfaced by the CXL root port bus driver, CXL memory device driver, and the PMEM or SCM region driver instances.

UEFI Management Stack

CXL Memory EFI Driver

CXL Bus EFI Driver

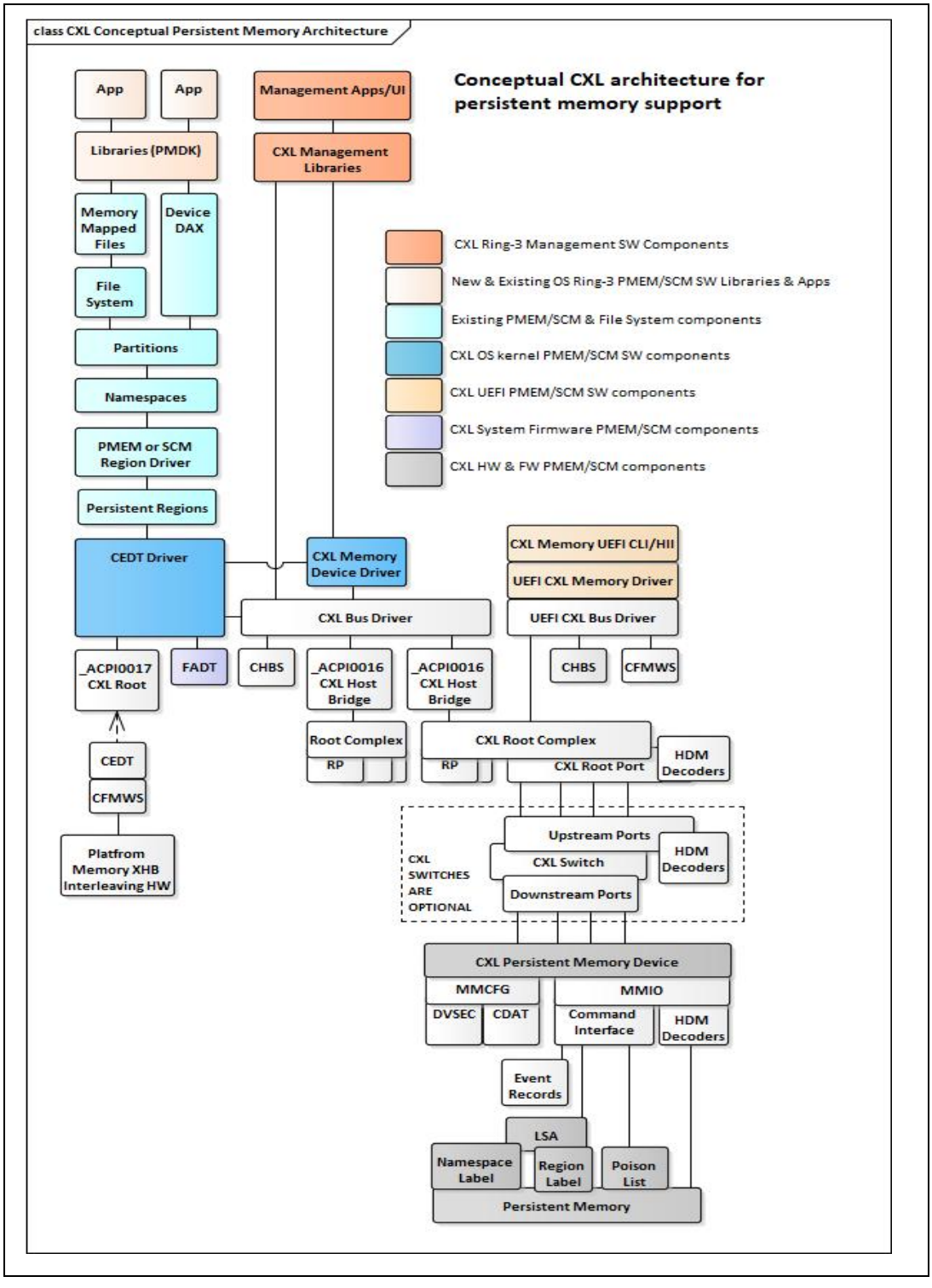
CXL Memory EFI Command Line Interface (CLI)/ Human Interface Infrastructure (HII)

UEFI pre-boot environment CXL bus driver that surfaces EFI_BLOCK_IO protocol (utilizing byte addressable persistent memory) for consumption by the OS boot loader. This also provides management interfaces for the UEFI based management stack. This may be implemented using a pre-boot CXL Bus driver and CXL Memory device driver.

2.2 Conceptual CXL Architecture for Persistent Memory Support

This section describes the additional new and updated CXL HW and SW components required to support CXL persistent memory devices. The architecture shown is not based on a specific OS implementation. See Section 2.3 for more details on the Linux CXL memory architecture.

Figure 2-2 Conceptual CXL Architecture for Persistent Memory Support



CXL Persistent Memory Device (Type 3)

Command Interface - The CXL Device Command Interface utilized by System Firmware, UEFI and OS drivers to expose additional persistent memory features:

LSA - The CXL Memory device is responsible for surfacing a persistent label storage area that the UEFI and OS drivers utilize to read and write Region (interleave set) configuration information and Namespace configuration information. This is required to re-assemble the persistent memory region configuration correctly.

Region Label – Persistent configuration information that describes to the UEFI and OS drivers the persistent memory region, the Universal Unique Identifier (UUID) of all devices involved, the amount of persistent capacity each contributes to the region, and the position each device contributes to the region. With persistent memory, the ordering of the device in the region must always be preserved to re-assemble the data from the region correctly.

Namespace Label – Persistent configuration information that describes to the UEFI and OS drivers how each persistent memory region is subdivided into namespaces. There are several types of namespaces including BTT based block storage emulation.

Poison List - The device is required to maintain a persistent poison list so the UEFI and OS drivers can quickly determine what areas of the media contain invalid data and must be avoided or corrected.

Fixed ACPI Description Table (FADT)

Fixed ACPI Description Table – Existing ACPI table that is utilized to report fixed attributes of the platform at platform boot time. For CXL, the new PERSISTENT_CPU_CACHES attribute is utilized by the platform to report if the CPU caches are considered persistent and by the OS to set application flushing policy.

Persistent Region

Each persistent region represents an HPA range that utilizes a set collection of CXL Memory devices with persistent capacity configured in a specific order. The configuration of each region is described in the region labels stored in the Label Storage Area (LSA) which is exposed through the command interface.

PMEM or SCM Region Driver

Each instance of a region (interleave set) will be consumed by a separate instance of the PMEM or SCM driver. This is probably significant re-use of the existing OS kernel NVDIMM components for this.

Namespaces

Each region can be subdivided into volumes referred to as Namespaces. The configuration of each Namespace is described in the namespace labels stored in the LSA which is exposed through the command interface. Persistent memory namespaces are described in detail in the *UEFI Specification*.

Partitions

Each Namespace is typically sub-divided into partitions by the OS.

File Systems

Existing file system drivers subdivide each partition into one or more files and supply standard file API and file protection for the user.

Memory Mapped Files
Regions are subdivided into Namespaces which are subdivided into partitions and finally subdivided into memory mapped files by the file system. This is one standard mechanism for applications to access persistent memory directly with the security and convenience of a file.

Device Direct Access (DAX)
A simplified direct pipeline between the application and the persistent memory namespace that bypasses the filesystem and memory mapped file usage.

Libraries (Persistent Memory Developer Kit [PMDK])

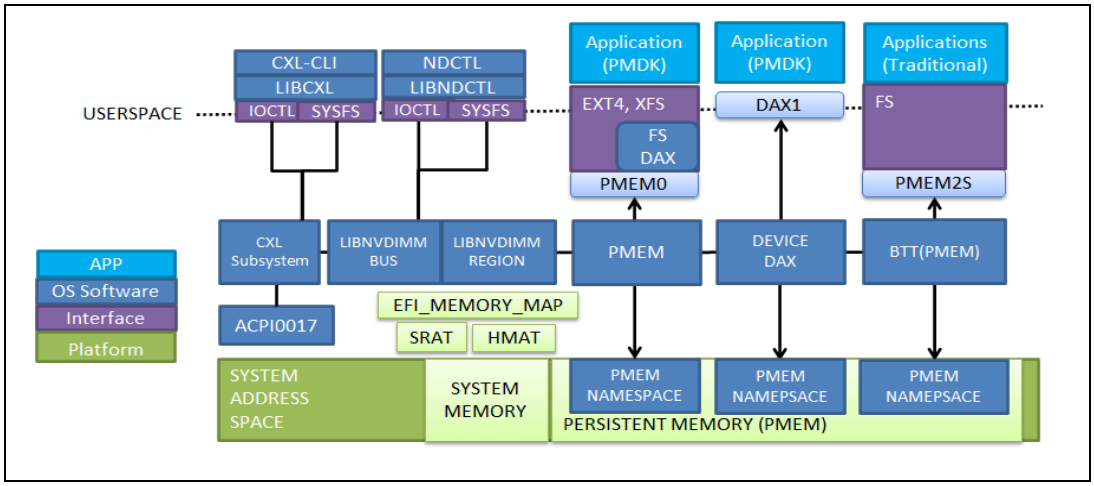
Most persistent memory aware applications make use of Ring3 libraries like PMDK to simplify the persistent memory programming model. These libraries typically make use of memory mapped files or direct device DAX to access the persistent memory. There will be additions to these libraries to surface new CXL features.

Applications

2.3 Basic Linux CXL Architecture

This section describes the basic Linux CXL architecture, the main SW components, and a brief description about each with special emphasis on persistent memory. CXL for memory devices utilizes the existing Linux NVDIMM architecture, replacing the NVDIMM Firmware Interface Table (NFIT) based interfaces with ACPI0017 CEDT based CXL interfaces, and adding a CXL subsystem to handle the new CXL specific functionality.

Figure 2-3 Basic Linux CXL PMEM Architecture



CXL Subsystem

New kernel component that utilizes the information provided through the ACPI0017 CEDT. Provides CXL specific services for the NVDIMM BUS component. Provides CXL specific IOCTL and SYSFS interfaces for management of CXL devices

LIBNVDIMM BUS

Existing LIBNVDIMM BUS component. Provides generic NVDIMM bus related functionality including namespace management. Enumerates memory device endpoints for the LIBNVDIMM REGION component

LIBNVDIMM REGION

Existing LIBNVDIMM REGION component. Consumes endpoint memory devices produced by the LIBNVDIMM BUS, consumes the region labels on each device (from the LSA), organizes the region labels in to interleave sets, validates the regions, and publishes regions to PMEM, DEVICE DAX and Block Translation Table (BTT) components.

PMEM

Kernel component that represents a single instance of a region

DEVICE DAX

A simplified direct pipeline between the application and the persistent memory namespace that bypasses the filesystem and memory mapped file usage.

FS DAX

Direct memory access for memory mapped file systems.

BTT

Component that consumes the devices block translation table and implements a block storage protocol on top of persistent memory.

PMEM NAMSPACE

Each region can be subdivided into volumes referred to as Namespaces. The configuration of each Namespace is described in the namespace labels stored in the LSA which is exposed through the Command Interface.

CXL-CLI

Linux CXL memory management Ring3 CLI.

LIBCXL

Linux CXL memory management Ring3 library.

NDCTL

Linux NVDIMM memory management Ring3 CLI utilized with CXL.

LIBNDCTL

Linux NVDIMM memory management Ring3 library.

2.4 High-level Software Component Responsibilities

In its most basic form, the delineation of SW component responsibilities is:

- The system firmware is responsible for enumerating and configuring volatile CXL memory capacity that is present at boot.
- The OS components are responsible for enumerating and configuring all topologies not covered by the previous system firmware.
- The UEFI driver is optionally responsible for enumerating and configuring persistent memory devices that are in the boot path.

The following tables describe these high-level roles and responsibilities for major SW components in greater detail. Most of these responsibilities are outlined in more detail in the following sections of this document.

Table 2-1 High-level Software Component Responsibilities – System Boot

Function	System Firmware CXL Responsibilities	UEFI CXL Bus and Memory Driver Responsibilities	OS CXL CEDT, Bus and Memory Driver Responsibilities
CXL hierarchy enumeration	Enumerate complete CXL hierarchy for volatile and persistent capacity	Optionally: Enumerate CXL hierarchy for persistent capacity in the boot path only	Enumerate complete CXL hierarchy for volatile and persistent capacity

Function	System Firmware CXL Responsibilities	UEFI CXL Bus and Memory Driver Responsibilities	OS CXL CEDT, Bus and Memory Driver Responsibilities
MMIO BAR Configuration	Configure BARs as needed to enumerate the CXL hierarchy it is responsible for	Optionally: Configure BARs as needed to enumerate the CXL hierarchy	Optionally: Configure BARs as needed to enumerate the CXL hierarchy
Creating persistent memory region labels	N/A – System Firmware does not create persistent memory region labels	When no region labels exist in the device's LSA: • Partition the device volatile and persistent boundary according to the device's capabilities and admin policy. If the OS and device support re-partitioning without a reboot (SetPartitionInfo w Immediate flag), UEFI and OS should assume the CDAT may have changed. • Check the available System Firmware programmed CFMWSs available and only allow configuring of persistent memory regions that match the available windows and HB interleave granularity and ways Write the region labels following the *CXL Specification* • Read the region labels from each memory device, verify requested configuration • Programming the device for the region label defined	
Consuming persistent memory region labels	N/A – System Firmware does not consume persistent memory region labels	• Read the region labels from each memory device, verify requested configuration • Programming the device for the region label defined	
Programming CXL HDM decoders for configured regions	Program platform for platform specific volatile CXL capacity: • For device HDM decoders, program device HDM decoder global control register to enable HDM use, disabling Designated Vendor Specific Extended Capability (DVSEC) decoder use • Program HDM decoders in the memory device, upstream switch ports, and CXL Host Bridges to decode volatile memory • Utilize the CDAT DSMAS memory ranges returned by the memory device, and program HDMs aligned to those ranges • For immutable volatile configurations, set fixed device configuration indicator in CFMWS and utilize Lock on Commit to prevent others from changing these HDMs	For all volatile and memory devices not configured by the system firmware: • Track which volatile capacity has already been assigned an HPA range by the System Firmware by checking the devices HDM decoders • Utilize the CDAT DSMAS memory ranges returned by the memory device, the QoS Throttling Group from the platform, and the QTG from the CFMWS and program HDMs aligned to the DSMAS ranges. • Place device in to fixed memory window and use HPA range for programming HDM decoders while avoiding HPA ranges already populated by the System Firmware. • For device HDM decoders, program device HDM decoder global control register to enable HDM use, disabling DVSEC decoder use • Program HDM decoders in the memory device upstream switch ports, and CXL Host Bridges to decode volatile and persistent memory	

Function	System Firmware CXL Responsibilities	UEFI CXL Bus and Memory Driver Responsibilities	OS CXL CEDT, Bus and Memory Driver Responsibilities
	• Program all platform HW for each fixed memory region that the platform supports and surface fixed windows through ACPI CFMWS for HB and XHB interleaving. It is assumed the windows reported represent the platforms best performance configuration • Implement QoS Throttling Group DSM object under to the ACPI0017 device and map device latency and bandwidth to supported platform CFMWS QTG		
Event interrupt enabling	FW First: For platform specific volatile CXL capacity: Program memory device event log interrupt steering for FW first MEFN vector based on platform options through _OSC, retain ownership of CXL device memory error reporting	N/A	• For all volatile and memory devices not configured by the System Firmware: Determine OS First/FW First using _OSC • OS First: Program memory device event log interrupt steering for OS First MSI/MSIx • FW First: Do not program the device event log interrupts
Event interrupt handling	FW First: • Use host specific means to determine which CXL device(s) generated MEFN • Check device event log source to determine what logs need harvesting • Harvest appropriate event logs, consume event log content, generate Microsoft Windows* Hardware Architecture (WHEA)/ELOG CPER entries for the CXL Event Records, notify OS via correctable (SCI) or uncorrectable interrupt (example: NMI) • Clear event records Repeat until all logs retrieved and cleared	N/A	OS First: • Use OS specific means to locate the device needing attention • Check if the device generated MSI/MSI-X because of memory events • Check device event log source to determine what logs need harvesting • Harvest appropriate event logs, consume event log content • Clear event records

Function	System Firmware CXL Responsibilities	UEFI CXL Bus and Memory Driver Responsibilities	OS CXL CEDT, Bus and Memory Driver Responsibilities
			• Repeat until all logs retrieved and cleared FW First: • Handle WHEA SCI and NMI interrupts, consume CPER CXL Event Records.
ACPI0016 ACPI object	• Create ACPI0016 ACPI objects for each CXL Host Bridge in ACPI namespace • Implement standard PCIe Host Bridge methods associated with CXL Host Bridges and CXL _OSC	N/A	• Consume ACPI0016 objects and utilize standard PCIe Host Bridge methods for enumeration and configuration
ACPI0017 ACPI object	• Create single ACPI0017 ACPI object for the platform • Publish CEDT, CHBS, CFMWS • Implement Get QTG DSM associated with ACPI0017	N/A	• Consume ACPI0017 object and consume CEDT, CHBS, CFMWS • Utilize the Get QTG DSM when determining best CFMWS to utilize when programming HDMs during enumeration
EFI_MEMORY_MAP	• Create EFI_MEMORY_DESCRIPTORs for all volatile CXL memory present at boot • No descriptors for persistent memory ranges as the System Firmware does not program those HDMs • No descriptors for hot plug memory ranges as the System Firmware does not program those HDMs	N/A	• Consume EFI_MEMORY_MAP as needed for volatile capacity for legacy functionality
SRAT	• Create proximity domains for CPUs, attached CXL Host Bridges using Generic Port Affinity Type, and all CXL volatile memory devices • No SRAT entries for intermediate switches • Build Memory Affinity Structures for each volatile proximity domain with the SRAT Enable flag set.	N/A	• Consume SRAT as needed for volatile capacity for legacy functionality

Function	System Firmware CXL Responsibilities	UEFI CXL Bus and Memory Driver Responsibilities	OS CXL CEDT, Bus and Memory Driver Responsibilities
HMAT and CDAT	For memory devices containing volatile capacity: Parse device and switch CDAT and create HMAT entries for CPU and volatile memory proximity domains found in the SRAT	N/A	For all persistent capacity: Utilize memory device CDAT, switch CDATs, and Generic Port entries to calculate total BW and Latency for the path from the CXL Host Bridge to each device
Security		For memory devices containing persistent capacity: • Unlock device before the memory is accessed	For memory devices containing persistent capacity: • Unlock device before the memory is accessed
Managing device health	For memory devices containing volatile capacity: • Check health status before configuring or utilizing memory device	For memory devices containing persistent capacity: • Check health status before configuring or utilizing memory device • SetShutdownState with state=DIRTY before mapping the capacity by programming HDM decoders	For memory devices containing unconfigured volatile or persistent capacity: • Check health status before configuring or utilizing memory device • Retrieve DSC from previous boot and make available to kernel or application components and if DSC has incremented, optionally prevent device access or verify critical data is intact before allowing device to be accessed • SetShutdownState with state=DIRTY before mapping the capacity by programming HDM decoders

Table 2-2 High-level Software Component Responsibilities - System Shutdown and Global Persistent Flush (GPF)

Function	System Firmware CXL Responsibilities	UEFI CXL Bus and Memory Driver Responsibilities	OS CXL CEDT, Bus and Memory Driver Responsibilities
Preparing for device shutdown	If System Firmware OS shutdown handler present: Check each device GetShutdownState == CLEAN and if DIRTY: • If CPU caches are part of the platform persistence domain: Flush all CPU caches, all Type 2 devices • SetShutdownState state=CLEAN after flushing data to the device	N/A	• Quiesce memory accesses to the device • If FADT.PERSISTENT_CPU_CACHES = 10b: Optionally flush all CPU caches, all Type 2 devices • If HDM is not locked: Un-program device HDMs to prevent any other writes • SetShutdownState state=CLEAN after flushing data to the device
GPF Initialization	• Enable GPF on the platform • Determine if CPU caches are persistent and export FADT.PERSISTENT_CPU_CACHES ACPI interface		• During CXL enumeration: Calculates and configures CXL GPF Timeouts for switches and host bridges the volatile and persistent capacity is connected to

Table 2-3 High-level Software Component Responsibilities - Hot Add

Function	System Firmware CXL Responsibilities	UEFI CXL Bus and Memory Driver Responsibilities	OS CXL CEDT, Bus and Memory Driver Responsibilities
Managing regions	Program all platform HW for each fixed memory region that the platform supports for hot-plug of volatile or persistent memory and surface fixed windows through ACPI CXL Fixed Memory Window Structures (CFMWS) for HB and XHB interleaving.	Not Supported	Upon hot add event: Hot added volatile and persistent memory devices: • Program HDM decoders for memory device based on assigned HPA range, HDM decoders for upstream switch ports, HDM decoders for CXL Host Bridges • Reprogram GPF timeouts and other values that depend on the number of device present, for CXL

Function	System Firmware CXL Responsibilities	UEFI CXL Bus and Memory Driver Responsibilities	OS CXL CEDT, Bus and Memory Driver Responsibilities
			switches and Host Bridges • Utilize the CDAT DSMAS memory ranges supported by the memory device, the QoS Throttling Group (QTG) from the platform, and the QTG from the CFMWS and program HDMs aligned to the DSMAS ranges. • Utilize the Get QTG DSM when determining best CFMWS to utilize when programming HDMs
		Not Supported	Hot added volatile and persistent memory devices: • OS First: Program memory device event log interrupt steering for OS First MSI/MSIx based on _OSC FW First: Skip programming memory device event log interrupts
Event interrupt steering	N/A	Not Supported	Hot added volatile and persistent memory devices: • OS First: Program memory device event log interrupt steering for OS First MSI/MSIx based on _OSC • FW First: Skip programming memory device event log interrupts
HMAT and CDAT	N/A - _HMA method not required since OS will handle natively	Not Supported	Hot added volatile and persistent memory devices: • Utilize memory device CDAT,

Function	System Firmware CXL Responsibilities	UEFI CXL Bus and Memory Driver Responsibilities	OS CXL CEDT, Bus and Memory Driver Responsibilities
			switch CDATs, and CXL Host Bridge HMAT information to calculate total BW and Latency for the path from the CXL Host Bridge to the new device
SRAT	Indicate hot pluggable proximity domains with Memory Affinity Structure HotPluggable indicator		
GPF	N/A	Not Supported	Hot added volatile and persistent memory devices: Read existing switch and host bridge GPF TO values, calculate updated values for persistent memory, and configure CXL GPF Timeouts for switches, and host bridges the persistent memory is connected to.

Table 2-4 High-level Software Component Responsibilities - Managed Hot Remove

Function	System Firmware CXL Responsibilities	UEFI CXL Bus and Memory Driver Responsibilities	OS CXL CEDT, Bus and Memory Driver Responsibilities
Preparing for device removal	N/A	Not Supported	Removal of volatile or persistent capacity that was previously mapped: • If memory pages cannot be vacated, or device removal cannot be supported, do not allow removal • Quiesce all memory accesses • Offline/unmap the memory range so the device memory becomes inaccessible • Flush all CPU caches (what GPF would have done) • If HDM decoders are not locked: Un-program the HDM Decoders for the device being removed • SetShutdownState state=CLEAN

Function	System Firmware CXL Responsibilities	UEFI CXL Bus and Memory Driver Responsibilities	OS CXL CEDT, Bus and Memory Driver Responsibilities
Managing regions			Managed Hot Remove of volatile capacity that was previously mapped: • Only allow removal of volatile capacity whose HDM decoders were not previously locked by the System Firmware. System Firmware should only lock HDM decoders for immutable configurations that cannot support hot add or managed hot remove without a system reboot. Managed hot removed of volatile or persistent capacity that was previously mapped: • Determine what HPA range is being removed, program HDM decoders for affected switch ports, HDM decoders for CXL Host Bridges to remove assigned HPA range • Reprogram GPF timeouts and other values that depend on the number of device present, for CXL switches and Host Bridges

2.5 High-level System Firmware Memory Interface Overview

The following table summarizes the legacy memory and CXL related tables the System Firmware produces for consumption by the OS or UEFI drivers. See the SRAT and HMAT section of this document for more details.

Table 2-5 System Firmware Memory Interface Summary

System Firmware Interface	Description	Responsibility for CXL Volatile Memory Capacity	Responsibility for CXL Persistent Memory Capacity	Responsibility for CXL Hot Added Memory Capacity	CXL UEFI and OS Driver Implications
EFI MEMORY MAP	ACPI/UEFI HPA based memory descriptors describing physically present memory ranges the System Firmware has configured	YES If CDAT DSEMTS is not provided: • Type EfiConventionalMemory set, EFI_MEMORY_SP attr set,	NO	NO	Must consume EFI Memory Map

System Firmware Interface	Description	Responsibility for CXL Volatile Memory Capacity	Responsibility for CXL Persistent Memory Capacity	Responsibility for CXL Hot Added Memory Capacity	CXL UEFI and OS Driver Implications
		NonVolatile clear If CDAT DSEMTS is provided: • Type EfiReservedMemoryType if so specified by CDAT DSEMTS, NonVolatile clear • Otherwise type EfiConventionalMemory set, EFI_MEMORY_SP attr set, NonVolatile clear (even if CDAT does not specify EFI_MEMORY_SP)			
CEDT CFMWS	ACPI table describing all CXL HPA ranges programmed by the System Firmware at boot time	YES	YES	YES	Must consume CEDT CFWMS
SRAT	ACPI table of Proximity Domains and associated memory ranges	YES -CPU proximity domain for each CPU like today	YES -CPU proximity domain for each CPU like today	YES –Volatile Memory proximity domains will indicate Hot Pluggable but the size field will represent boot time capacity	

System Firmware Interface	Description	Responsibility for CXL Volatile Memory Capacity	Responsibility for CXL Persistent Memory Capacity	Responsibility for CXL Hot Added Memory Capacity	CXL UEFI and OS Driver Implications
		-CXL Host Bridge Generic port proximity domain for each CXL Host Bridge -CXL volatile memory proximity domain for each volatile region configured by System Firmware NO SRAT entries for CXL volatile memory not configured by the System Firmware Bit 3 (Specific Purpose) flag in SRAT must be consistent with the EFI_MEMORY_SP flag in EFI memory map	-CXL Host Bridge Generic port proximity domain for each host bridge NO SRAT entries for CXL persistent memory	NO SRAT entries for CXL persistent memory	Without persistent memory information in the SRAT and HMAT, must calculate NUMA distances for persistent memory devices using the Generic Port information in the SRAT combined with the memory device and intermediate switch CDAT. See Section 2.11 for details
HMAT	ACPI table that describes a matrix of BW and Latency performance characteristics between each Initiator (CPU or GI) proximity domain and each Memory proximity domain	YES, covers all of the Initiator and Target proximity domains that are listed in the SRAT	NO	N/A	

2.6 Memory Provisioning

- The proposed memory allocation scheme outlined here utilizes a fixed, platform defined set of HPA memory windows that are fixed at platform boot time based on the supported configurations and features for that platform.

- System Firmware fixed HPA based memory windows and restrictions for using each window, are described in the CEDT CFMWS to the UEFI and OS drivers.
- CFWMS entries are produced by System Firmware and consumed by the OS.
- The System Firmware will configure fixed memory windows for use with volatile and persistent memory.
- The System Firmware will allocate volatile capacity from the volatile memory windows and program HDM decoders for the volatile capacity.
- The OS will allocate persistent capacity from the persistent memory windows and program HDM decoders for persistent capacity based on region labels read from the LSA.
- The OS will also allocate volatile and persistent capacity from the same windows for hot adding new volatile/persistent memory or managed hot removing existing volatile/persistent memory.
- The OS must check programmed CXL memory HDM decoders during enumeration and understand what devices are already utilizing portions of the fixed window HPA range.
- By utilizing this simple fixed mechanism, runtime OS interactions to request the System Firmware to set up or tear down resources during OS boot time enumeration, runtime Hot Add and Managed Hot Remove events, are eliminated.

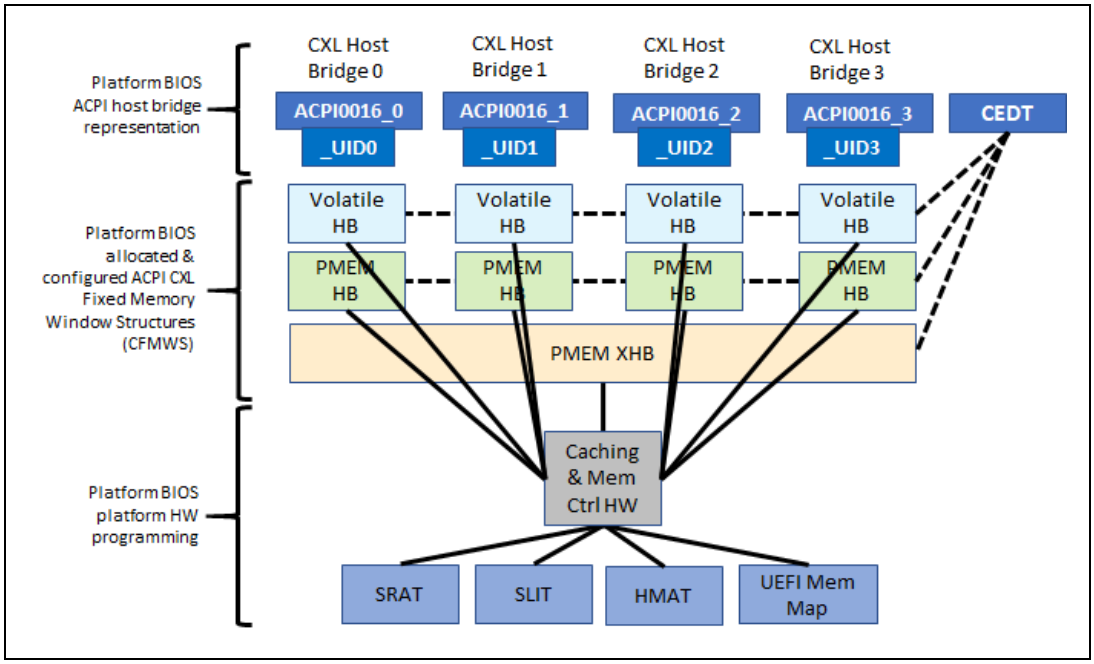
The following requirements limit what the System Firmware may surface to the UEFI and OS drivers:

- The architecture would allow mixing of any combination of the Window Restrictions. It is the responsibility of the System Firmware to only surface windows with Window Restrictions that the platform supports. If the platform HW does not allow T2 and T3 CXL memory devices to utilize the same HPA range, the System Firmware cannot report both T2 and T3 Window Restrictions in a single window.
- The System Firmware may surface windows that provide the UEFI and OS drivers multiple options for configuring a given region. It is UEFI and OS driver policy specific as to which possible window is utilized for configuring the region.
- There cannot be any overlap in the HPA ranges described in any of the CFMWS instances.

The following figure demonstrates an example of the intended architecture with the following example fixed windows:

- **Volatile HB** – Window for adding interleaved volatile memory that is local to this HB.
- **PMEM HB** – Window for adding interleaved and non-interleaved persistent memory that is local to this HB.
- **PMEM XHB** – Window for adding interleaved persistent memory that spans multiple HBs.

Figure 2-4 Example per CXL Host Bridge Fixed Memory Allocation



See the next examples that outline the intended content of the CFMWS, for various CXL topologies.

2.6.1 EFI_MEMORY_MAP

The CEDT CFMWS structures are based on the resources the System Firmware has allocated and programmed with the platform HW. See previous sections for the outline of the EFI_MEMORY_MAP content.

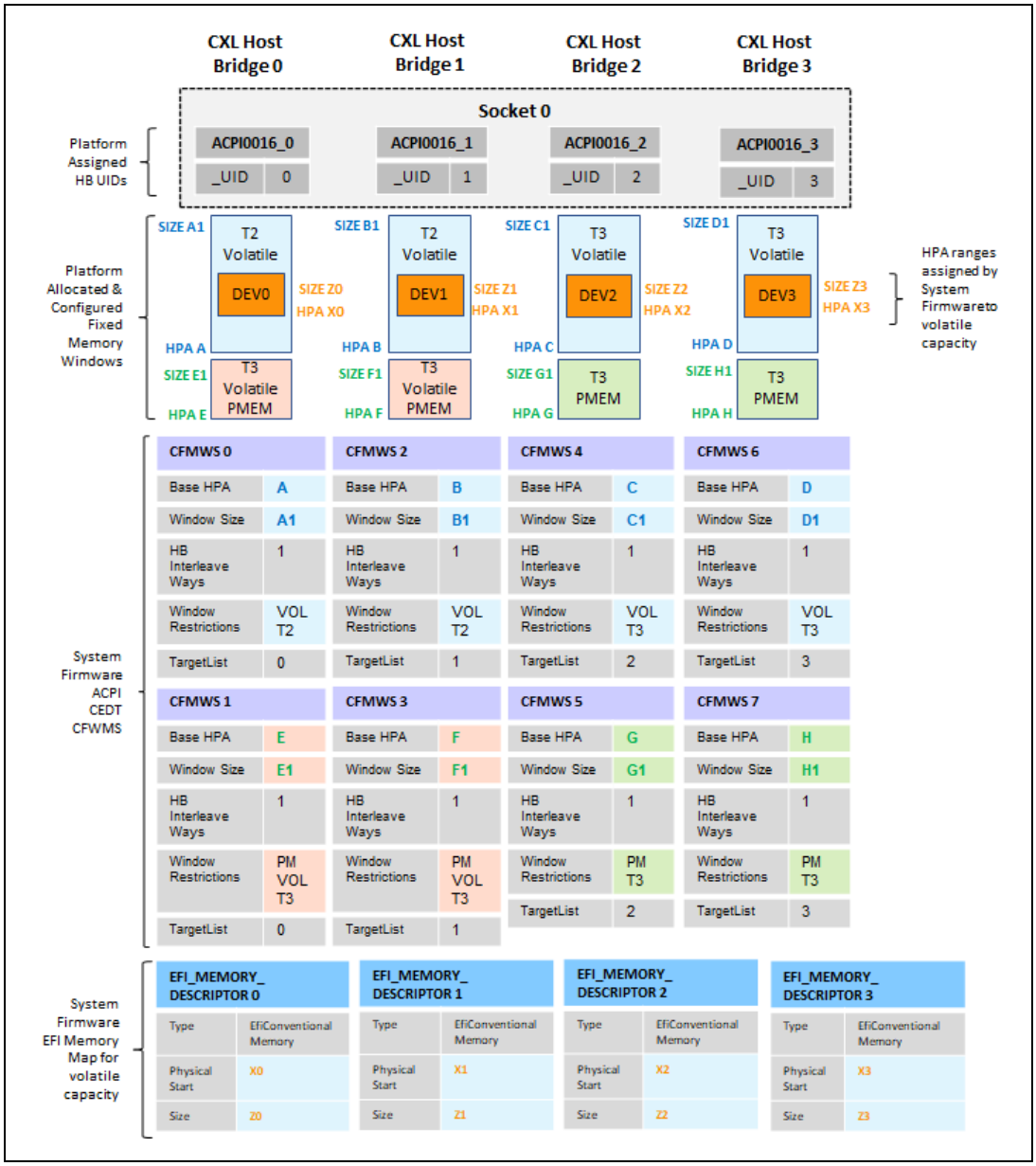
The next examples outline the intended content of both the EFI_MEMORY_MAP and CFMWS, for various CXL topologies.

2.6.2 CEDT CFMWS Example – No XHB Interleaving

The following example demonstrates how the System Firmware might set up the fixed memory windows for each CXL Host Bridge and the resulting CEDT CFMWS.

In this example there is no XHB interleaving.

Figure 2-5 CEDT CFMWS Example - No XHB Interleaving

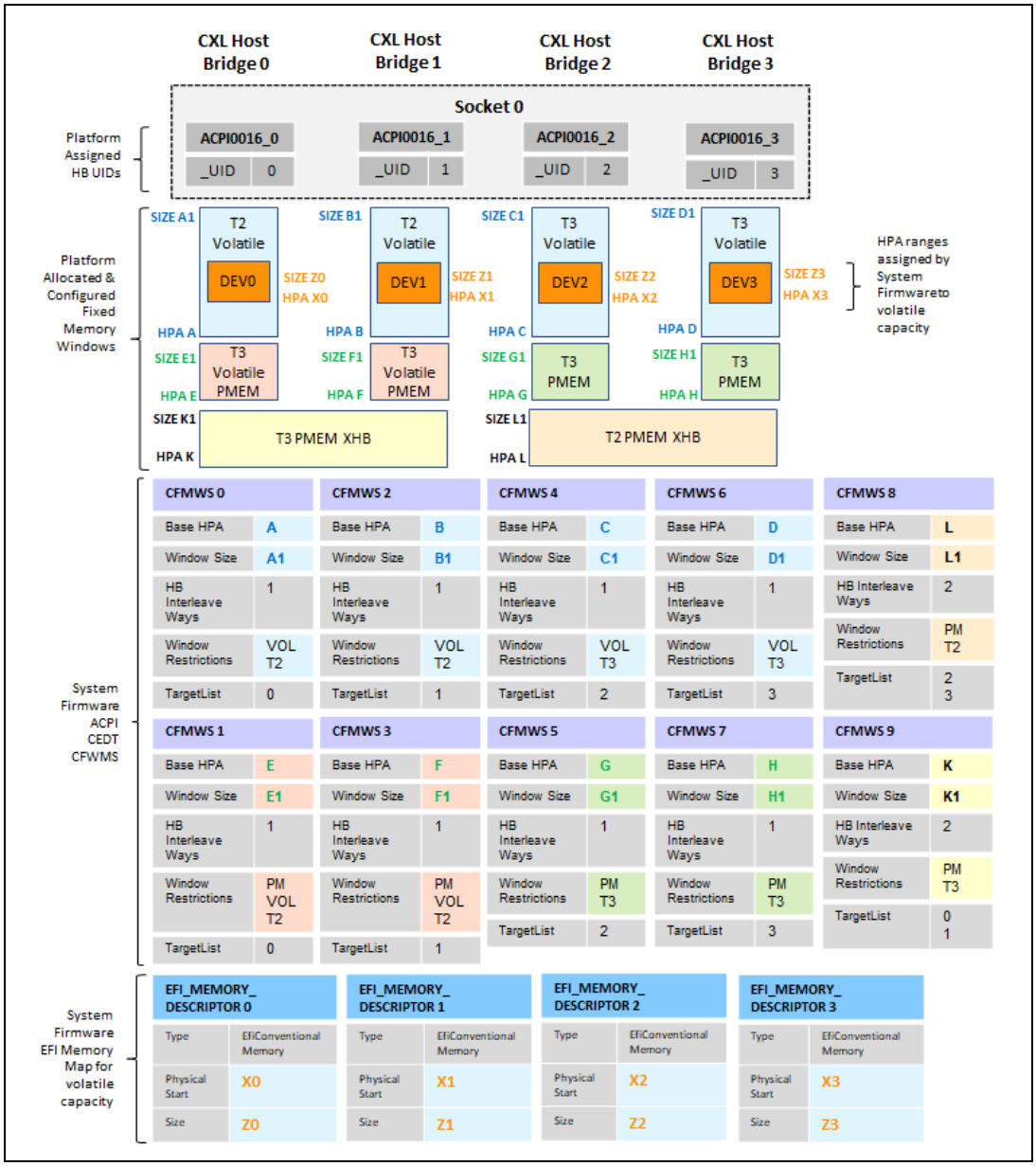


2.6.3 CEDT CFMWS Example – x2 XHB Interleaving

The following example demonstrates how the System Firmware might set up the fixed memory windows for each CXL Host Bridge and the resulting CEDT CFMWS.

In this example, the platform supports a x2 Persistent XHB interleave between CXL Host Bridges 0 and 1 and a x2 Persistent XHB interleave between CXL Host Bridges 2 and 3.

Figure 2-6 CEDT CFMWS Example - x2 XHB Interleave

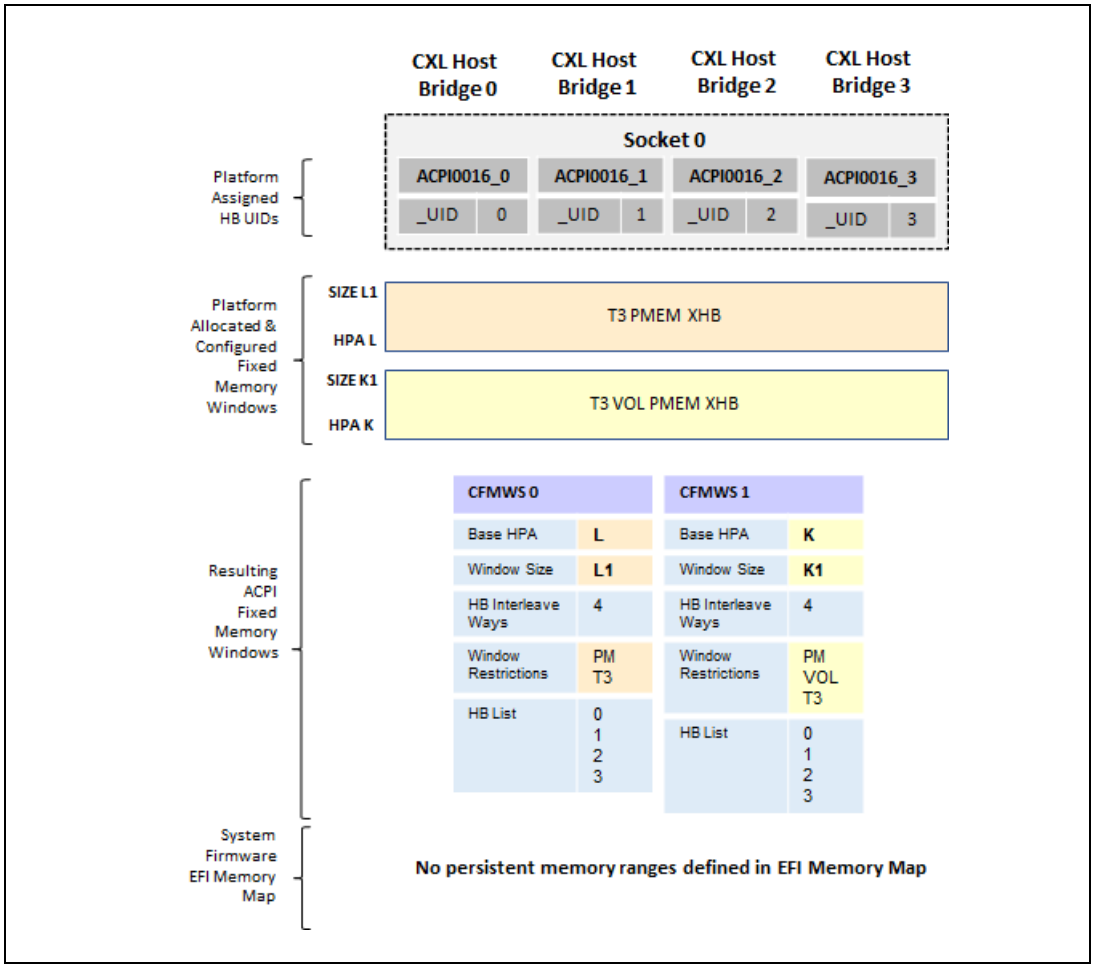


2.6.4 CEDT CFMWS Example – x4 XHB Interleaving

The following example demonstrates how the System Firmware might set up the fixed memory windows for each CXL Host Bridge and the resulting CEDT CFMWS.

In this example, the platform supports a x4 Type 3 persistent XHB interleave between CXL Host Bridges 0, 1, 2 and 3 and a x4 Type 3 volatile or persistent XHB interleave between CXL Host Bridges 0, 1, 2 and 3. Note that in this example, the EFI MEMORY MAP contains no persistent memory descriptors, but it is assumed there would be other volatile memory descriptors, not shown.

Figure 2-7 CEDT CFMWS Example - x4 XHB Interleave

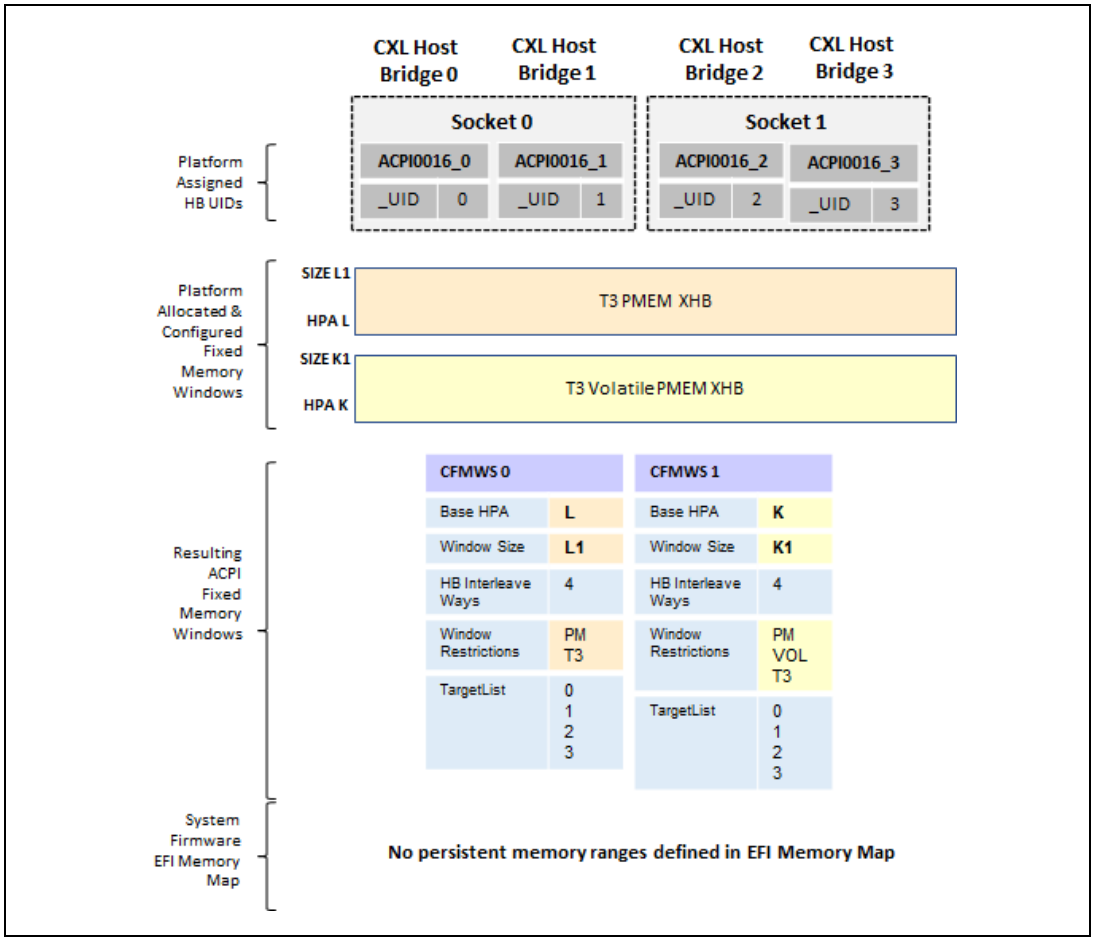


2.6.5 CEDT CFMWS Example – x4 XHB Interleaving Across Multiple Sockets

The following example demonstrates how the System Firmware might set up the fixed memory windows for each CXL Host Bridge and the resulting CEDT CFMWS.

In this example, the platform supports a x4 Type 3 persistent XHB interleave can send between CXL Host Bridges 0, 1, 2 and 3 and a x4 Type 3 volatile or persistent XHB interleave between CXL Host Bridges 0, 1, 2 and 3. CXL Host Bridges 0 and 1 are in socket 0 and CXL Host Bridges 3 and 4 are in socket 1. Note that in this example, the EFI MEMORY MAP contains no persistent memory descriptors, but it is assumed there would be other volatile memory descriptors, not shown.

Figure 2-8 CEDT CFMWS Example - x4 XHB Interleave Across Multiple Sockets



Since the CXL Fixed Memory Windows surfaced in the CEDT are based on the CXL Host Bridge _UIDs, which must be unique across the platform, the resulting windows are identical to the previous example. Thus, the organization of CXL Host Bridges into sockets from a hardware perspective, imposes no restrictions on which CXL Host Bridges can be utilized in an XHB interleave. The platform is free to choose which CXL Host Bridges it will allow to be interleaved together without having to describe the hardware restrictions to the UEFI and OS drivers.

2.7 Managing Persistent Regions

The next text outlines the responsibilities of the System Firmware, UEFI and OS drivers to manage persistent regions across the CXL fabric.

HDM decoders

- Contain the HPA Base and Size programmed by the System Firmware, UEFI and OS drivers at CXL hierarchy enumeration time.

- There is no DPA programming in the HDM decoders. The DPA is inferred by the device based on the HDM decoder programming.
- In this architecture the System Firmware should utilize the "lock on Commit" feature to lock programmed HDMs for volatile configurations that utilize any CFMWS with the CFMWS.WindowRestritions.FixedDeviceConfiguration set.
- UEFI and OS drivers should not utilize the "lock on Commit" for the HDM decoders so the UEFI and OS driver are free to re-program HDMs for any device, at any time, governed by the CFMWS Windows Restrictions and the QoS Throttling Group.
- The *CXL Specification* requires the memory device Get Partition Info reported volatile capacity must start with DPA 0. The volatile or persistent capacity also does not need to be described by a single decoder; however, all the volatile capacity must be programmed first. If there is no volatile capacity, the first device decoder can be used for persistent memory ranges.
- These rules force the System Firmware, UEFI and OS drivers to program the lowest HDM decoder with volatile capacity first, and only after all volatile DPA range has been accounted for (either programmed or skipped) can the System Firmware, UEFI and OS drivers program any persistent memory ranges in the remaining HDM decoders.

System Firmware Region Management Responsibilities

- The System Firmware reports the base address to the CXL Host Bridge MMIO Register Block (CHBCR) in the CEDT for each CXL Host Bridge discovered by the System Firmware, at platform boot time.
- The System Firmware creates a separate ACPI0016 device instance in the ACPI device tree for every CXL Host Bridge discovered by the System Firmware, at platform boot time.
- The System Firmware produces the CEDT ACPI table which should contain a CHBS instance for each host bridge present at boot and one or more CFMWS instances to describe the platform resources available to the UEFI and OS CXL bus driver.

UEFI and OS Region Management Responsibilities

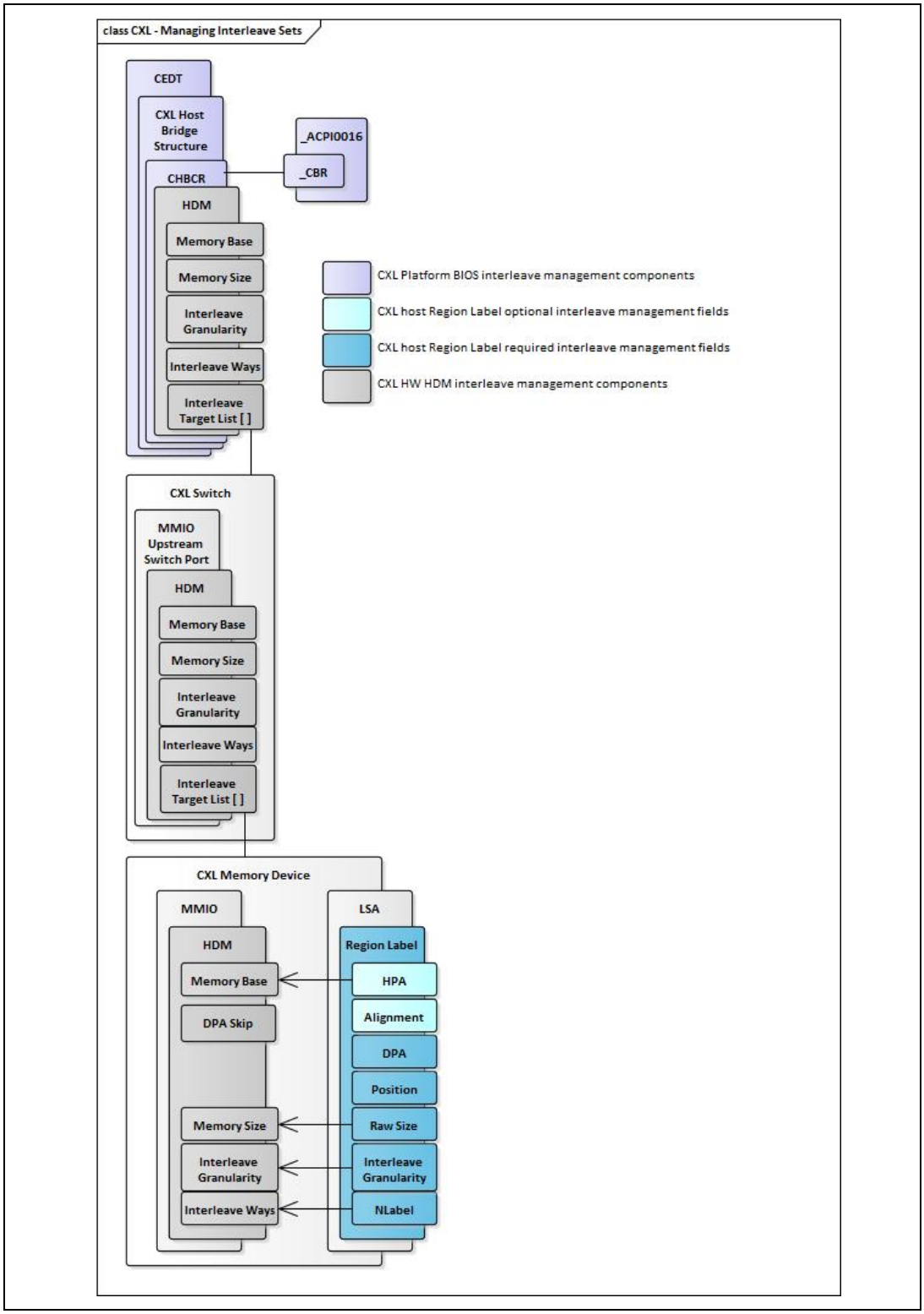
- UEFI and OS drivers enumerate the entire CXL hierarchy of CXL memory devices and CXL switches following the PCIe/CXL specified bus enumeration sequence.
- For each CXL memory device found UEFI and OS drivers:
  - Retrieve device HDM decoder capabilities.
  - OS drivers only: Program device HDM decoder for volatile memory not configured by System Firmware.
  - Program device HDM decoders for persistent memory. Each region label on the device will require a separate HDM decoder to be programmed on the device. The UEFI driver will only need to

program HDMs for persistent memory capacity that is part of the persistent memory boot path.

- For each CXL Switch UEFI and OS drivers:
  - Programs the CXL Switch's Upstream Port matching HDM decoder Memory Base, Memory Size, Interleave Granularity, Interleave Ways based on the summation of all the downstream devices and switches (relative to this Host Bridge) HDM decoder programming.
  - Based on the Pos field in the region label on each device in the region, program the CXL Switch's Upstream Port matching HDM decoder Target List. The order of the CXL Downstream Switch Ports in the target list must match the configured ordering of each device in the region to maintain consistent region data.
- For each CXL Host Bridge the UEFI and OS drivers:
  - Program the CXL Host Bridge matching HDM decoder Memory Base, Memory Size, Interleave Granularity, Interleave Ways based on the summation of all the downstream devices and switches (relative to this HB) HDM decoder programming.
  - Based on the Pos field in the region label on each device in the region, program the CXL Host Bridge’s root port matching HDM decoder Target List. The order of the CXL Root Ports in the target list must match the configured ordering of each device in the region to maintain consistent region data.
- For UEFI and OS drivers that update the interleave set/region configuration data as part of managing the interleave sets:
  - In addition to the previous responsibilities, UEFI and OS drivers that update the region labels must follow the power fail safe label update rules as specified in the *CXL Specification*.
  - After updating region configuration information on the device, UEFI and OS drivers are responsible for re-programming the CXL HDM decoders for all effected switch ports and CXL Host Bridges, as outlined previously.

The following figure demonstrates the components involved in managing regions.

Figure 2-9 Components for Managing Regions

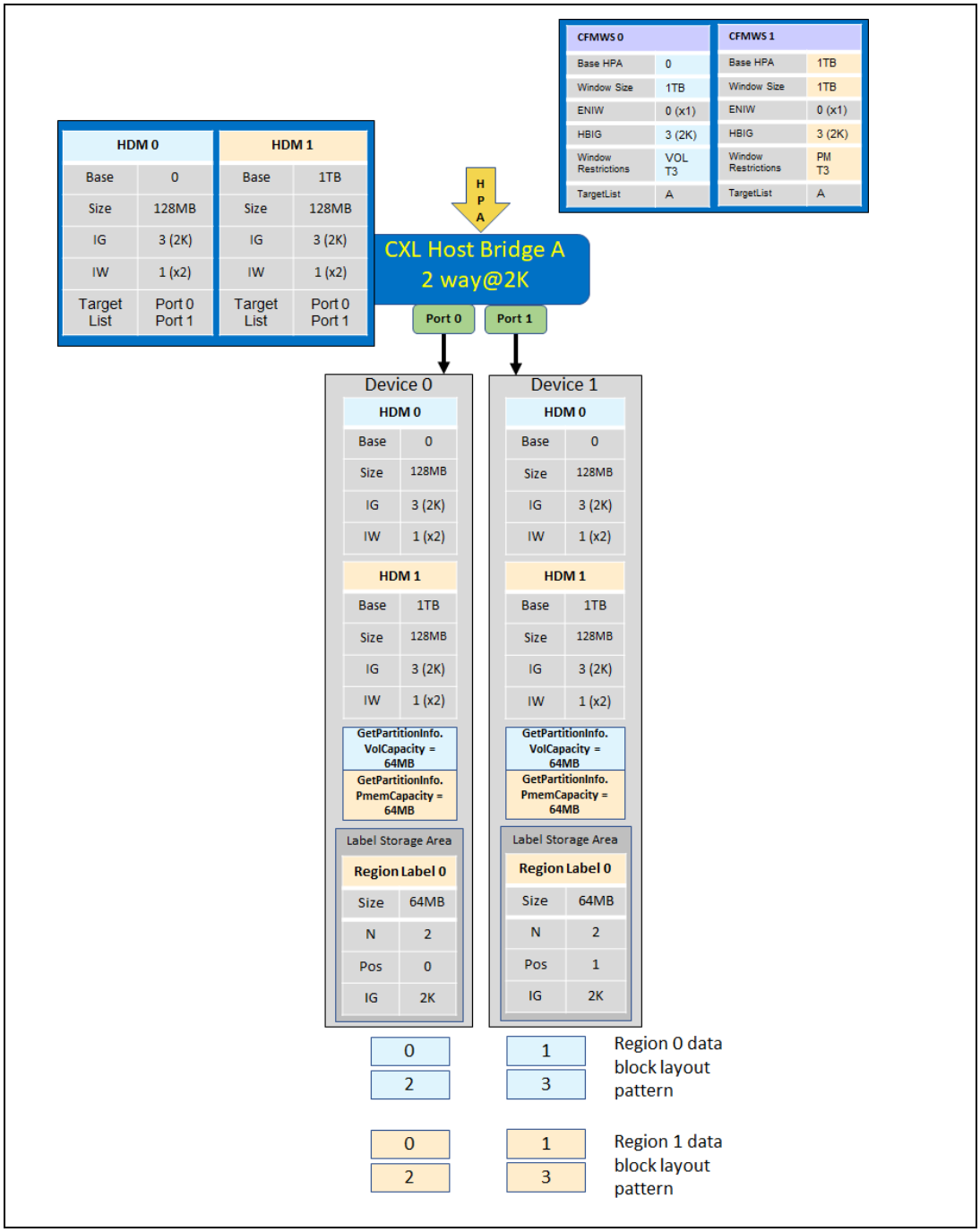


2.7.1 Example – Volatile and Persistent x2 Interleaved Regions

This example demonstrates contents of the region label retrieved from each device's LSA, the System Firmware CFMWS, and how the System Firmware, UEFI and OS drivers would program the HDM decoders in each memory device, each switch, and each HB to implement the desired interleave.

In this example, 2 devices are interleaved together with volatile capacity and the same 2 devices are interleaved together with persistent capacity and each device contributes 64MB of volatile and persistent capacity, for simplicity.

Figure 2-10 Example - Volatile and Persistent x2 Interleaved Regions



Here is example Linux SYSFS hierarchy demonstrating how this configuration would be interpreted by the CXL Subsystem component in the Linux architecture.

Note: This is provisional/draft output and will be updated with more accurate text in another release.

```
/sys/bus/cxl/devices/root0
├── address_space0                                    // CFMWS0
│   ├── devtype                                       // cxl_address_space
│   ├── end                                           // 1TB
│   ├── start                                         // 0
│   ├── supports_ram
│   ├── supports_type3
│   └── uevent
├── address_space1                                    // CFMWS1
│   ├── devtype                                       // cxl_address_space
│   ├── end                                           // 1TB
│   ├── start                                         // 1TB
│   ├── supports_pmem
│   ├── supports_type3
│   └── uevent
...
├── devtype                                           // cxl_root
├── dport0 -> ../LNXSYSTM:00/LNXSYBUS:00/ACPI0016:00  // Host Bridge A
├── port1                                             // Port 0
│   ├── decoder1.0                                    // HDM 0
│   │   ├── devtype
│   │   ├── end
│   │   ├── locked
│   │   ├── start
│   │   ├── subsystem -> ../../../../bus/cxl
│   │   ├── target_list
│   │   ├── target_type
│   │   └── uevent
│   ├── devtype                                       // cxl_port
│   ├── dport0 -> ../../pci0000:34/0000:34:00.0       // Port 0 Address
│   ├── subsystem -> ../../../bus/cxl
│   ├── uevent
│   └── uport -> ../../LNXSYSTM:00/LNXSYBUS:00/ACPI0016:00  // HB assignment
├── subsystem -> ../../bus/cxl
├── uevent
└── uport -> ../platform/ACPI0017:00                  // HB parent
```

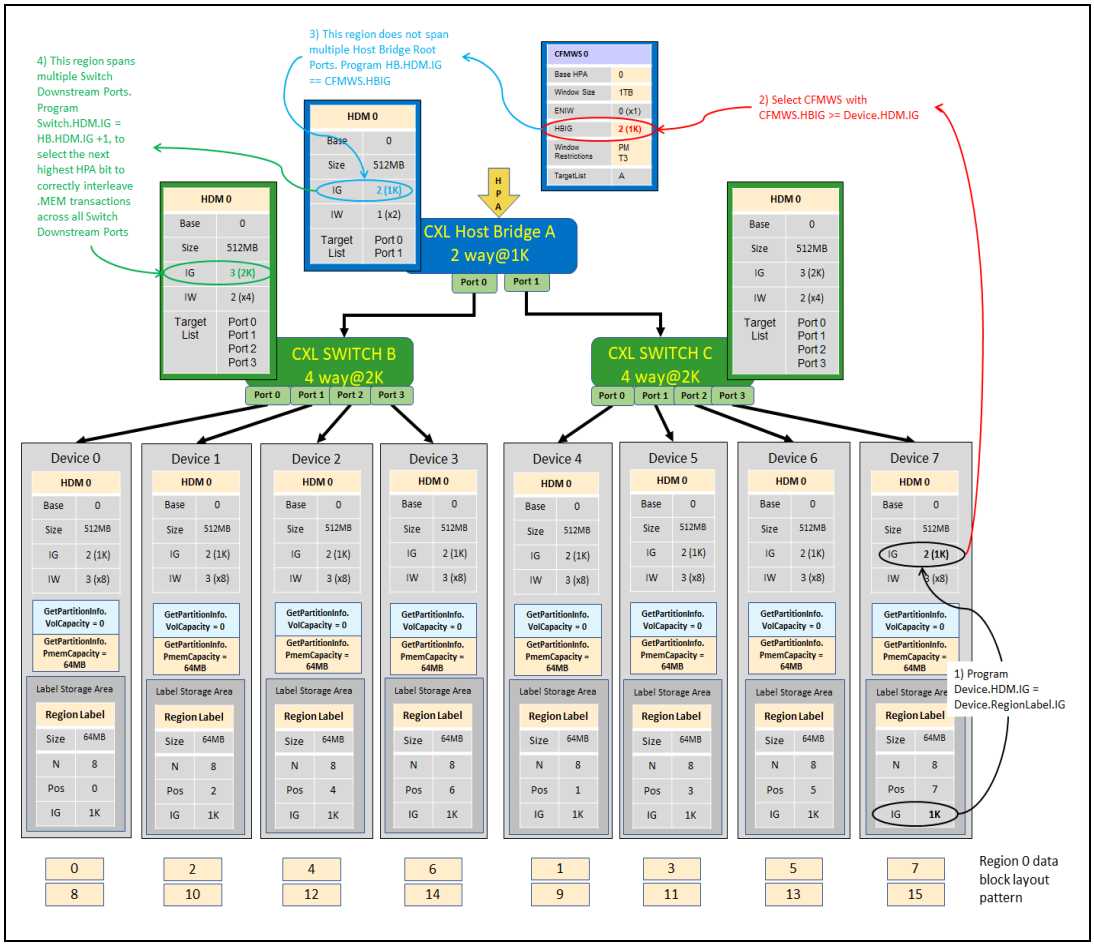
2.7.2 Example – Region Interleaved Across 2 CXL Switches

This example demonstrates contents of the region label retrieved from each device's LSA, the System Firmware CFMWS, and how the System Firmware, UEFI and OS drivers would program the HDM decoders in each memory device, each switch, and each HB to implement the desired interleave.

In this example, all 8 devices are interleaved together into a single region and each device contributes 64MB of persistent capacity, for simplicity.

Note how the Switch interleave granularity is programmed to 2K and the Host Bridge interleave granularity is programmed to 1K, so that each switch utilizes HPA[12:11] and the Host Bridge utilizes HPA[10] to select the target port. This allows proper distribution of the HPA in a round robin fashion across all the ports in each HostBridge.HDM.InterleaveTargetList[ ] and Switch.HDM.InterleaveTargetList[ ] the region is associated with. This leads to good performance since maximum distribution of memory requests across all HBs and switches is achieved, as shown in the resulting region data block layout pattern at the bottom of the figure.

Figure 2-11 Example - Region Interleaved Across 2 Switches



Here is example Linux SYSFS hierarchy demonstrating how this configuration would be interpreted by the CXL Subsystem component in the Linux architecture.

```
# cxl list -BDPTMEu -b cxl_test
{
  "bus":"root3",
  "provider":"cxl_test",
  "nr_dports":1,
  "dports":[
    {
      "dport":"cxl_host_bridge.0",
      "alias":"cxl_host_bridge.0",
      "id":"0"
    }
  ],
  "ports:root3":[
    {
      "port":"port4",
      "host":"cxl_host_bridge.0",
```

```
"nr_dports":2,
"dports":[
  {
    "dport":"cxl_root_port.0",
    "id":"0"
  },
  {
    "dport":"cxl_root_port.1",
    "id":"0x1"
  }
],
"ports:port4":[
  {
    "port":"port5",
    "host":"cxl_switch_uport.0",
    "nr_dports":4,
    "dports":[
      {
        "dport":"cxl_switch_dport.0",
        "id":"0"
      },
      {
        "dport":"cxl_switch_dport.2",
        "id":"0x2"
      },
      {
        "dport":"cxl_switch_dport.4",
        "id":"0x4"
      },
      {
        "dport":"cxl_switch_dport.6",
        "id":"0x6"
      }
    ],
    "endpoints:port5":[
      {
        "endpoint":"endpoint9",
        "host":"mem3",
        "memdev":{
          "memdev":"mem3",
          "pmem_size":"256.00 MiB (268.44 MB)",
          "ram_size":"256.00 MiB (268.44 MB)",
          "serial":"0x2",
          "numa_node":0,
          "host":"cxl_mem.2"
        }
      },
      {
```

```
        "endpoint":"endpoint6",
        "host":"mem1",
        "memdev":{
          "memdev":"mem1",
          "pmem_size":"256.00 MiB (268.44 MB)",
          "ram_size":"256.00 MiB (268.44 MB)",
          "serial":"0",
          "numa_node":0,
          "host":"cxl_mem.0"
        }
      },
      {
        "endpoint":"endpoint11",
        "host":"mem5",
        "memdev":{
          "memdev":"mem5",
          "pmem_size":"256.00 MiB (268.44 MB)",
          "ram_size":"256.00 MiB (268.44 MB)",
          "serial":"0x4",
          "numa_node":0,
          "host":"cxl_mem.4"
        }
      },
      {
        "endpoint":"endpoint13",
        "host":"mem7",
        "memdev":{
          "memdev":"mem7",
          "pmem_size":"256.00 MiB (268.44 MB)",
          "ram_size":"256.00 MiB (268.44 MB)",
          "serial":"0x6",
          "numa_node":0,
          "host":"cxl_mem.6"
        }
      }
    ]
  },
  {
    "port":"port7",
    "host":"cxl_switch_uport.1",
    "nr_dports":4,
    "dports":[
      {
        "dport":"cxl_switch_dport.7",
        "id":"0x7"
      },
      {
        "dport":"cxl_switch_dport.1",
```

```
      "id":"0x1"
    },
    {
      "dport":"cxl_switch_dport.3",
      "id":"0x3"
    },
    {
      "dport":"cxl_switch_dport.5",
      "id":"0x5"
    }
  ],
  "endpoints:port7":[
    {
      "endpoint":"endpoint10",
      "host":"mem4",
      "memdev":{
        "memdev":"mem4",
        "pmem_size":"256.00 MiB (268.44 MB)",
        "ram_size":"256.00 MiB (268.44 MB)",
        "serial":"0x3",
        "numa_node":1,
        "host":"cxl_mem.3"
      }
    },
    {
      "endpoint":"endpoint12",
      "host":"mem6",
      "memdev":{
        "memdev":"mem6",
        "pmem_size":"256.00 MiB (268.44 MB)",
        "ram_size":"256.00 MiB (268.44 MB)",
        "serial":"0x5",
        "numa_node":1,
        "host":"cxl_mem.5"
      }
    },
    {
      "endpoint":"endpoint14",
      "host":"mem8",
      "memdev":{
        "memdev":"mem8",
        "pmem_size":"256.00 MiB (268.44 MB)",
        "ram_size":"256.00 MiB (268.44 MB)",
        "serial":"0x7",
        "numa_node":1,
        "host":"cxl_mem.7"
      }
    },
```

```
            {
              "endpoint":"endpoint8",
              "host":"mem2",
              "memdev":{
                "memdev":"mem2",
                "pmem_size":"256.00 MiB (268.44 MB)",
                "ram_size":"256.00 MiB (268.44 MB)",
                "serial":"0x1",
                "numa_node":1,
                "host":"cxl_mem.1"
              }
            }
          ]
        }
      ]
    }
  ],
  "decoders:root3":[
    {
      "decoder":"decoder3.1",
      "resource":"0x8020000000",
      "size":"256.00 MiB (268.44 MB)",
      "volatile_capable":true,
      "nr_targets":1,
      "targets":[
        {
          "target":"cxl_host_bridge.0",
          "alias":"cxl_host_bridge.0",
          "position":0,
          "id":"0"
        }
      ]
    },
    {
      "decoder":"decoder3.3",
      "resource":"0x8040000000",
      "size":"256.00 MiB (268.44 MB)",
      "pmem_capable":true,
      "nr_targets":1,
      "targets":[
        {
          "target":"cxl_host_bridge.0",
          "alias":"cxl_host_bridge.0",
          "position":0,
          "id":"0"
        }
      ]
    },
```

```
    {
      "decoder":"decoder3.0",
      "resource":"0x8010000000",
      "size":"256.00 MiB (268.44 MB)",
      "volatile_capable":true,
      "nr_targets":1,
      "targets":[
        {
          "target":"cxl_host_bridge.0",
          "alias":"cxl_host_bridge.0",
          "position":0,
          "id":"0"
        }
      ]
    },
    {
      "decoder":"decoder3.2",
      "resource":"0x8030000000",
      "size":"256.00 MiB (268.44 MB)",
      "pmem_capable":true,
      "nr_targets":1,
      "targets":[
        {
          "target":"cxl_host_bridge.0",
          "alias":"cxl_host_bridge.0",
          "position":0,
          "id":"0"
        }
      ]
    }
  ]
}
```


2.7.3 Example – Region Interleaved Across 2 HBs

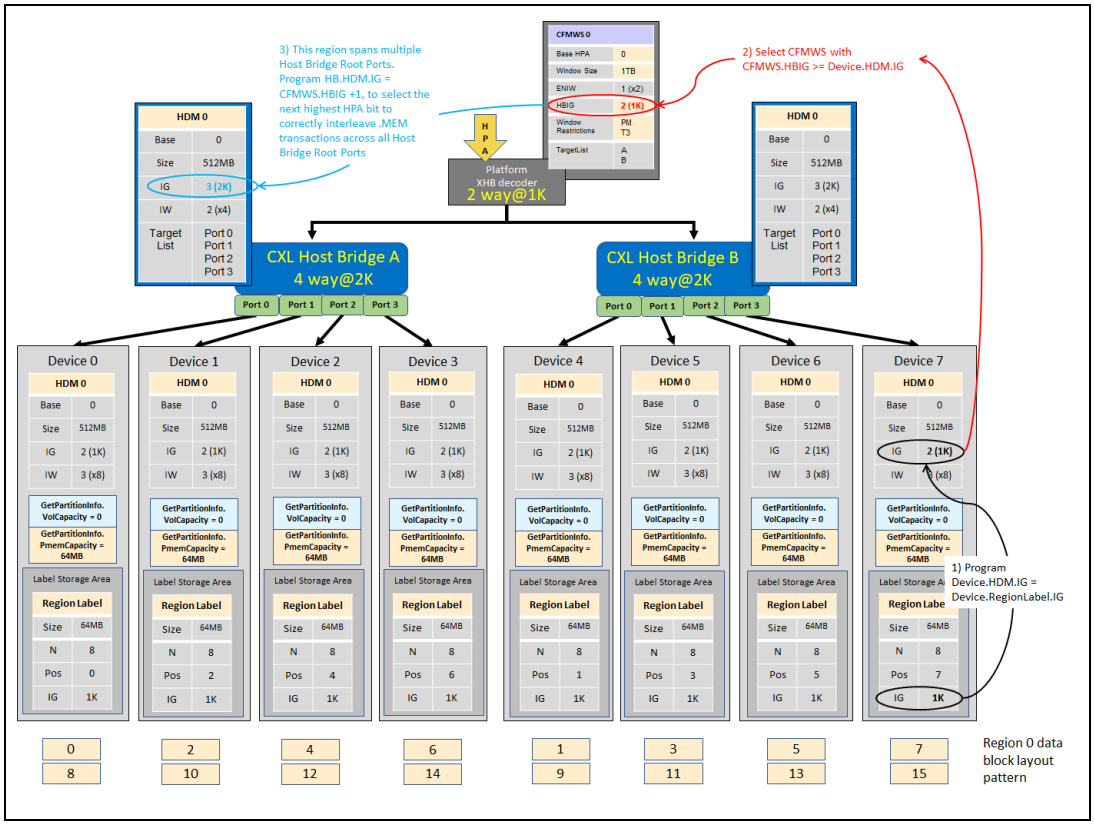
This example demonstrates contents of the region label retrieved from each device's LSA, the System Firmware CFMWS, and how the System Firmware, UEFI and OS drivers would program the HDM decoders in each memory device, and each HB to implement the desired interleave.

In this example, all 8 devices are interleaved together into a single region that spans multiple CXL Host Bridges and each device contributes 64 MB bytes of persistent capacity, for simplicity.

Note how the Host Bridge interleave granularity is programmed to 2K and the platform XHB interleave granularity is pre-programmed to 1K so that the Host Bridge utilizes HPA[11] to select the root port and the platform XHB interleave granularity utilizes HPA[10] to select the Host Bridge. This allows proper distribution of the HPA in a round robin fashion across all the Host Bridges in

the platform XHB CFMWS.InterleaveTargetList[ ] and each root port in the HostBridge.HDM.InterleaveTargetList[ ] the region is associated with. This leads to good performance since maximum distribution of memory requests across all HBs and switches is achieved, as shown in the resulting region data block layout pattern at the bottom of the figure.

Figure 2-12 Example - Region Interleaved Across 2 HBs



Here is an example Linux SYSFS output demonstrating how this configuration would be interpreted by the CXL Subsystem component in the Linux architecture.

Note: This is provisional/draft output and will be updated with more accurate text in another release.

```
/sys/bus/cxl/devices/root0
├── address_space0                                      // CFMWS0
│   ├── devtype                                         // cxl_address_space
│   ├── end                                             // 1TB
│   ├── start                                           // 0
│   ├── supports_pmem
│   ├── supports_type3
│   └── uevent
...
├── devtype                                             // cxl_root
├── dport0 -> ../LNXSYSTM:00/LNXSYBUS:00/ACPI0016:00    // Host Bridge A
├── port1                                               // Port 0
│   ├── decoder1.0                                      // HDM 0
│   ├── devtype                                         // cxl_port
│   ├── dport0 -> ../../pci0000:34/0000:34:00.0         // Port 0 Address
│   ├── subsystem -> ../../../bus/cxl
│   ├── uevent
│   └── uport -> ../../LNXSYSTM:00/LNXSYBUS:00/ACPI0016:00  // HB assignment
├── port2                                               // Port 1
```

```
│  ├── decoder1.0                                          // HDM 0
│  ├── devtype                                             // cxl_port
│  ├── dport0 -> ../../pci0000:35/0000:35:00.0             // Port 1 Address
│  ├── subsystem -> ../../../bus/cxl
│  ├── uevent
│  └── uport -> ../../LNXSYSTM:00/LNXSYBUS:00/ACPI0016:00  // HB assignment
├── port3                                                  // Port 2
│  ├── decoder1.0                                          // HDM 0
│  ├── devtype                                             // cxl_port
│  ├── dport0 -> ../../pci0000:36/0000:36:00.0             // Port 2 Address
│  ├── subsystem -> ../../../bus/cxl
│  ├── uevent
│  └── uport -> ../../LNXSYSTM:00/LNXSYBUS:00/ACPI0016:00  // HB assignment
├── port4                                                  // Port 3
│  ├── decoder1.0                                          // HDM 0
│  ├── devtype                                             // cxl_port
│  ├── dport0 -> ../../pci0000:37/0000:37:00.0             // Port 2 Address
│  ├── subsystem -> ../../../bus/cxl
│  ├── uevent
│  └── uport -> ../../LNXSYSTM:00/LNXSYBUS:00/ACPI0016:00  // HB assignment
├── subsystem -> ../../bus/cxl
├── uevent
├── uport -> ../platform/ACPI0017:00                       // HB parent
├── devtype                                                // cxl_root
├── dport5 -> ../LNXSYSTM:00/LNXSYBUS:00/ACPI0016:01       // Host Bridge B
├── port6                                                  // Port 0
│  ├── decoder1.0                                          // HDM 0
│  ├── devtype                                             // cxl_port
│  ├── dport0 -> ../../pci0000:38/0000:38:00.0             // Port 0 Address
│  ├── subsystem -> ../../../bus/cxl
│  ├── uevent
│  └── uport -> ../../LNXSYSTM:00/LNXSYBUS:00/ACPI0016:00  // HB assignment
├── port7                                                  // Port 1
│  ├── decoder1.0                                          // HDM 0
│  ├── devtype                                             // cxl_port
│  ├── dport0 -> ../../pci0000:39/0000:39:00.0             // Port 1 Address
│  ├── subsystem -> ../../../bus/cxl
│  ├── uevent
│  └── uport -> ../../LNXSYSTM:00/LNXSYBUS:00/ACPI0016:00  // HB assignment
├── port8                                                  // Port 2
│  ├── decoder1.0                                          // HDM 0
│  ├── devtype                                             // cxl_port
│  ├── dport0 -> ../../pci0000:3A/0000:3A:00.0             // Port 2 Address
│  ├── subsystem -> ../../../bus/cxl
│  ├── uevent
│  └── uport -> ../../LNXSYSTM:00/LNXSYBUS:00/ACPI0016:00  // HB assignment
├── port9                                                  // Port 3
│  ├── decoder1.0                                          // HDM 0
│  ├── devtype                                             // cxl_port
│  ├── dport0 -> ../../pci0000:3B/0000:3B:00.0             // Port 2 Address
│  ├── subsystem -> ../../../bus/cxl
│  ├── uevent
│  └── uport -> ../../LNXSYSTM:00/LNXSYBUS:00/ACPI0016:00  // HB assignment
├── subsystem -> ../../bus/cxl
├── uevent
└── uport -> ../platform/ACPI0017:00                       // HB parent
```

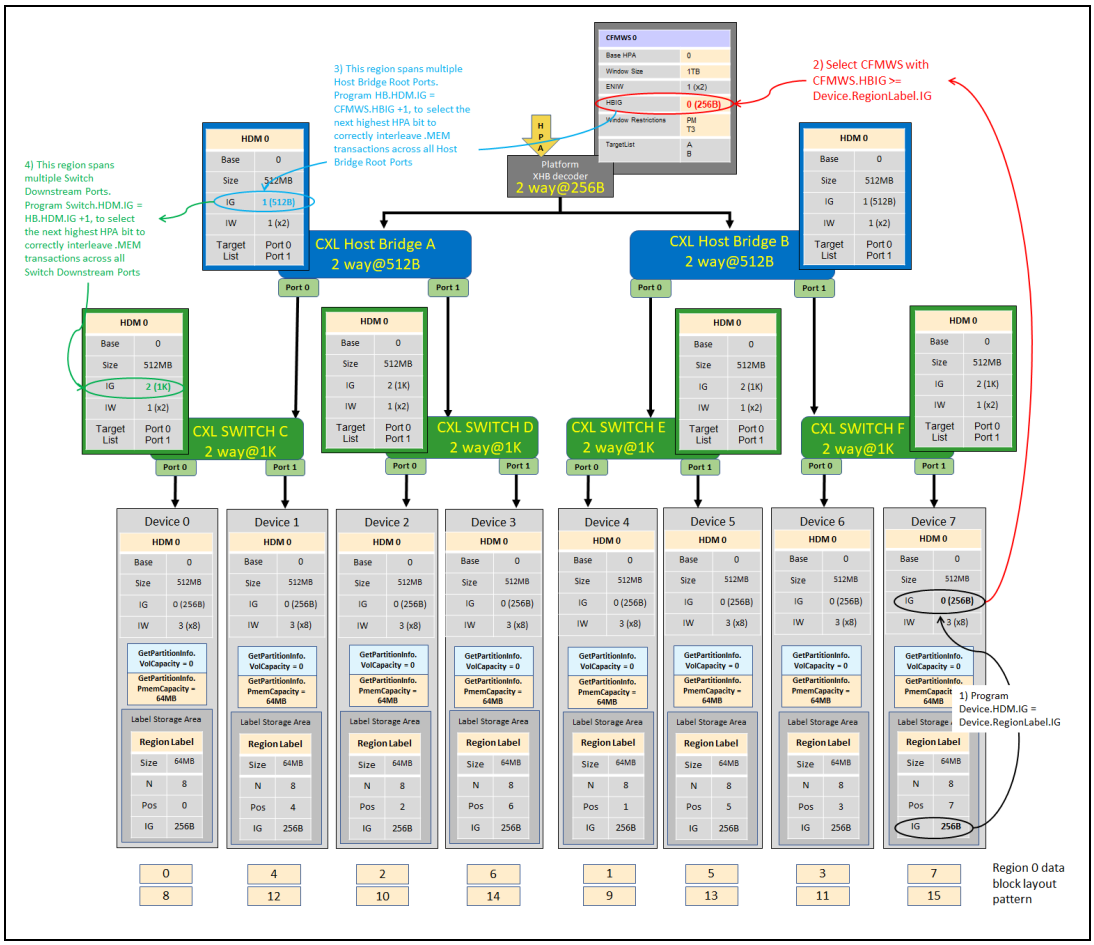
2.7.4 Example – Region Interleaved Across 2 HBs and 4 Switches

This example demonstrates contents of the region label retrieved from each device's LSA, the System Firmware CFMWS, and how the System Firmware, UEFI and OS drivers would program the HDM decoders in each memory device, and each HB to implement the desired interleave.

In this example, all 8 devices are interleaved together into a single region that spans multiple CXL Host Bridges and multiple Switches, and each device contributes 64 MB of persistent capacity, for simplicity.

Note how the switch interleave granularity is programmed to 1K, the Host Bridge interleave granularity is programmed to 512B and the platform XHB interleave granularity is programmed to 256B so that each switch utilizes HPA[10] to select the Downstream Port, each Host Bridge utilizes HPA[9] to select the root port and the platform XHB interleave granularity utilizes HPA[8] to select the Host Bridge. This allows proper distribution of the HPA in a round robin fashion across all of the Host Bridges in the platform XHB CFMWS.InterleaveTargetList, each root port in the HostBridge.HDM.InterleaveTargetList, and each switch port in the Switch.HDM.InterleaveTargetlist[ ] the region is associated with. This leads to good performance since maximum distribution of memory requests across all HBs and switches is achieved, as shown in the resulting region data block layout pattern at the bottom of the figure.

Figure 2-13 Example - Region Interleaved Across 2 HBs and 4 Switches



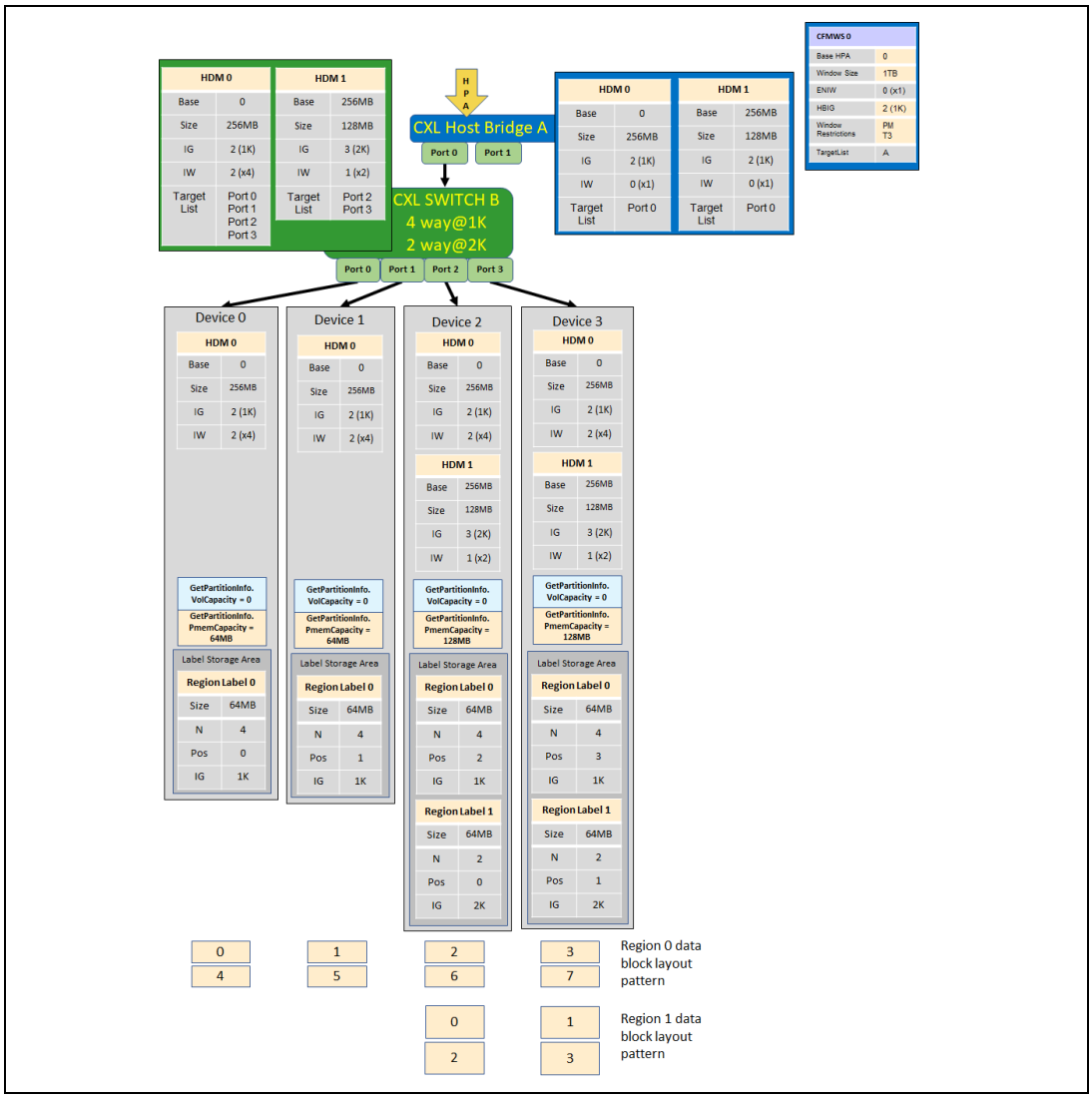
2.7.5 Example – 2 Regions Interleaved Across 2 and 4 Devices

This example demonstrates contents of the region label retrieved from each device's LSA, the System Firmware CFMWS, and how the system firmware,

UEFI and OS drivers would program the HDM decoders in each memory device, each switch, and each HBs to implement the desired interleave.

In this example, 4 devices are interleaved together into region 0, 2 of the same devices are also interleaved together into region 1, and each device contributes 64 MB bytes of persistent capacity, for simplicity. Note that a second set of region labels are utilized to describe the second interleave and that HDM decoder 1 on the device, switches, and the CXL Host Bridge is utilized to program the second HPA range.

Figure 2-14 Example – 2 Regions Interleaved Across 2 and 4 Devices



2.7.6 Example – Out of Order Devices Within a Switch or Host

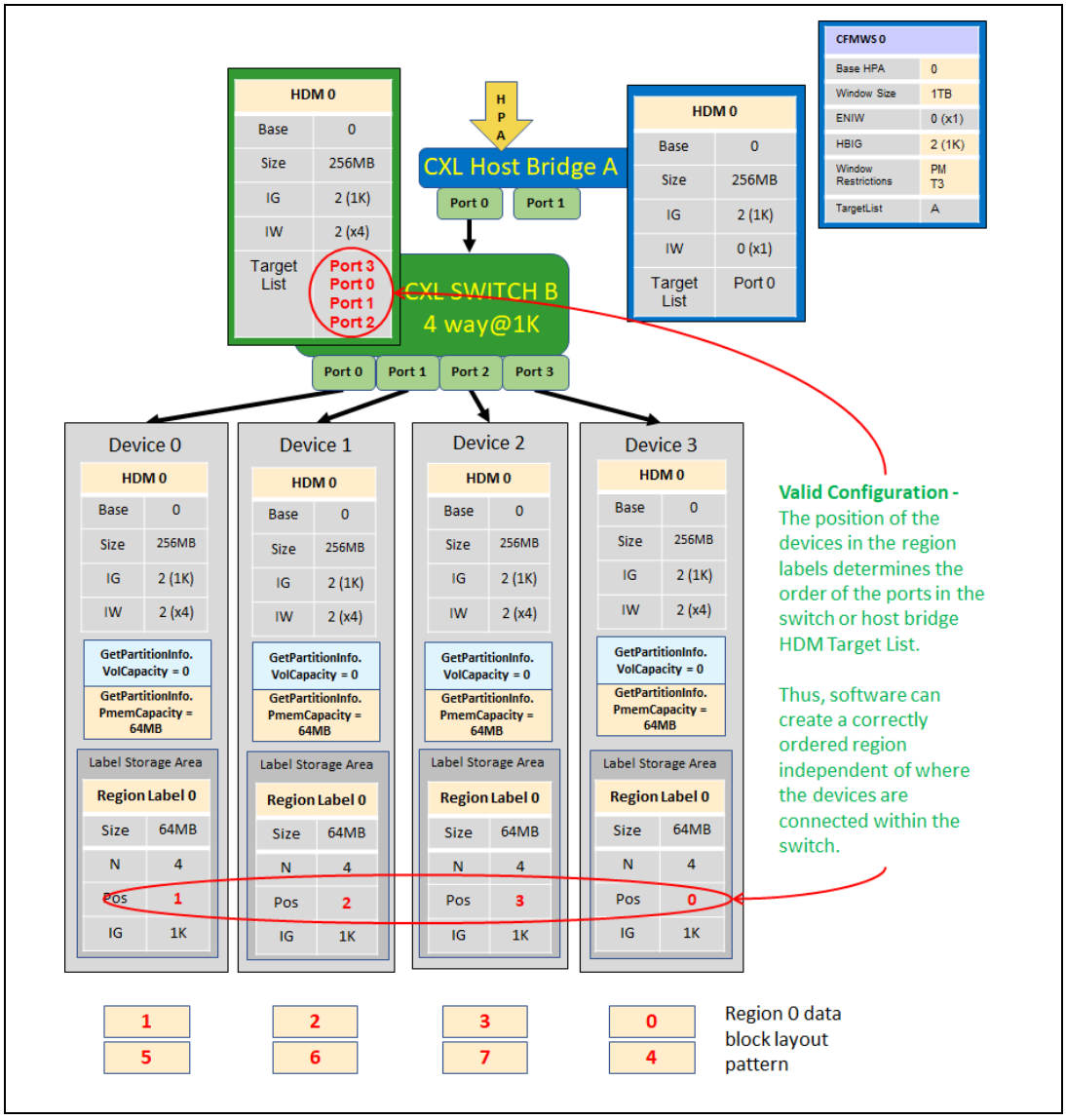
This example demonstrates contents of the region label retrieved from each device's LSA, the system firmware CFMWS, and how the system firmware, UEFI

and OS drivers would program the HDM decoders in each memory device, each switch, and each HBs to implement the desired interleave.

In this example, 4 devices are interleaved together into region 0 on the same CXL host bridge and each device contributes 64 MB bytes of persistent capacity, for simplicity. However, the devices are plugged into the CXL root ports of the CXL switch in a random order compared to the ordering specified by the region label position field found on each device. This could happen if the system the devices were originally plugged into failed and the devices were moved to an identical system for data recovery purposes, but the original ordering was shuffled in the process. Since the CXL switch and CXL host bridge HDMs contains a target list that specifies the order the root ports are interleaved, there is no need to reject this configuration. The UEFI and OS drivers program the CXL switch or CXL host bridge HDMs target list to match the position of the device specified in the region label storage in the device's LSA without the need for a configuration change.

If all the devices are plugged in to the same CXL host bridge, either directly, or through a CXL switch as shown in the example case here, the target list in the CXL switch HDM decoder or CXL host bridge HDM decoder can be programmed to fix the ordering. See the next examples for additional constraints when verifying device ordering across CXL host bridges.

Figure 2-15 Example - Out of Order Devices Within a Switch or Host Bridge



2.7.7 Example – Out of Order Devices Across HBs (Failure Case)

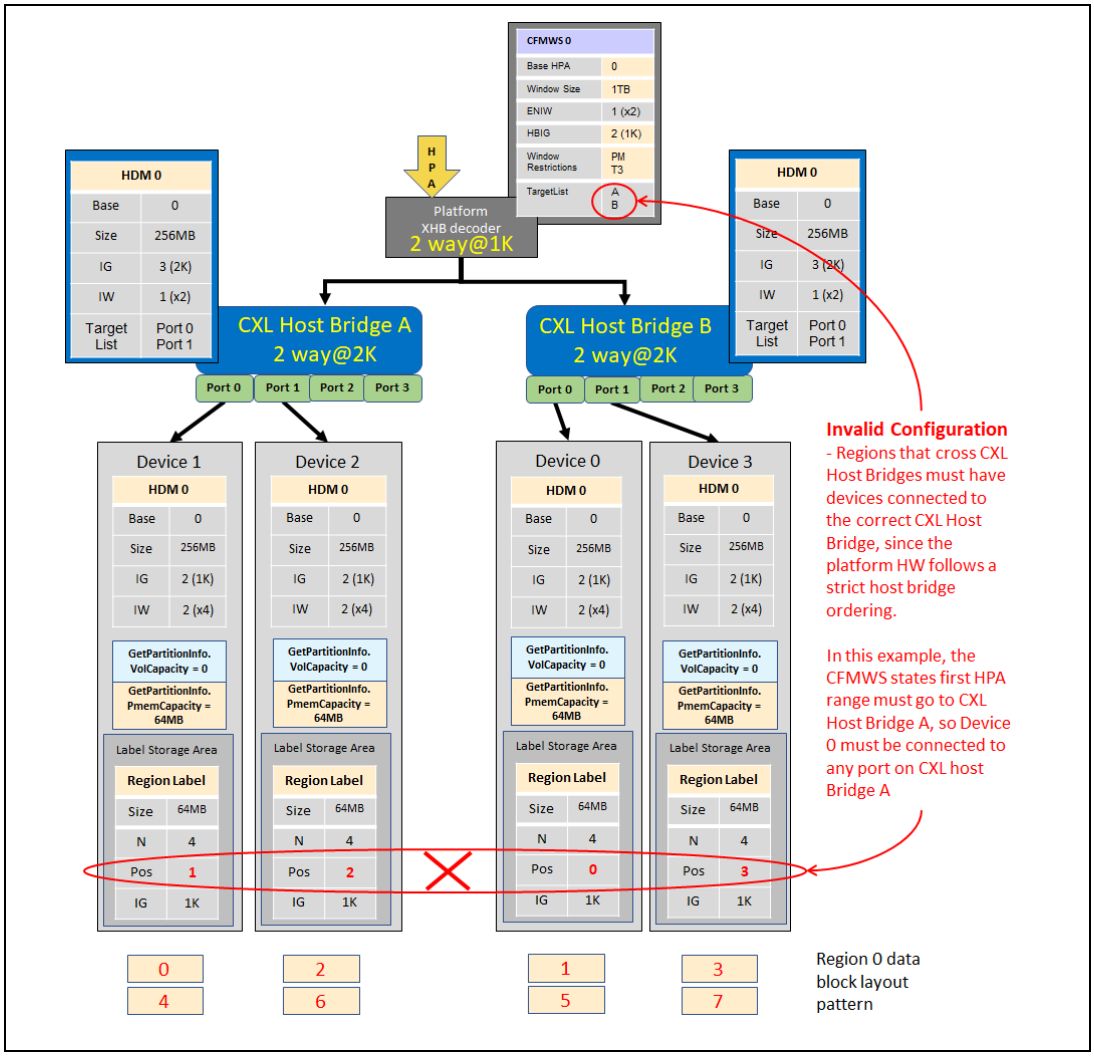
This example demonstrates contents of the region label retrieved from each device's LSA, the system firmware CFMWS, and how the system firmware, UEFI and OS drivers would program the HDM decoders in each memory device, each switch, and each HBs to implement the desired interleave.

In this example, 4 devices are interleaved together into region 0 with 2 devices on one host bridge and 2 devices on the next host bridge using an XHB region. Because platform HW may restrict ordering across host bridges and the OS cannot re-configure it, the ordering of the devices across host bridges now matters and adds an additional responsibility when checking for valid regions.

This example shows an illegal configuration because Device 0 and Device 1 must be on CXL host bridge A and Device 2 and Device 3 must be on CXL host bridge B. The first HPA range will be claimed by CXL host bridge A, so the device in POS 0 must be on CXL host bridge A. The second HPA range will be claimed by CXL host bridge B, so the device in POS 1 must be on CXL host bridge B. The third HPA range will be claimed by CXL host bridge A, so the device in POS 2 must be on A. The fourth HPA range will be claimed by CXL host bridge B, so the device in POS 3 must be on B, and so on.

The suggested algorithm for software to check for proper device connection to each host bridge is outlined in the Section 2.13.14.

Figure 2-16 Example - Out of Order Devices Across HBs (Failure Case)



2.7.8 Example – Verifying Device Position on Each HB Root Port

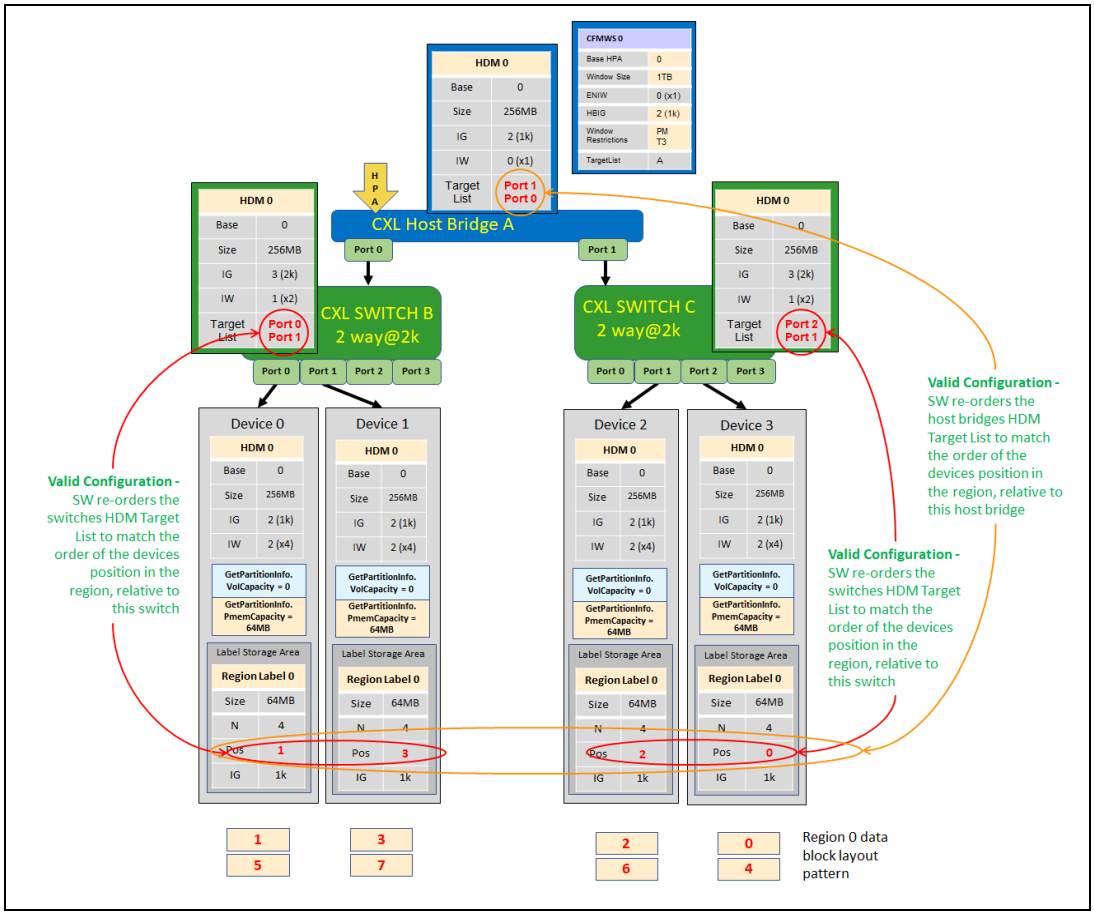
This example demonstrates contents of the region label retrieved from each device's LSA, the system firmware CFMWS, and how the system firmware, UEFI

and OS drivers would program the HDM decoders in each memory device, each switch, and each HBs to implement the desired interleave.

This example demonstrates the verification required to ensure that each devices position in the region lands on the correct root port of the host bridge. While the ordering of root ports in the HB can be changed by reshuffling the HB.HDM.InterleaveTargetList[], the devices must still be connected in such a way that a valid setting for the HB.HDM.InterleaveTargetList[] array can be computed. In this case the devices are connected in such a way that valid settings exist.

The suggested algorithm for software to check for proper device connection to each root port on the host bridge is outlined in the Section 2.13.15.

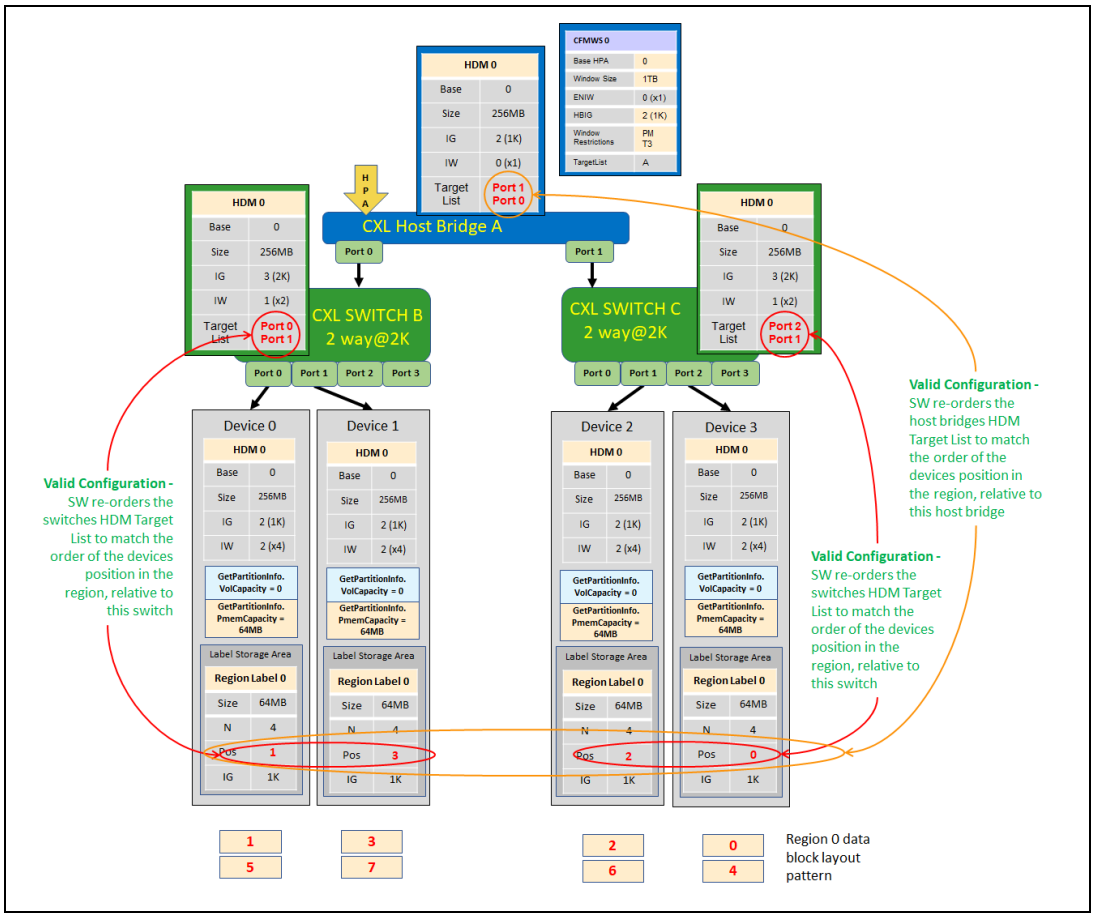
Figure 2-17 Example - Valid x2 HB Root Port Device Ordering



This example demonstrates the verification required to ensure that each devices position in the region lands on the correct root port of the host bridge. While the ordering of root ports in the HB can be changed by reshuffling the HB.HDM.InterleaveTargetList[], the devices must still be connected in such a way that a valid setting for the HB.HDM.InterleaveTargetList[] array can be computed. In this case the devices are not connected in such a way that valid settings exist.

The suggested algorithm for software to check for proper device connection to each root port on the host bridge is outlined in the Section 2.13.15.

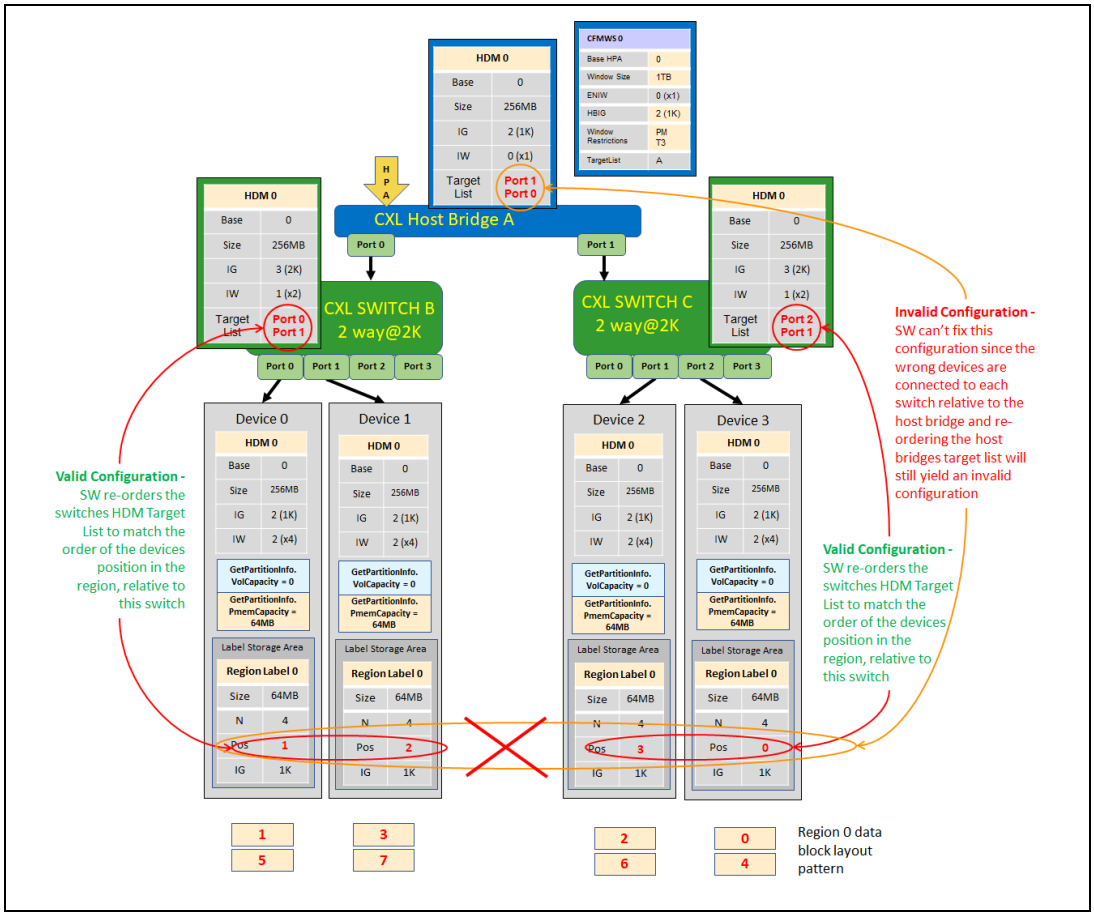
Figure 2-18 Example - Invalid x2 HB Root Port Device Ordering



This example demonstrates the verification required to ensure that each devices position in the region lands on the correct root port of the host bridge. While the ordering of root ports in the HB can be changed by reshuffling the HB.HDM.InterleaveTargetList[], the devices must still be connected in such a way that a valid setting for the HB.HDM.InterleaveTargetList[] array can be computed. In this case the devices are not connected in such a way that valid settings exist because the devices are not balanced across all the root ports that the region spans.

The suggested algorithm for software to check for proper device connection to each root port on the host bridge is outlined in the Section 2.13.15.

Figure 2-19 Example - Unbalanced Region Spanning x2 HB Root Ports



2.8 Partitioning and Configuration Sequences

The following sections outline the basic responsibilities and sequences for partitioning the amount of volatile and persistent capacity on the device, configuring the persistent memory interleaving, and enumerating partitions and regions.

2.8.1 Volatile and Persistent Memory Partitioning

The following sequences outline the basic steps in partitioning the amount of volatile and persistent capacity the device will utilize. This sequence can be performed by several entities including system firmware, UEFI driver, and OS driver. These sequences are only executed when the device partitioning needs to be changed. The OS can optionally utilize the managed hot remove and hot add sequences to re-partition memory without rebooting.

Figure 2-20 High-level Sequence: System Firmware and UEFI Driver Memory Partitioning

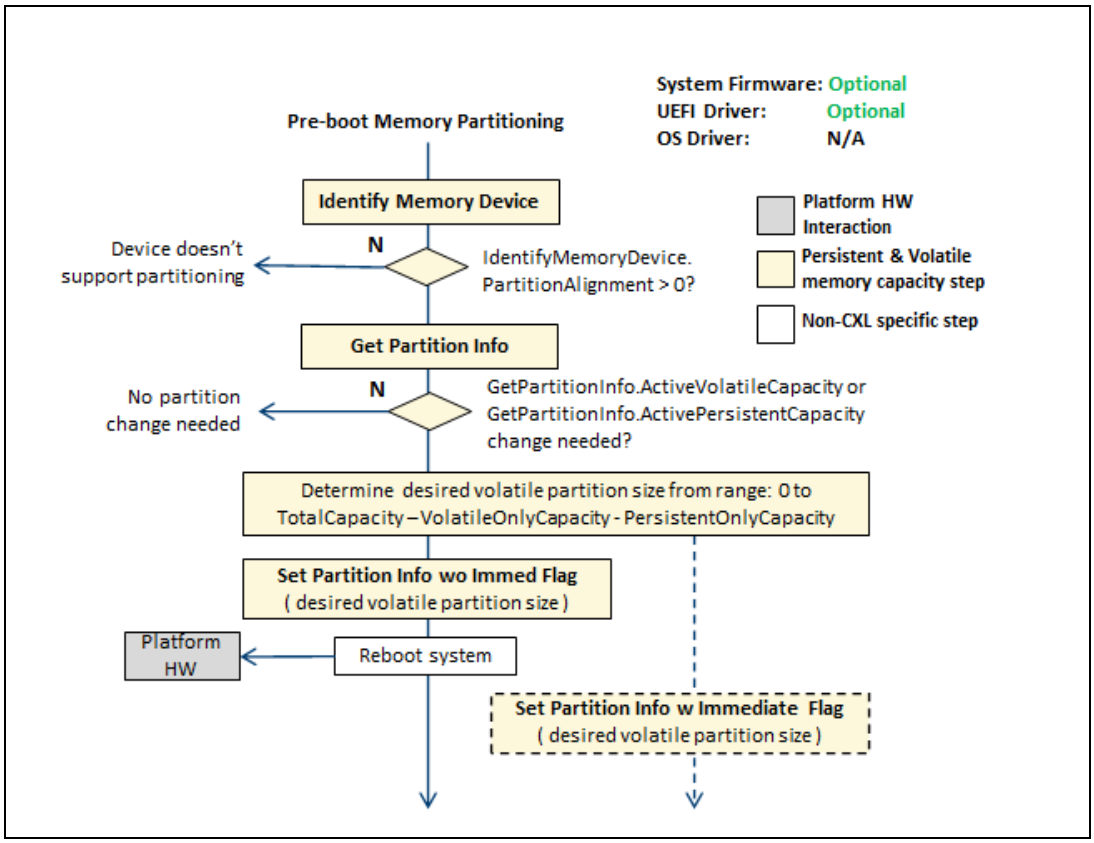
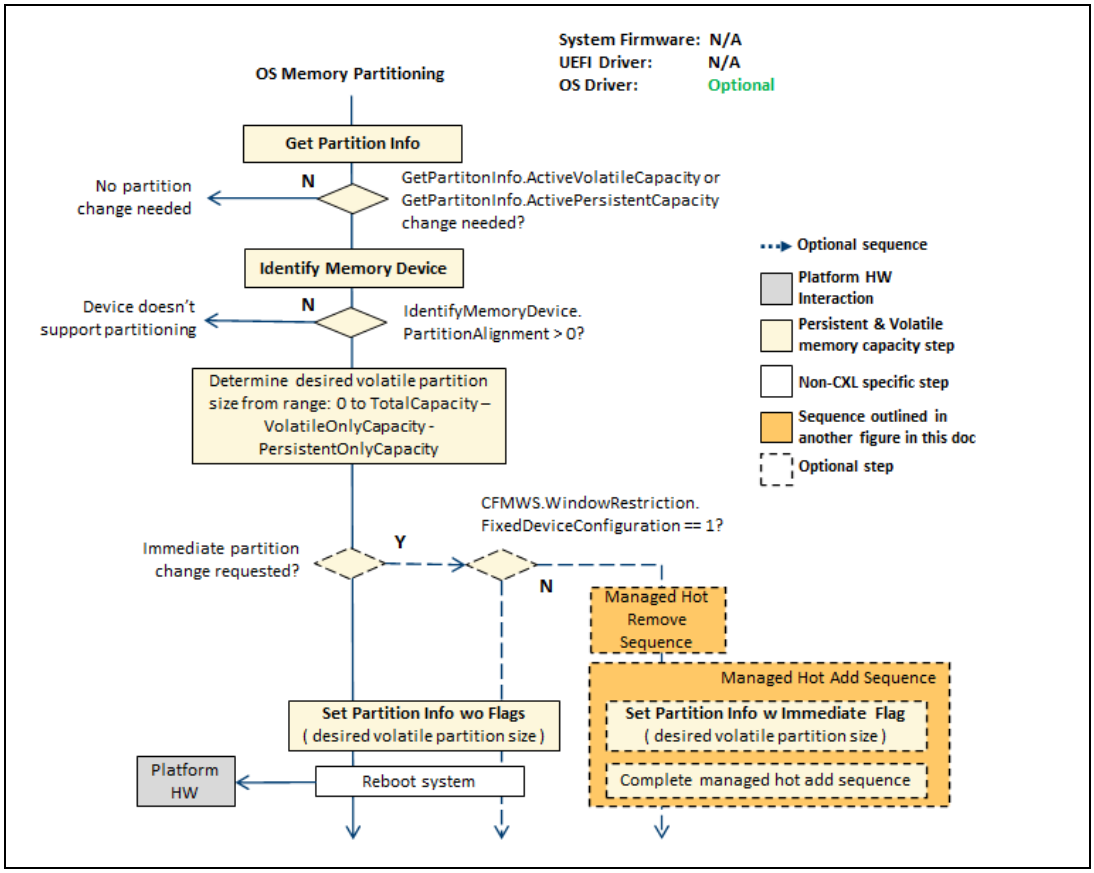


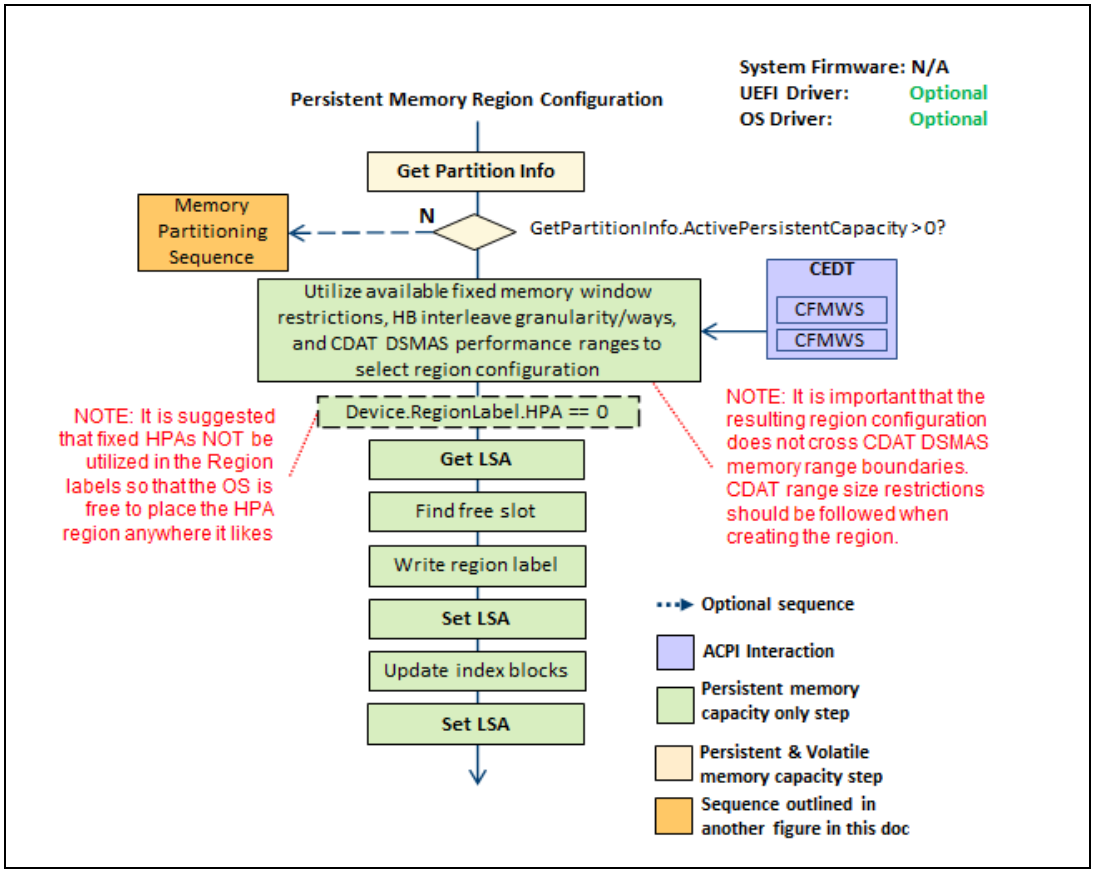
Figure 2-21 High-level Sequence: OS Memory Partitioning



2.8.2 Persistent Memory Region Configuration

The following sequence outlines the basic steps in configuring persistent memory regions to interleave memory across multiple devices. This sequence can be performed by several entities including UEFI driver, and OS driver. This sequence is only executed when the device's persistent memory region configuration needs to be changed and the data does not have to be preserved (example: data is backed up somewhere).

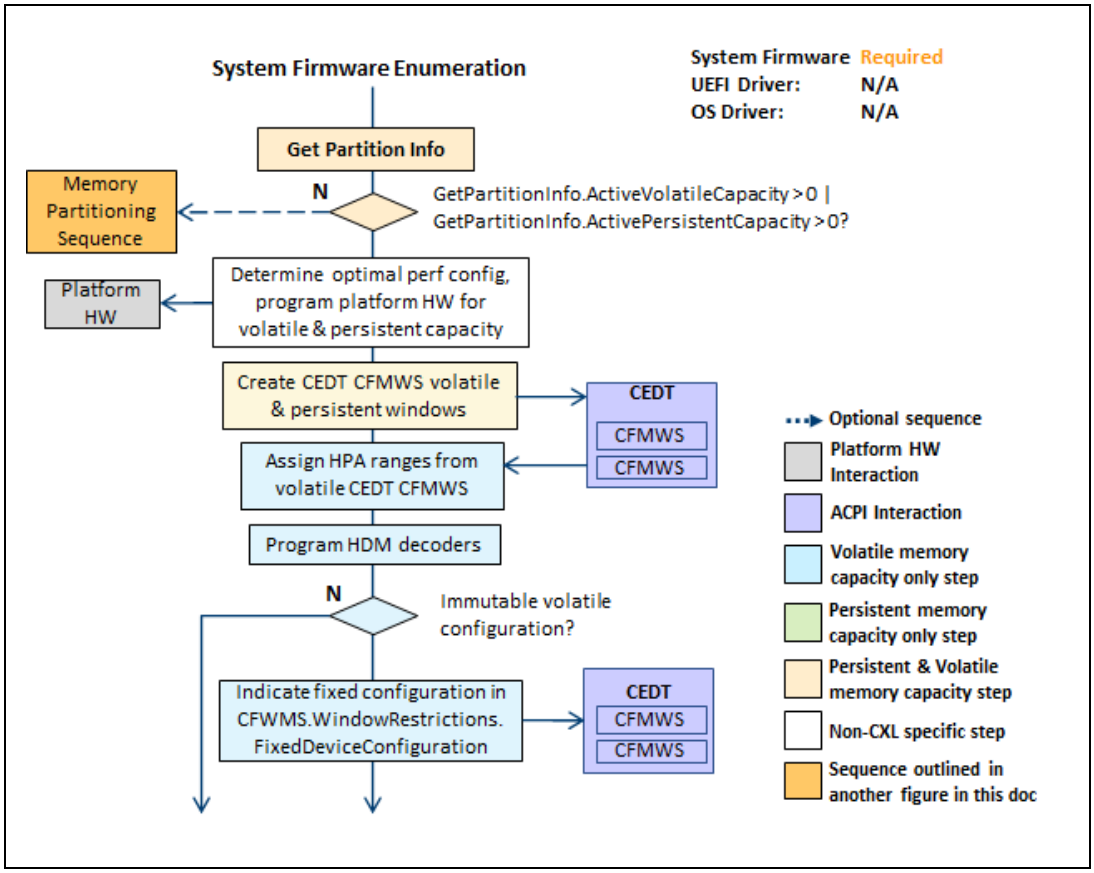
Figure 2-22 High-level Sequence: Persistent Memory Region Configuration



2.8.3 System Firmware Enumeration

The following sequence outlines the basic steps the system firmware performs when enumerating memory devices with volatile and persistent capacity. This sequence is performed by the system firmware on every system boot.

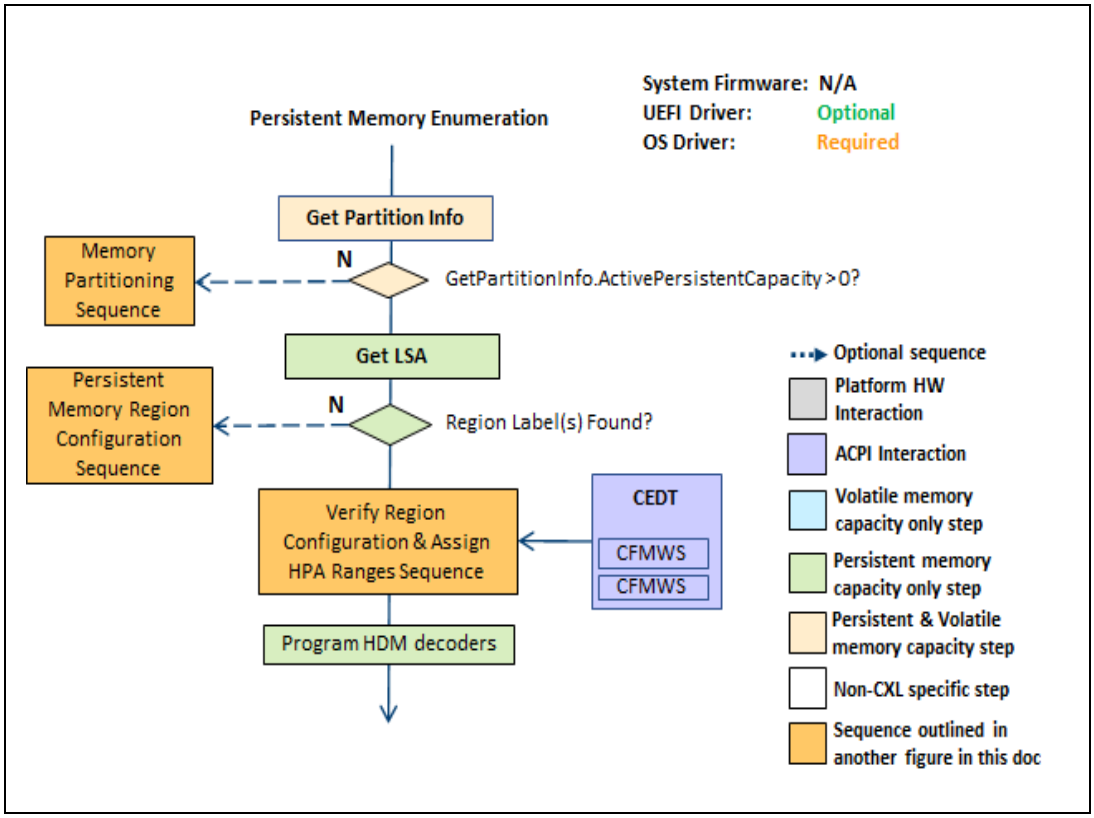
Figure 2-23 High-level Sequence: System Firmware Enumeration



2.8.4 OS and UEFI Driver Enumeration

The following sequence outlines the basic steps the OS and UEFI drivers performs when enumerating memory devices with volatile and persistent capacity. This sequence is performed by the UEFI Driver and the OS on every system boot.

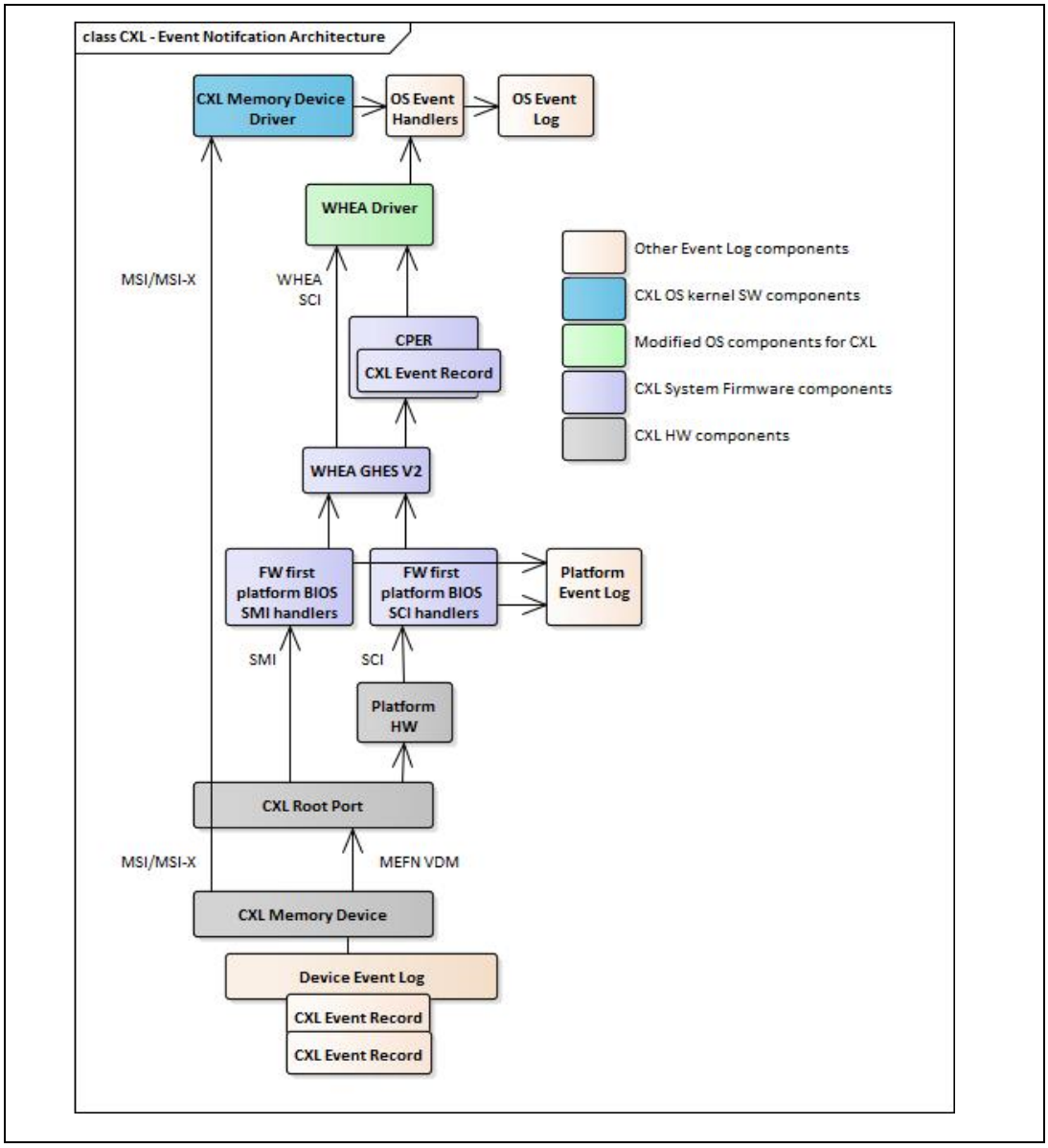
Figure 2-24 High-level Sequence: UEFI and OS Enumeration



2.9 Asynchronous Event Handling

The following figure outlines the basic architecture of asynchronous event handling and the associated components. There are two memory error notification paths with CXL, the OS first PCIe/CXL MSI/MSIx path that notifies the OS CXL memory device driver directly in the form of the interrupt, or the FW first PCIe/CXL MEFN VDM where the platform HW turns the VDM MEFN message in to a platform firmware interrupt such as System Control Interrupt (SCI) or System Management Interrupt (SMI) on IA platforms.

Figure 2-25 CXL Event Notification Architecture



2.10 Dirty Shutdown Count Handling

There are some basic rules that influenced the resulting dirty shutdown count handling flow presented here. These rules include:

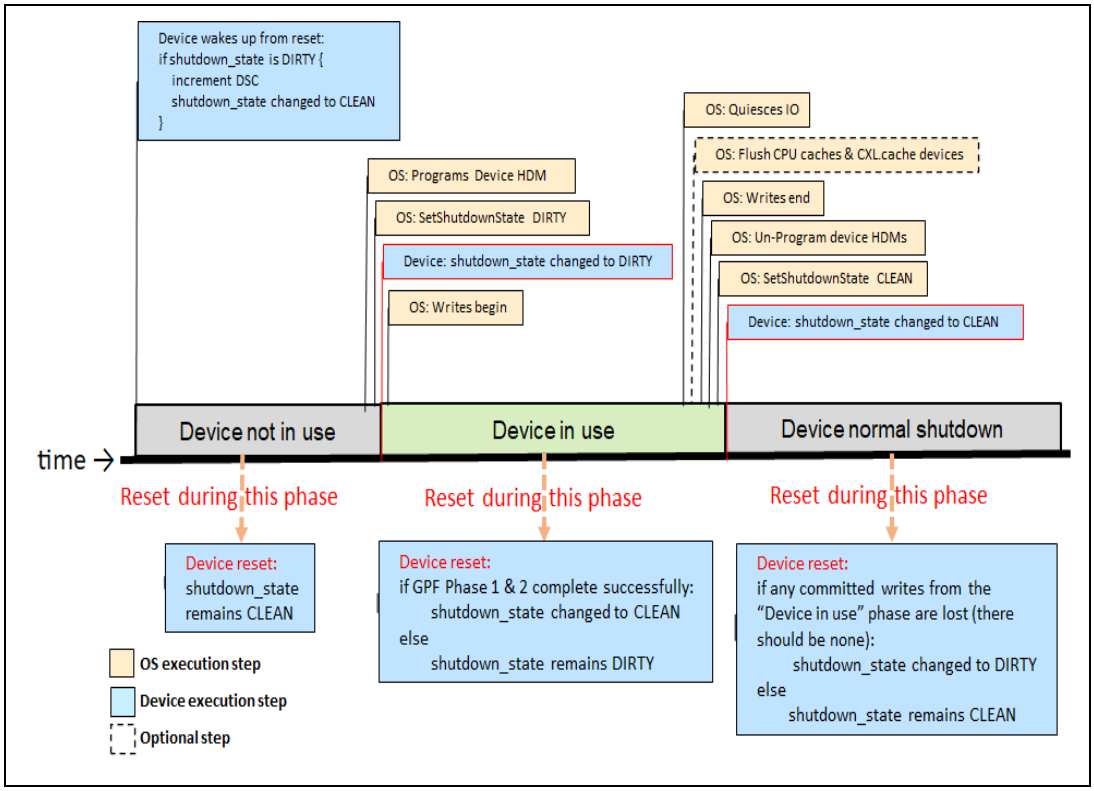
- If there are writes that the OS previously sent (before marking the device as CLEAN) and they are still sitting in some device buffer that is not persistent, then it is the device's responsibility to maintain the promise that those stores are persistent, and if it cannot, change the state to DIRTY.
- If the device's shutdown state is set to CLEAN and it drops writes that occur after being set to CLEAN, the state can remain CLEAN. It is the OS's

responsibility to set the device shutdown state to DIRTY before allowing any writes.

- Assuming the device's HDM decoders are not locked: The OS should un-program the HDM decoders for the memory device after the last write has completed and before setting the SetShutdownState state=CLEAN.
- It is an OS error to send more writes after setting the device's shutdown state to CLEAN and un-programming the HDM decoders prevents writes after the device is clean.
- The device is free to implement the persistent domain using flush-on-power fail volatile buffers and if those flushes fail, the device must change the state to DIRTY, even if software decided it is done using the device and marked it as CLEAN.
- It is the OS's responsibility to ensure CPU caches are properly flushed if caching of persistent data is enabled before setting the device's shutdown state to CLEAN.
- It is the OS's responsibility to quiesce all further CXL.MEM activity to the device and optionally flush CPU caches, before setting the shutdown state to CLEAN.
- In case there are conditions where the OS fails to complete the flushing and does not set the device's shutdown state to CLEAN, the system firmware should flush CPU caches, before setting the shutdown state to CLEAN.

The following figure outlines the basic system firmware, OS and device steps for handling, detecting, and reporting a dirty shutdown, the device phases, and timeline.

Figure 2-26 CXL Dirty Shutdown Count Handling Logic



2.11 SRAT and HMAT

The following examples outline the architectural assumptions for implementing the SRAT and HMAT with CXL volatile and persistent memory capacity.

The following proximity domains may appear in the system firmware generated SRAT:

Table 2-6 SRAT and HMAT Content

Proximity Domain	Affinity Type	Affinity Structure	Memory Range
Each CPU socket	0 – Processor	Enabled: SET	Local memory (example: attached via DDR)
Each CXL host bridge	6 – Generic Port	Enabled: SET	N/A
Each volatile memory range reported by CXL type 2 and 3 region	1 – Memory	Enabled: SET HotPluggable: Platform specific NonVolatile: CLEAR	System Firmware is responsible for creating SRAT memory range entries for every portion of the CMFWS it has programmed volatile device HDM decoders from

Proximity Domain	Affinity Type	Affinity Structure	Memory Range
Each CXL type 1 and 2 accelerator device initiator	5 – Generic Initiator	Enabled: SET	System Firmware is responsible for creating SRAT memory range entries for Type 2 accelerator memory ranges

The following general rules apply to the System Firmware generated HMAT:

- HMAT returns best case latency (unloaded) L, and best-case bandwidth B, between pairs of proximity domains described in the SRAT.
- Since the SRAT does not contain proximity domains for persistent memory capacity, the HMAT will not contain performance information related to persistent memory capacity.
- If a path P is made up of subpaths P1, P2, …, Pn
  - L(P) = L(P1)+L(P2), ..+L (Pn) where L is the latency
  - B(P) = MIN [B(P1), B(P2), ..,B(Pn)] where B is the bandwidth
- Subpaths can be one of two types
  - External Link
- L and B can be computed based on knowledge of the link width, frequency and optionally retimer count. This is described in further detail below.
- Internal Link
- Internal to a CXL device – Device CDAT returns L and B of each subpath.
- Internal to a CXL switch - Switch CDAT returns L and B of each subpath.
- Internal to the CPU – system firmware gets L and B from CPU datasheet. OS infers this from the HMAT.

The following general rules apply to the OS generated NUMA paths:

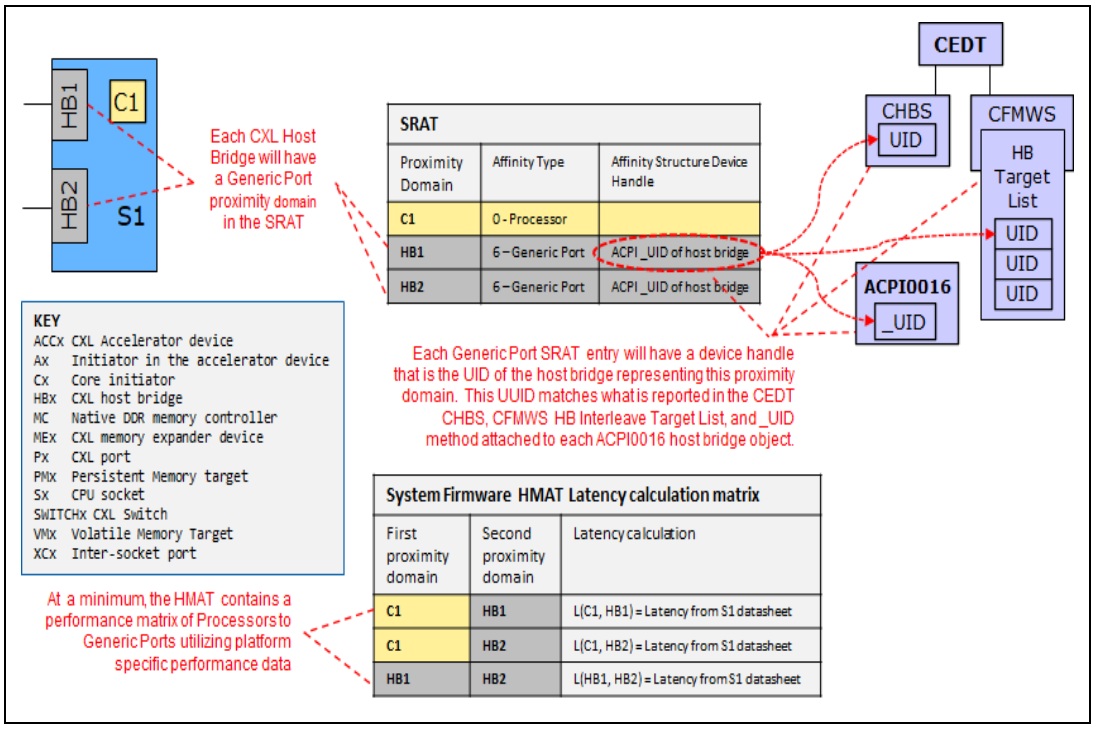
- SRAT does not contain persistent memory proximity domains.
- Therefore, HMAT does not contain persistent memory performance information.
- OS is responsible for calculating the NUMA path performance for all persistent memory enumerated at OS boot.
- OS is responsible for calculating the NUMA path performance for all volatile and persistent memory added at OS runtime through the Hot Add sequence.
- OS utilizes a combination of HMAT information, device and switch CDAT information, and the CXL link status to calculate the performance L and B for each NUMA path.

2.11.1 SRAT, HMAT and OS NUMA Calculation Examples

The following examples outline the previous responsibilities and demonstrate how the L and B are calculated by the System Firmware and the OS.

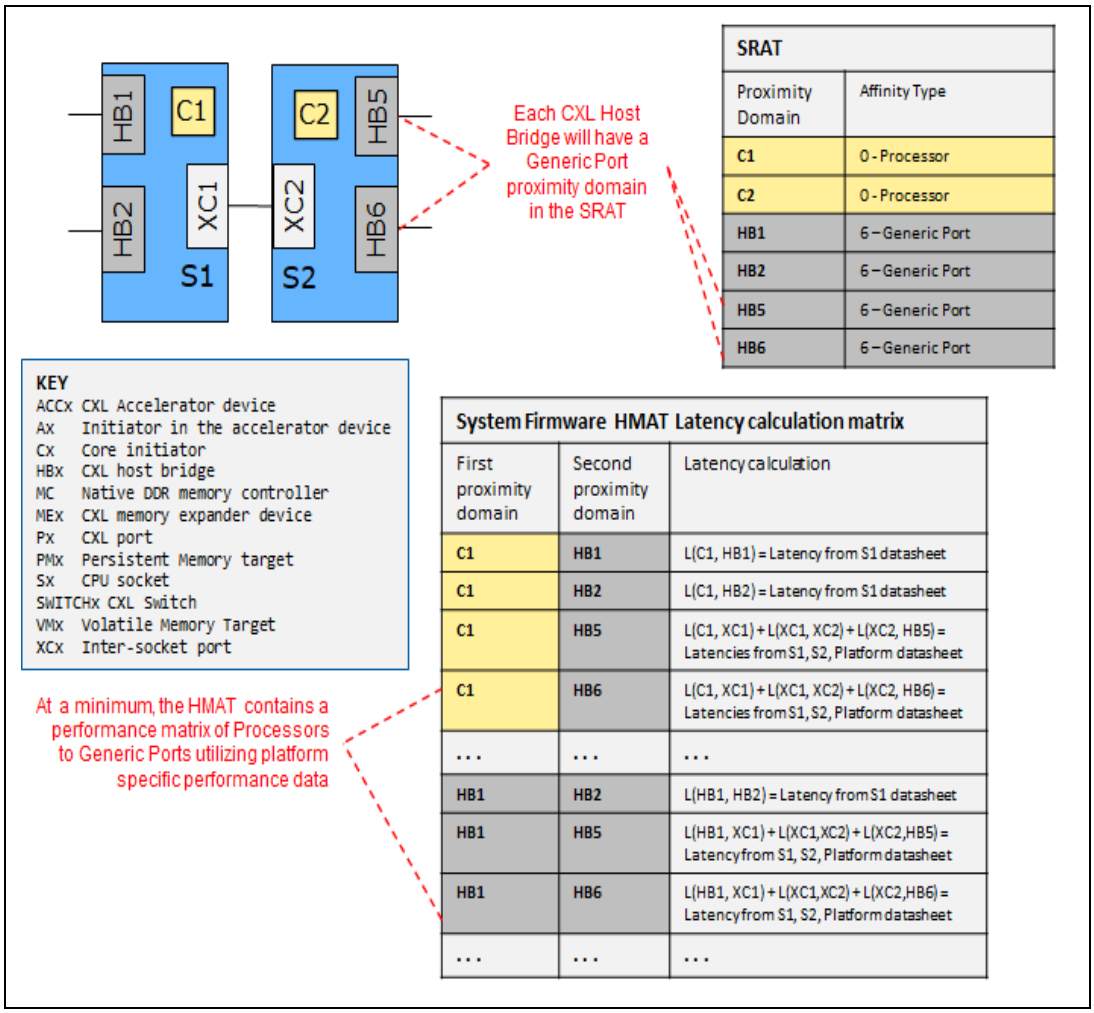
In this example, a single socket system has no memory attached at system boot. The SRAT is limited to proximity domains for CPU and root ports attached to the CPU. Note that each CXL host bridge will have a unique SRAT proximity domain. Also note how the device handle for the generic port SRAT entries identifies the UID of the host bridge representing the proximity domain and that matches the UID in the CEDT CHBS, CFWMS HB interleave target list, and the UID returned from the ACPI0016 _UID method.

Figure 2-27 SRAT and HMAT Example: Latency Calculations for 1 Socket System with no Memory Present at Boot



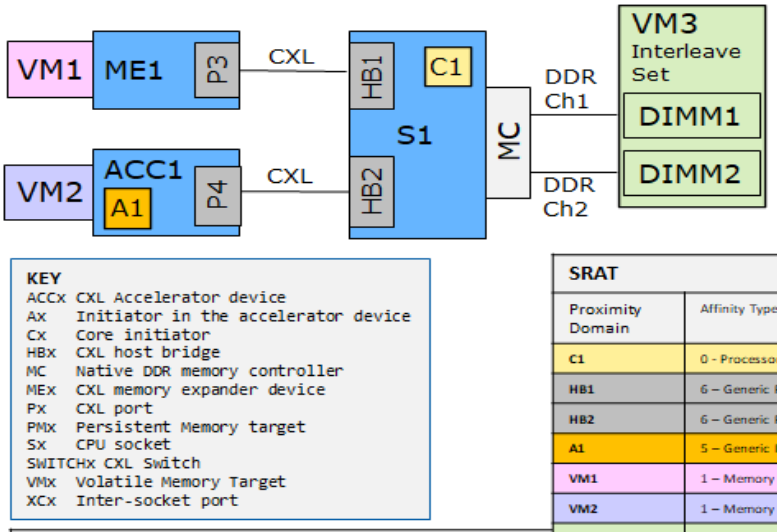
In this example, a two-socket system has no memory attached at system boot. The SRAT is limited to proximity domains for each CPU and the root ports attached to each CPU. Note that each CXL host bridge will have a unique SRAT proximity domain.

Figure 2-28 SRAT and HMAT Example: Latency Calculations for 2 Socket System with no Memory Present at Boot



In this example, a one socket system has native DDR attached memory, a CXL Type 2 accelerator device with volatile memory, and a CXL Type 3 memory expander device with volatile memory attached. This shows example latency calculations the system firmware performs to build the latency portion of the HMAT.

Figure 2-29 SRAT and HMAT Example: Latency Calculations for 1 Socket System with Volatile Memory Attached at Boot



KEY

```
ACCx CXL Accelerator device
Ax   Initiator in the accelerator device
Cx   Core initiator
HBx  CXL host bridge
MC   Native DDR memory controller
MEx  CXL memory expander device
Px   CXL port
PMx  Persistent Memory target
Sx   CPU socket
SWITCHx CXL Switch
VMx  Volatile Memory Target
XCx  Inter-socket port
```

SRAT

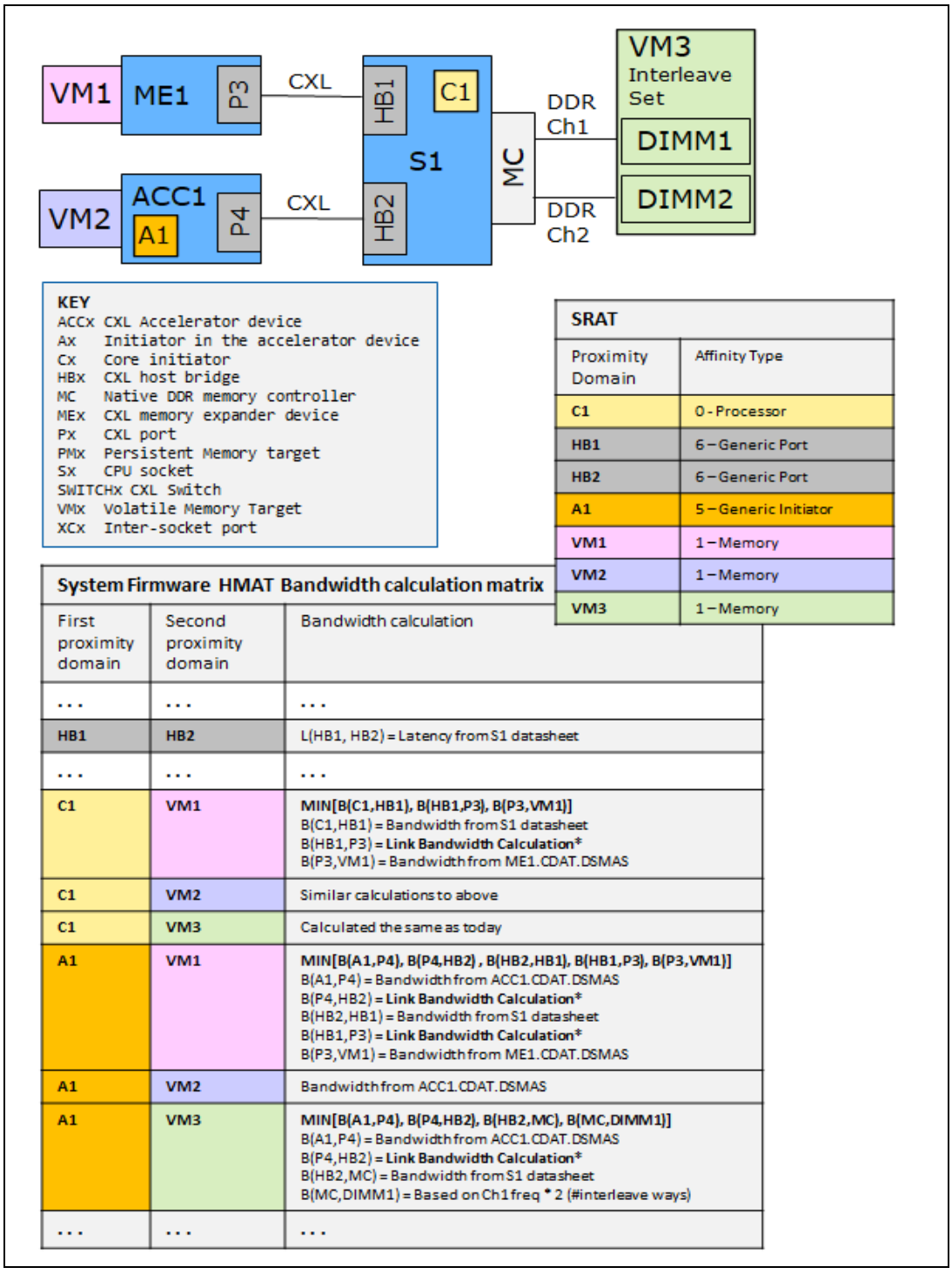
Proximity Domain	Affinity Type
C1	0 - Processor
HB1	6 – Generic Port
HB2	6 – Generic Port
A1	5 – Generic Initiator
VM1	1 – Memory
VM2	1 – Memory
VM3	1 – Memory

System Firmware HMAT Latency calculation matrix

First proximity domain	Second proximity domain	Latency calculation
...	...	...
HB1	HB2	L(HB1, HB2) = Latency from S1 datasheet
...	...	...
C1	VM1	**L(C1,HB1) + L(HB1,P3) + L(P3,VM1)** L(C1,HB1) = Latency from S1 datasheet L(HB1,P3) = **Link Latency Calculation*** L(P3,VM1) = Latency from ME1.CDAT.DSMAS
C1	VM2	**L(C1,HB2) + L(HB2,P4) + L(P4,VM2)** Similar calculations to above
C1	VM3	Calculated the same as today
A1	VM1	**L(A1,P4) + L(P4,HB2) + L(HB2,HB1) + L(HB1,P3) + L(P3,VM1)** L(A1,P4) = Latency from ACC1.CDAT.DSMAS L(P4,HB2) = **Link Latency Calculation*** L(HB2,HB1) = Latency from S1 datasheet L(HB1,P3) = **Link Latency Calculation*** L(P3,VM1) = Latency from ME1.CDAT.DSMAS
A1	VM2	Latency from ACC1.CDAT.DSMAS
A1	VM3	**L(A1,P4) + L(P4, HB2) + L(HB2,MC) + L(MC,DIMM1)** L(A1,P4) = Latency from ACC1.CDAT.DSMAS L(P4,HB2) = **Link Latency Calculation*** L(HB2,MC) = Latency from S1 datasheet L(MC,DIMM1) = Based on DIMM1 SPD & DDR Ch1 freq
...	...	...

In this example, the same one socket system as the previous example has native DDR attached memory, a CXL Type 2 accelerator device with volatile memory, and a CXL Type 3 memory expander device with volatile memory attached. This shows example bandwidth calculations the system firmware performs to build the bandwidth portion of the HMAT.

Figure 2-30 SRAT and HMAT Example: Bandwidth Calculations for 1 Socket System with Volatile Memory Attached at Boot



SRAT

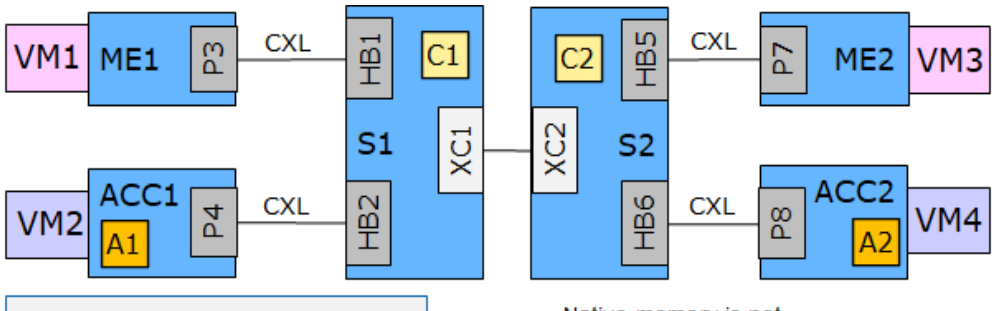
Proximity Domain	Affinity Type
C1	0 - Processor
HB1	6 – Generic Port
HB2	6 – Generic Port
A1	5 – Generic Initiator
VM1	1 – Memory
VM2	1 – Memory
VM3	1 – Memory

System Firmware HMAT Bandwidth calculation matrix

First proximity domain	Second proximity domain	Bandwidth calculation
...	...	...
HB1	HB2	L(HB1, HB2) = Latency from S1 datasheet
...	...	...
C1	VM1	**MIN[B(C1,HB1), B(HB1,P3), B(P3,VM1)]** B(C1,HB1) = Bandwidth from S1 datasheet B(HB1,P3) = **Link Bandwidth Calculation*** B(P3,VM1) = Bandwidth from ME1.CDAT.DSMAS
C1	VM2	Similar calculations to above
C1	VM3	Calculated the same as today
A1	VM1	**MIN[B(A1,P4), B(P4,HB2) , B(HB2,HB1), B(HB1,P3), B(P3,VM1)]** B(A1,P4) = Bandwidth from ACC1.CDAT.DSMAS B(P4,HB2) = **Link Bandwidth Calculation*** B(HB2,HB1) = Bandwidth from S1 datasheet B(HB1,P3) = **Link Bandwidth Calculation*** B(P3,VM1) = Bandwidth from ME1.CDAT.DSMAS
A1	VM2	Bandwidth from ACC1.CDAT.DSMAS
A1	VM3	**MIN[B(A1,P4), B(P4,HB2), B(HB2,MC), B(MC,DIMM1)]** B(A1,P4) = Bandwidth from ACC1.CDAT.DSMAS B(P4,HB2) = **Link Bandwidth Calculation*** B(HB2,MC) = Bandwidth from S1 datasheet B(MC,DIMM1) = Based on Ch1 freq * 2 (#interleave ways)
...	...	...

In this example, a two-socket system has two CXL Type 2 accelerator devices with volatile memory, and two CXL Type 3 memory expander devices with volatile memory attached. This shows example latency calculations the system firmware performs to build the latency portion of the HMAT.

Figure 2-31 SRAT and HMAT Example: Latency Calculations for 2 Socket System with Volatile Memory Attached at Boot



KEY

```
ACCx CXL Accelerator device
Ax   Initiator in the accelerator device
Cx   Core initiator
HBx  CXL host bridge
MC   Native DDR memory controller
MEx  CXL memory expander device
Px   CXL port
PMx  Persistent Memory target
Sx   CPU socket
SWITCHx CXL Switch
VMx  Volatile Memory Target
XCx  Inter-socket port
```

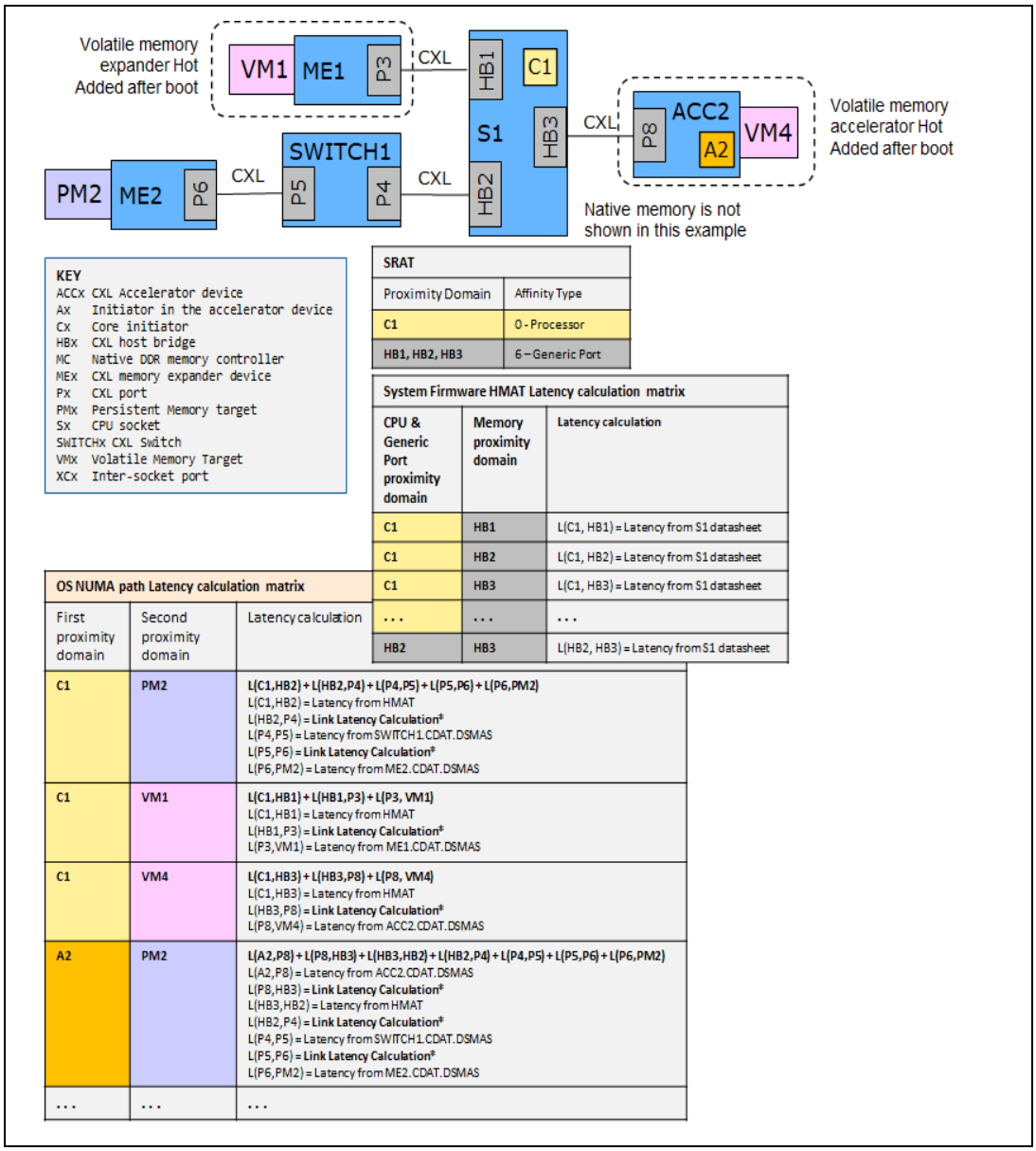
Native memory is not shown in this example

SRAT	
Proximity Domain	Affinity Type
C1, C2	0 - Processor
HB1, HB2, HB5, HB6	6 – Generic Port
A1, A2	5 – Generic Initiator
VM1, VM3	1 – Memory
VM2, VM4	1 – Memory

System Firmware HMAT Latency calculation matrix		
First proximity domain	Second proximity domain	Latency calculation
C2	HB5	L(C2, HB5) = Latency from S2 datasheet
C2	HB6	L(C2, HB6) = Latency from S2 datasheet
C2	HB1	L(C2, XC2) + L(XC2, XC1) + L(XC1, HB1) = Latencies from S1, S2, Platform datasheet
C2	HB2	L(C2, XC2) + L(XC2, XC1) + L(XC1, HB2) = Latencies from S1, S2, Platform datasheet
...	...	...
HB5	HB6	L(HB1, HB2) = Latency from S2 datasheet
HB1	HB5	L(HB1, XC1) + L(XC1,XC2) + L(XC2,HB5) = Latency from S1, S2, Platform datasheet
HB1	HB6	L(HB1, XC1) + L(XC1,XC2) + L(XC2,HB6) = Latency from S1, S2, Platform datasheet
...	...	...
C2	VM2	**L(C2,XC2) + L(XC2,XC1) + L(XC1,HB2) + L(HB2,P4) + L(P4,VM2)** L(C2,XC2), L(XC2,XC1), L(XC1,HB2) = Latency from S1/S2 datasheet L(HB2,P4) = **Link Latency Calculation*** L(P4,VM2) = Latency from ACC1.CDAT.DSMAS
A1	VM3	**L(A1,P4) + L(P4,HB2) + L(HB2,XC1) + L(XC1,XC2) + L(XC2,HB5) + L(HB5,P7) + L(P7, VM3)** L(A1,P4) = Latency from ACC1.CDAT.DSMAS L(P4,HB2) = **Link Latency Calculation*** L(HB2,XC1), L(XC1,XC2), L(XC2,HB5) = Latency from S1/S2 datasheet L(HB5,P7) = **Link Latency Calculation*** L(P7,VM3) = Latency from ME2.CDAT.DSMAS
...	...	...

In this example, a one socket system has a CXL Type 3 memory expander device with persistent memory attached via a switch, either at OS boot time, or added after OS boot via the hot add sequence. The system also has a CXL Type 3 memory expander device with volatile memory, added after OS boot via the hot add sequence. This shows example NUMA path latency calculations the OS performs for each persistent memory device present at OS boot time, and volatile and persistent memory devices added later via the hot add sequence. Note that the OS relies on SRAT and HMAT information, the switch and device CDAT information, and PCIe link status to determine the overall latency and bandwidth for these devices.

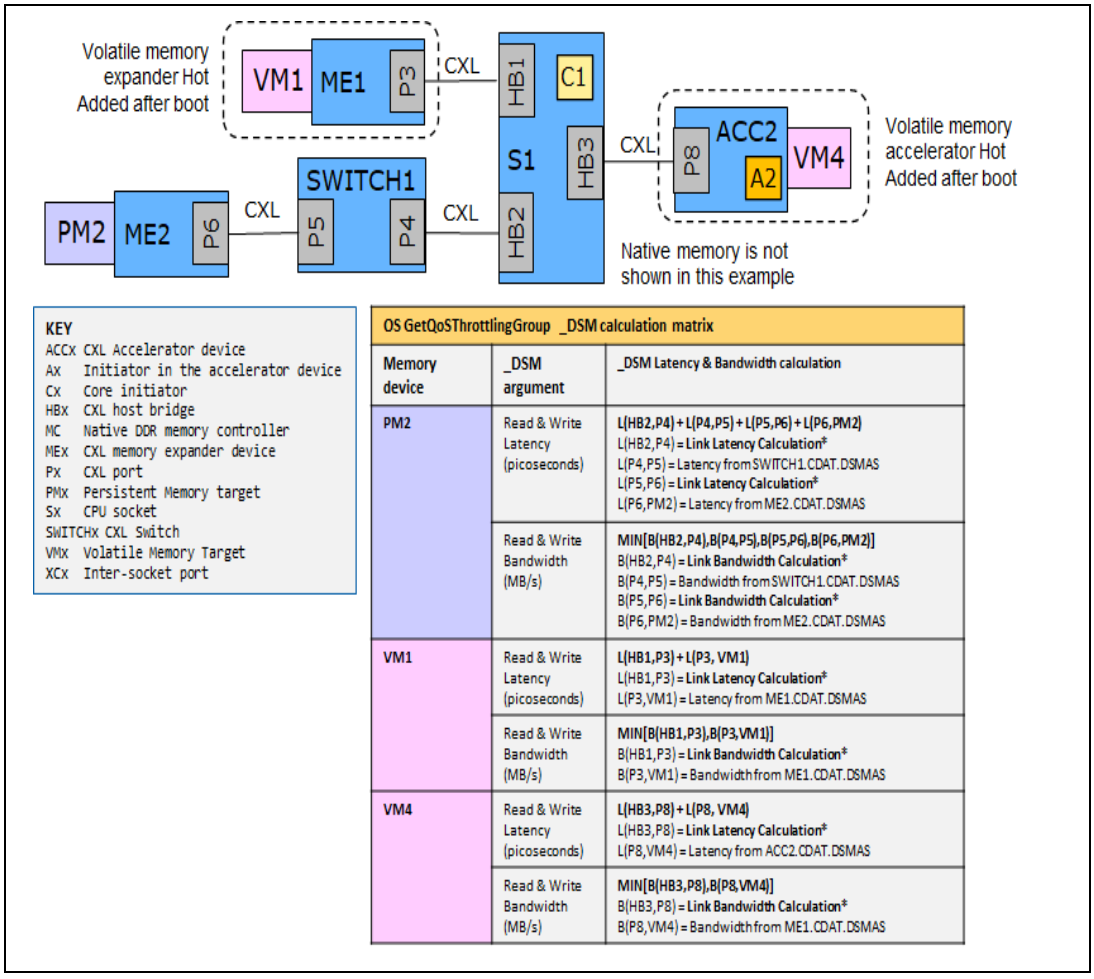
Figure 2-32 SRAT and HMAT Example: Latency Calculations for 1 Socket System with Persistent Memory and Hot Added Memory



2.11.2 GetQoSThrottlingGroup _DSM Calculation Examples

To execute the GetQoSThrottlingGroup _DSM, the OS needs to calculate the bandwidth and latency from the CXL host bridge to the device. The calculations required for the _DSM are identical to the OS NUMA path performance calculations, except the CXL host bridge root port to the CPU performance component is not included. The following shows the previous PMEM and hot add example and the calculations required to call the GetQoSThrottlingGroup _DSM.

Figure 2-33 GetQosThrottlingGroup _DSM Example



KEY
ACCx CXL Accelerator device
Ax Initiator in the accelerator device
Cx Core initiator
HBx CXL host bridge
MC Native DDR memory controller
MEx CXL memory expander device
Px CXL port
PMx Persistent Memory target
Sx CPU socket
SWITCHx CXL Switch
VMx Volatile Memory Target
XCx Inter-socket port

OS GetQoSThrottlingGroup _DSM calculation matrix

Memory device	_DSM argument	_DSM Latency & Bandwidth calculation
PM2	Read & Write Latency (picoseconds)	**L(HB2,P4) + L(P4,P5) + L(P5,P6) + L(P6,PM2)** L(HB2,P4) = **Link Latency Calculation*** L(P4,P5) = Latency from SWITCH1.CDAT.DSMAS L(P5,P6) = **Link Latency Calculation*** L(P6,PM2) = Latency from ME2.CDAT.DSMAS
	Read & Write Bandwidth (MB/s)	**MIN[B(HB2,P4),B(P4,P5),B(P5,P6),B[P6,PM2)]** B(HB2,P4) = **Link Bandwidth Calculation*** B(P4,P5) = Bandwidth from SWITCH1.CDAT.DSMAS B(P5,P6) = **Link Bandwidth Calculation*** B(P6,PM2) = Bandwidth from ME2.CDAT.DSMAS
VM1	Read & Write Latency (picoseconds)	**L(HB1,P3) + L(P3, VM1)** L(HB1,P3) = **Link Latency Calculation*** L(P3,VM1) = Latency from ME1.CDAT.DSMAS
	Read & Write Bandwidth (MB/s)	**MIN[B(HB1,P3),B(P3,VM1)]** B(HB1,P3) = **Link Bandwidth Calculation*** B(P3,VM1) = Bandwidth from ME1.CDAT.DSMAS
VM4	Read & Write Latency (picoseconds)	**L(HB3,P8) + L(P8, VM4)** L(HB3,P8) = **Link Latency Calculation*** L(P8,VM4) = Latency from ACC2.CDAT.DSMAS
	Read & Write Bandwidth (MB/s)	**MIN[B(HB3,P8),B(P8,VM4)]** B(HB3,P8) = **Link Bandwidth Calculation*** B(P8,VM4) = Bandwidth from ME1.CDAT.DSMAS

2.11.3 Link Bandwidth Calculation

Here is the basic CXL link bandwidth calculation the system firmware and OS utilize in the previous examples. This calculation is performed at each CXL host bridge root port and, if present, at the downstream ports of the switch as outlined in the previous examples.

LinkOperatingFrequency (GT/s) = Use the current negotiated link speed, PCIeLinkStatus.Speed, to reference the device's supported link speeds bit mask, PCIeLinkCapabilities2.SupportedLinkSpeedsVector

DataRatePerLink (MB/s) = LinkOperatingFrequency (GT/s) / 8 (bytes/Transfer)

LinkBandwidth (MB/s) = PCIeLinkStatus.NegotiatedLinkWidth x DataRatePerLink

2.11.4 Link Latency Calculation

Here is the basic CXL link latency calculation the system firmware and OS utilize in the previous examples. This calculation is performed at each CXL host bridge root port and, if present, at the downstream ports of the switch as outlined in the previous examples. Unless the PCIe link speed or width are severely degraded, all these terms may be considered negligible, relative to the device latencies, by the OS implementation.

LinkPropagationLatency (ps) = This term is assumed to be negligible relative to the other latencies and 0 is used for the rest of the calculation

FlitLatecny (ps) = FlitSize (bytes – From *CXL Specification*) / LinkBandwidth (MB/s – From Link Bandwidth calculation)

RetimerLatency (ps) = This term is assumed to be negligible relative to the other latencies and 0 is used

LinkLatency (ps) = LinkPropagationLatency + FlitLatency + RetimerLatency

2.12 Operation Ordering Restrictions

The following section outlines specific ordering of operations requirements that the UEFI and OS drivers shall adhere to when implementing a CXL memory driver.

- System Firmware, UEFI and OS drivers shall wait for MemoryDeviceStatusRegister.MailboxInterfaceReady before executing mailbox commands.
- System Firmware, UEFI and OS drivers shall wait for MemoryDeviceStatusRegister.MediaStatus==Ready before executing .MEM transactions.
- Persistent capacity required ordering requirements to maintain data consistency:
  - If GetSecurityState.SecurityState == Locked, the device shall be unlocked before persistent memory requests utilizing the HDMs begin.
  - Anytime the memory device transitions from security locked to security unlocked, the OS shall invalidate all CPU caches and all Type 2 cache lines that map to persistent memory, before memory requests are started.

- Shall set SetShutdownState state=DIRTY before the first write to pmem via CXL.memory.
- If FADT.PERSISTENT_CPU_CACHES == 10b, OS shall flush all CPU caches and all Type 2 cache lines that map to persistent memory, before setting SetShutdownState state=CLEAN.
- OS should un-program the memory device's HDM decoders after the last write to pmem via memory.
- OS shall set SetShutdownState state=CLEAN after writes have completed and HDM decoders un-programmed.

- Persistent capacity optional RAS requirements for best practices:
  - SetTimestamp before SetEventInterruptPolicy to minimize the device generating logs without timestamps.

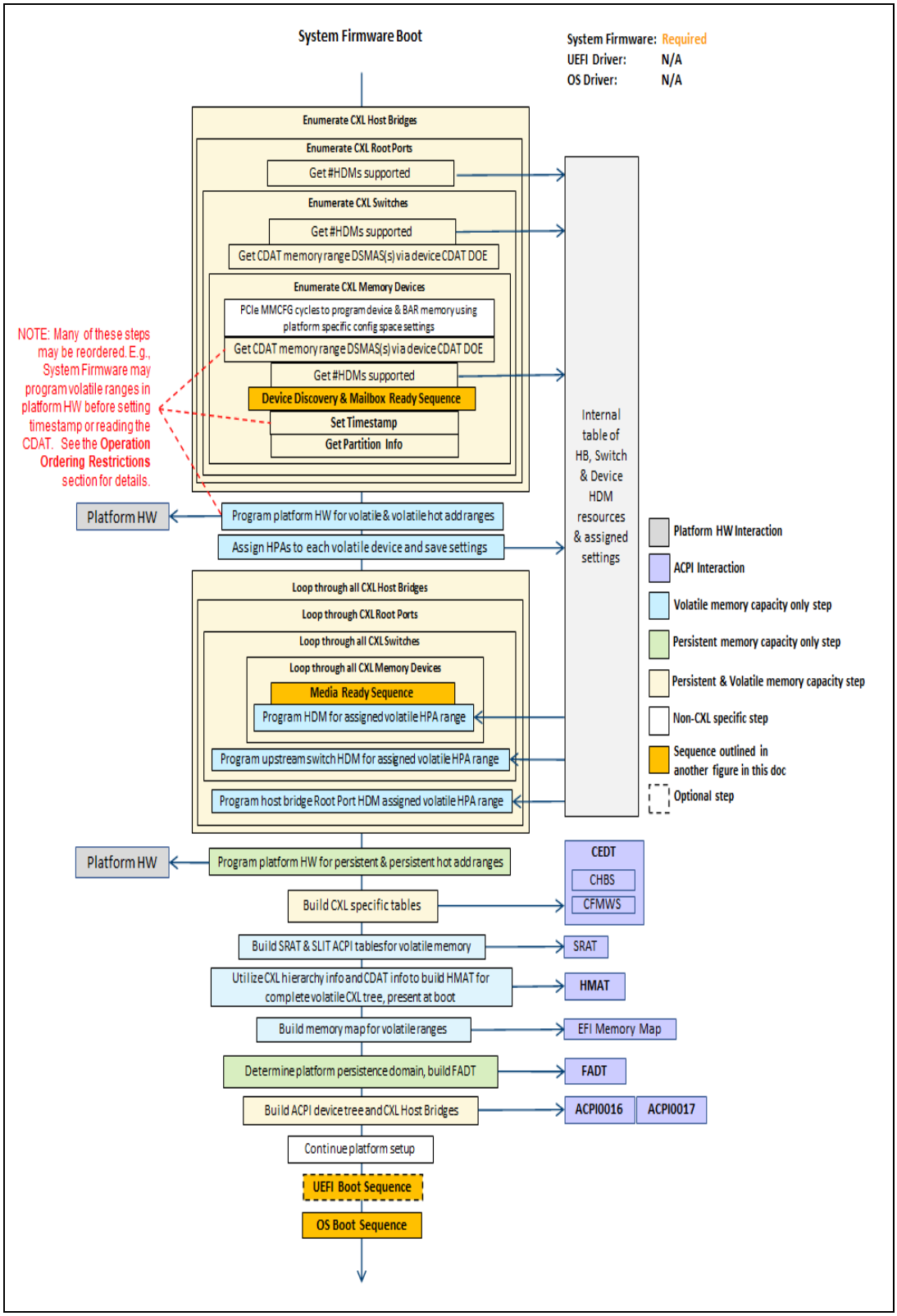
2.13 Basic High-level Sequences

The following sections outline the basic high-level sequences with major steps outlined in the previous high-level responsibilities table. This only covers high-level steps that are specific to the CXL memory device driver sequences. The rest of the document provides details on these steps.

2.13.1 System Firmware Boot Sequence

The following figure outlines the basic high-level system firmware boot sequence.

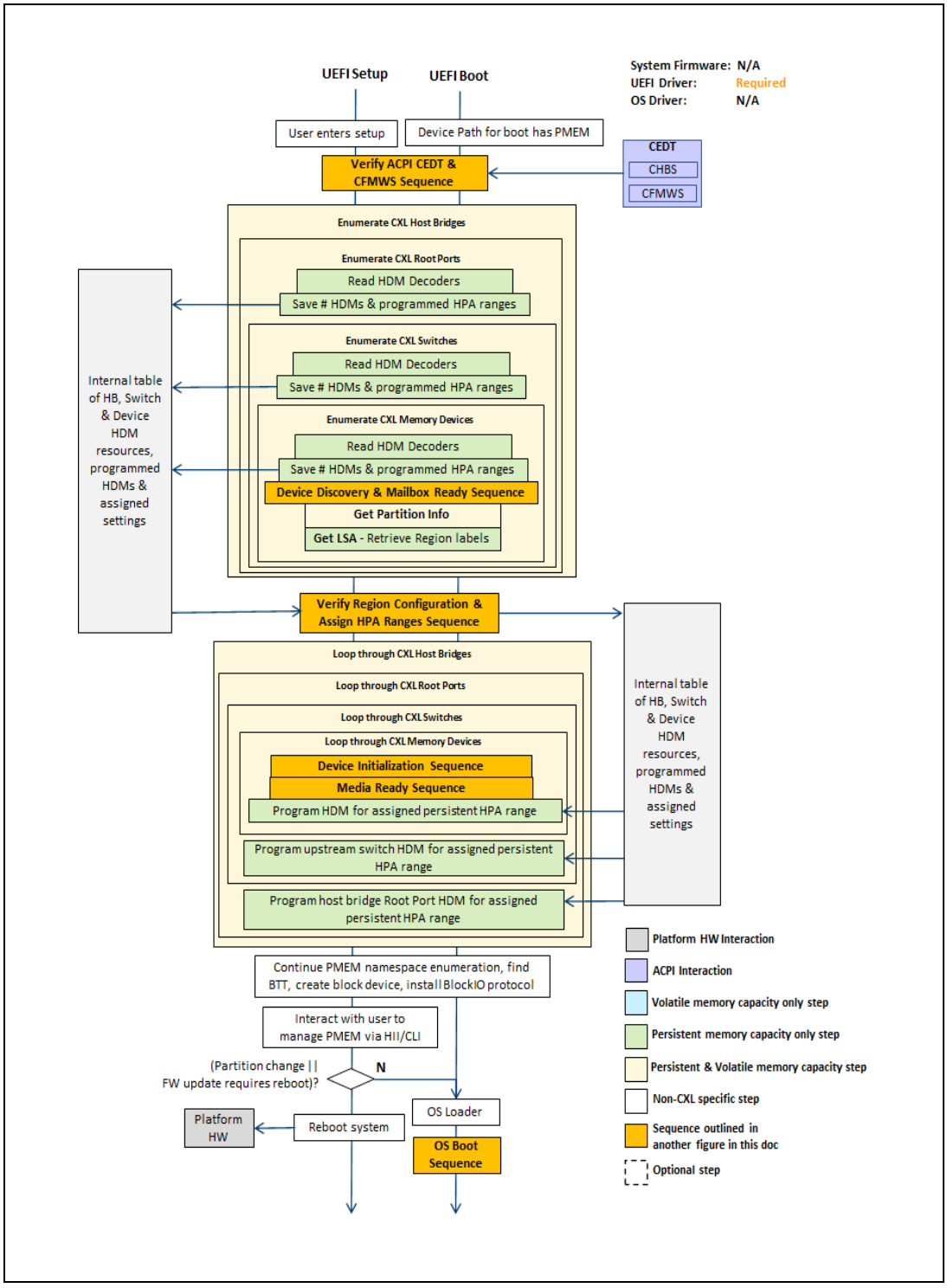
Figure 2-34 High-level Sequence: System Firmware Boot



2.13.2 UEFI Setup and Boot Sequence

The following figure outlines the basic high-level UEFI setup and boot sequence.

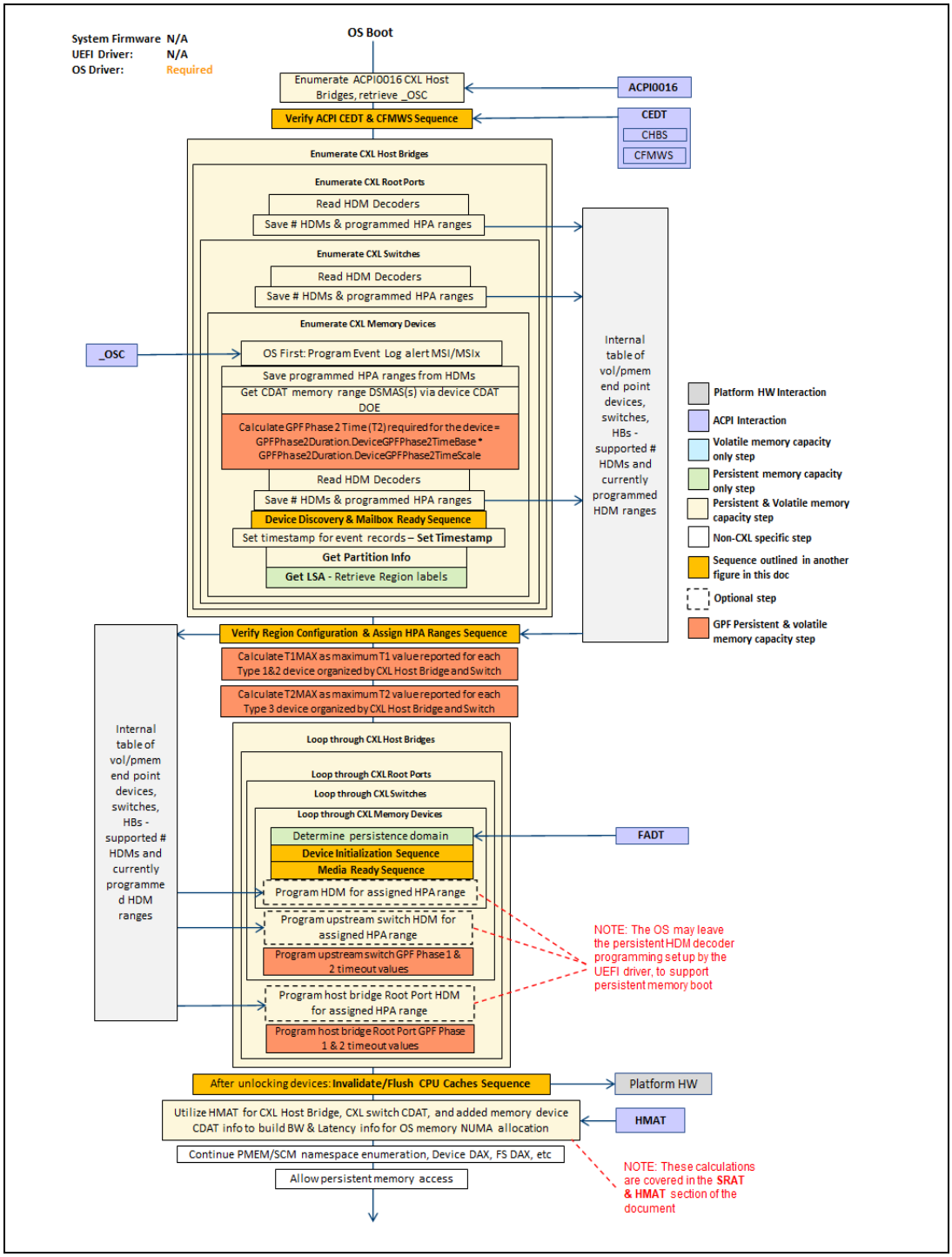
Figure 2-35 High-level Sequence: UEFI Setup and Boot



2.13.3 OS Boot Sequence

The following figure outlines the basic high-level OS boot sequence.

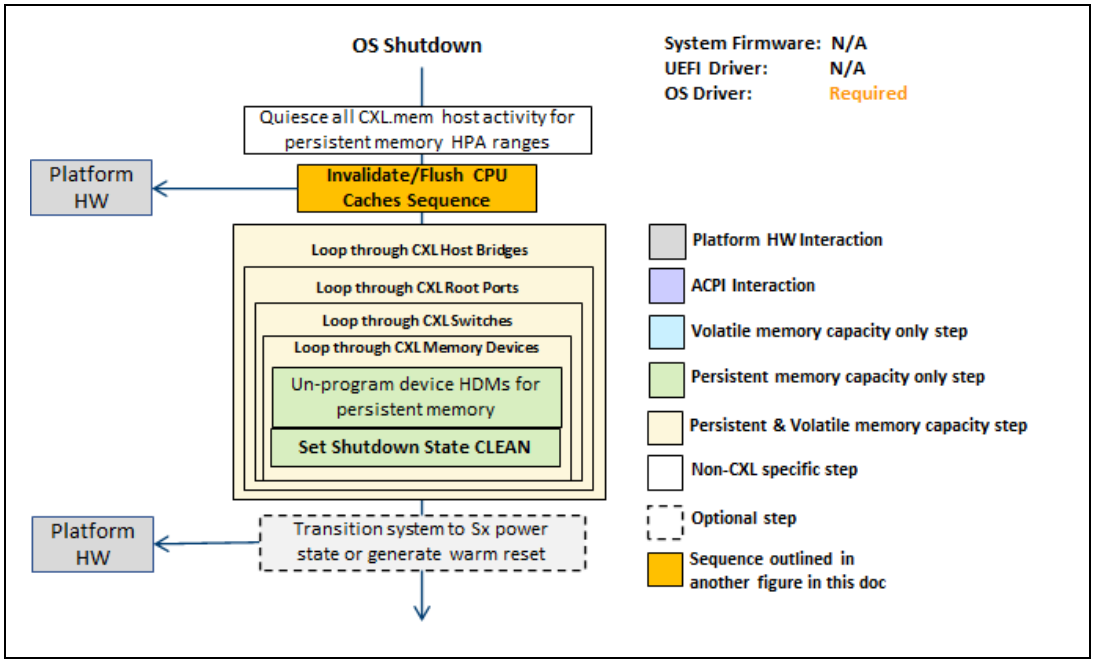
Figure 2-36 High-level Sequence: OS Boot



2.13.4 OS Shutdown Sequence

The following figure outlines the basic high-level OS graceful shutdown sequence.

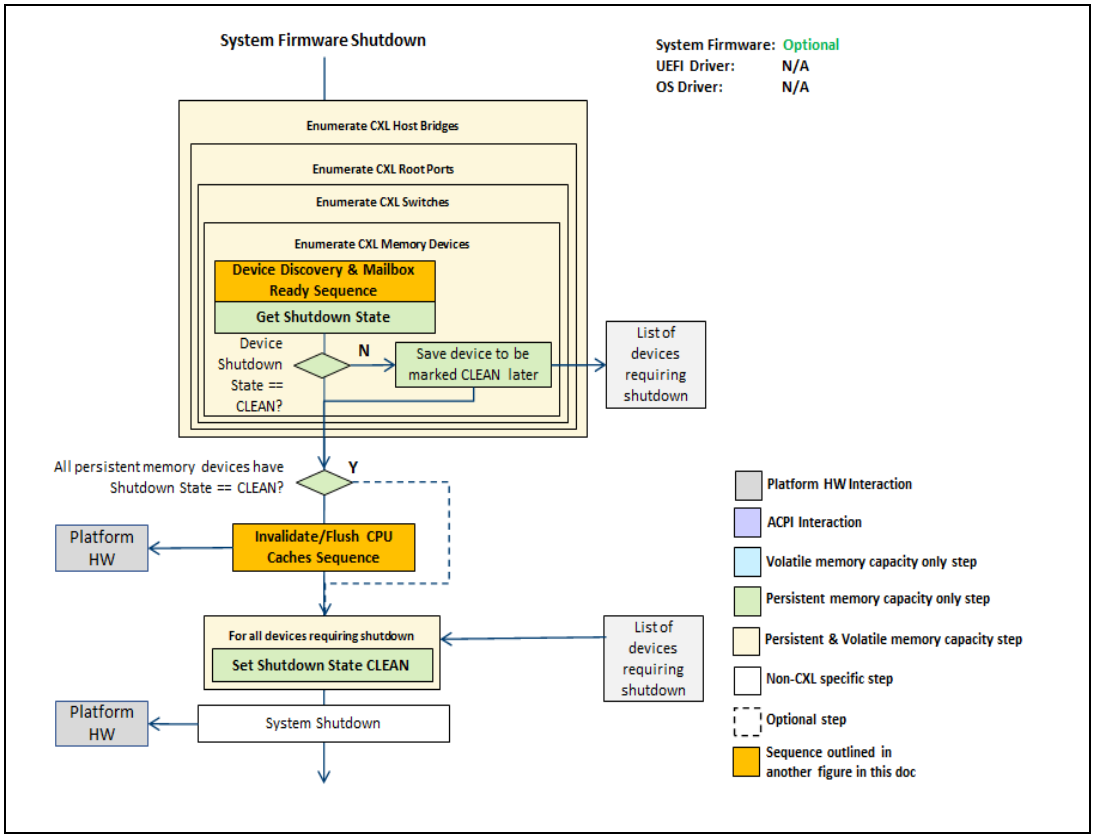
Figure 2-37 High-level Sequence: OS Shutdown



2.13.5 System Firmware Shutdown Sequence

The following figure outlines the basic high-level system firmware graceful shutdown sequence. This flow only applies to platform designs that utilize a system firmware shutdown handler invoked from the OS on a graceful shutdown.

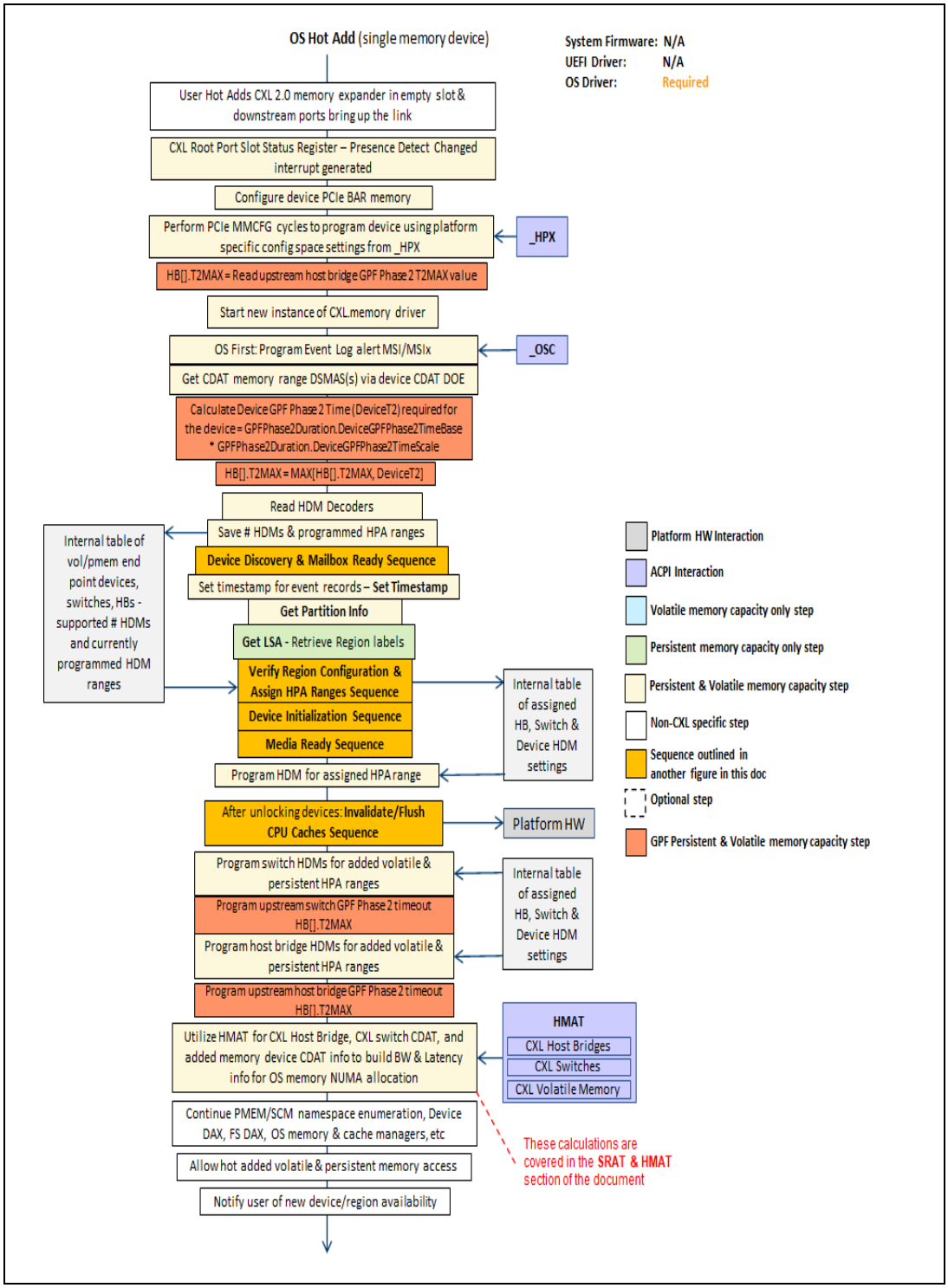
Figure 2-38 High-level Sequence: System Firmware Shutdown



2.13.6 OS Hot Add Sequence

The following figure outlines the basic high-level OS memory device hot add sequence for a single memory device.

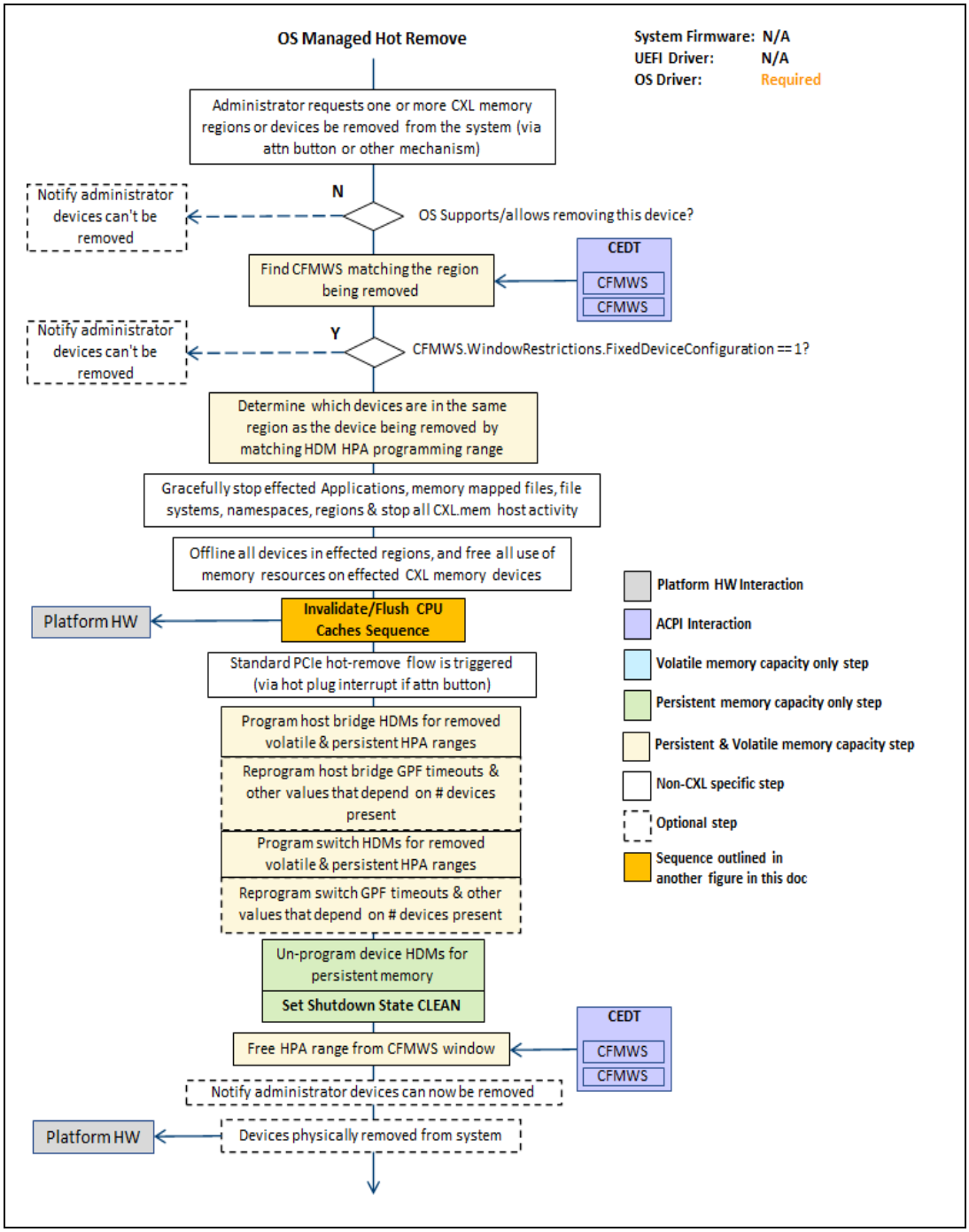
Figure 2-39 High-level Sequence: OS Hot Add



2.13.7 OS Managed Hot Remove Sequence

The following figure outlines the basic high-level OS memory managed hot remove sequence.

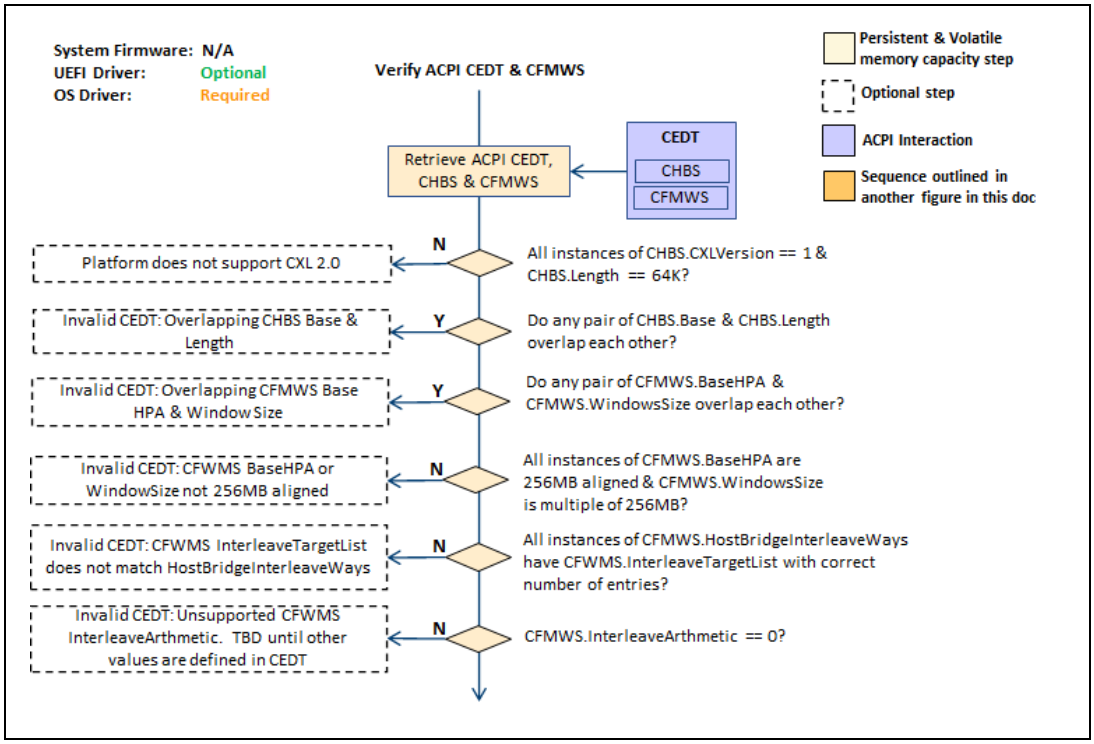
Figure 2-40 High-level Sequence: OS Managed Hot Remove



2.13.8 Verifying ACPI CEDT, CHBS and CFMWS Sequence

The following sequence outlines the basic steps the OS and UEFI drivers perform when verifying the consistency of the ACPI CEDT, CHBS and CFMWS tables produced by the system firmware.

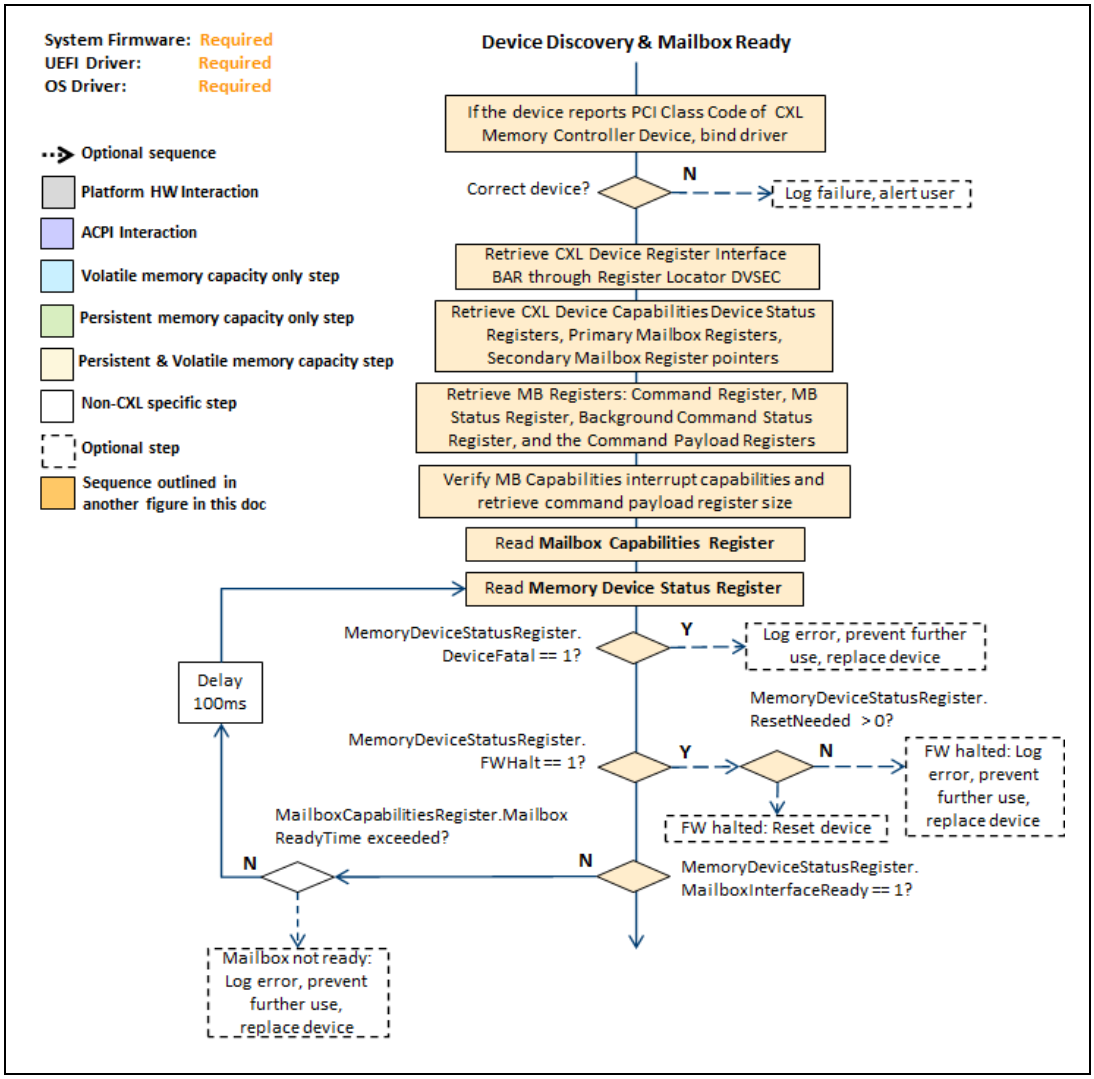
Figure 2-41 High-level Sequence: Verifying CEDT, CHBS, CFMWS



2.13.9 Device Discovery and Mailbox Ready Sequence

The following figure outlines the basic high-level sequence the System Firmware, UEFI and OS drivers will execute to determine if the CXL end point device found during PCI enumeration is a CXL memory device, discovery of the mailbox interface and polling on mailbox ready status for the device. After this sequence the device's mailbox interface can be utilized.

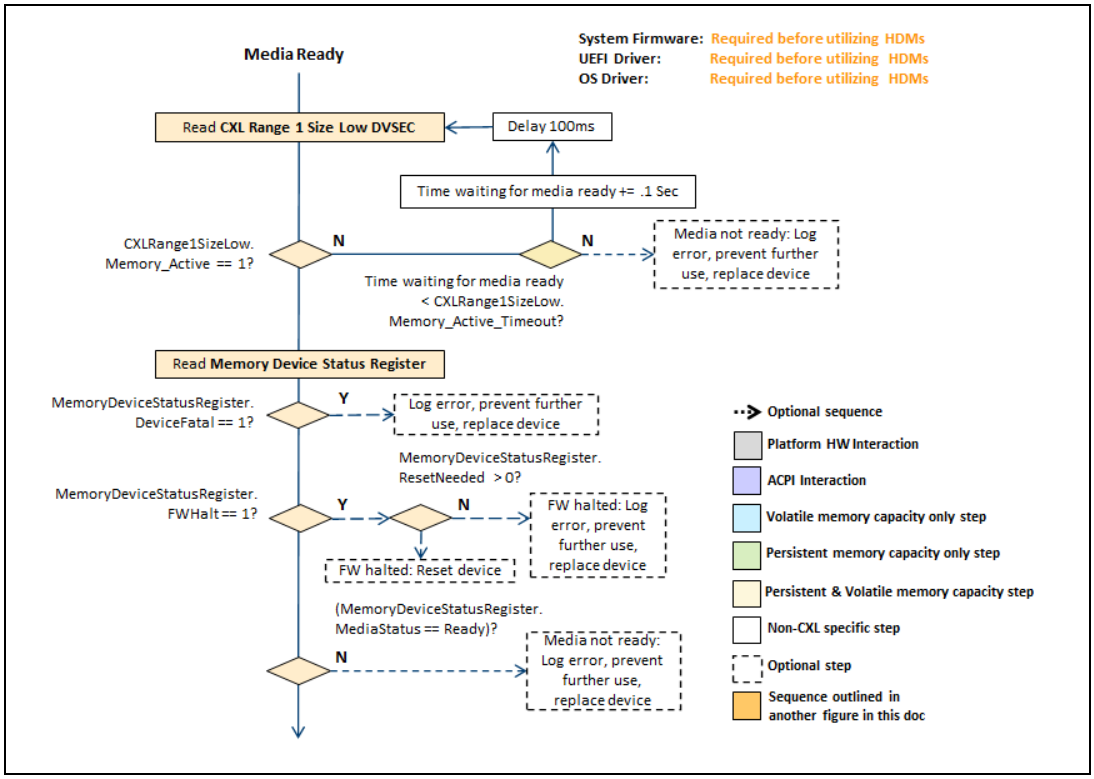
Figure 2-42 High-level Sequence: Device Discovery and Mailbox Ready



2.13.10 Media Ready Sequence

The following figure outlines the basic high-level sequence the system firmware, UEFI and OS drivers will execute to determine if the memory device is ready for media accesses using the HDMs. After this sequence the device's media can be utilized using the HDMs.

Figure 2-43 High-level Sequence: Media Ready

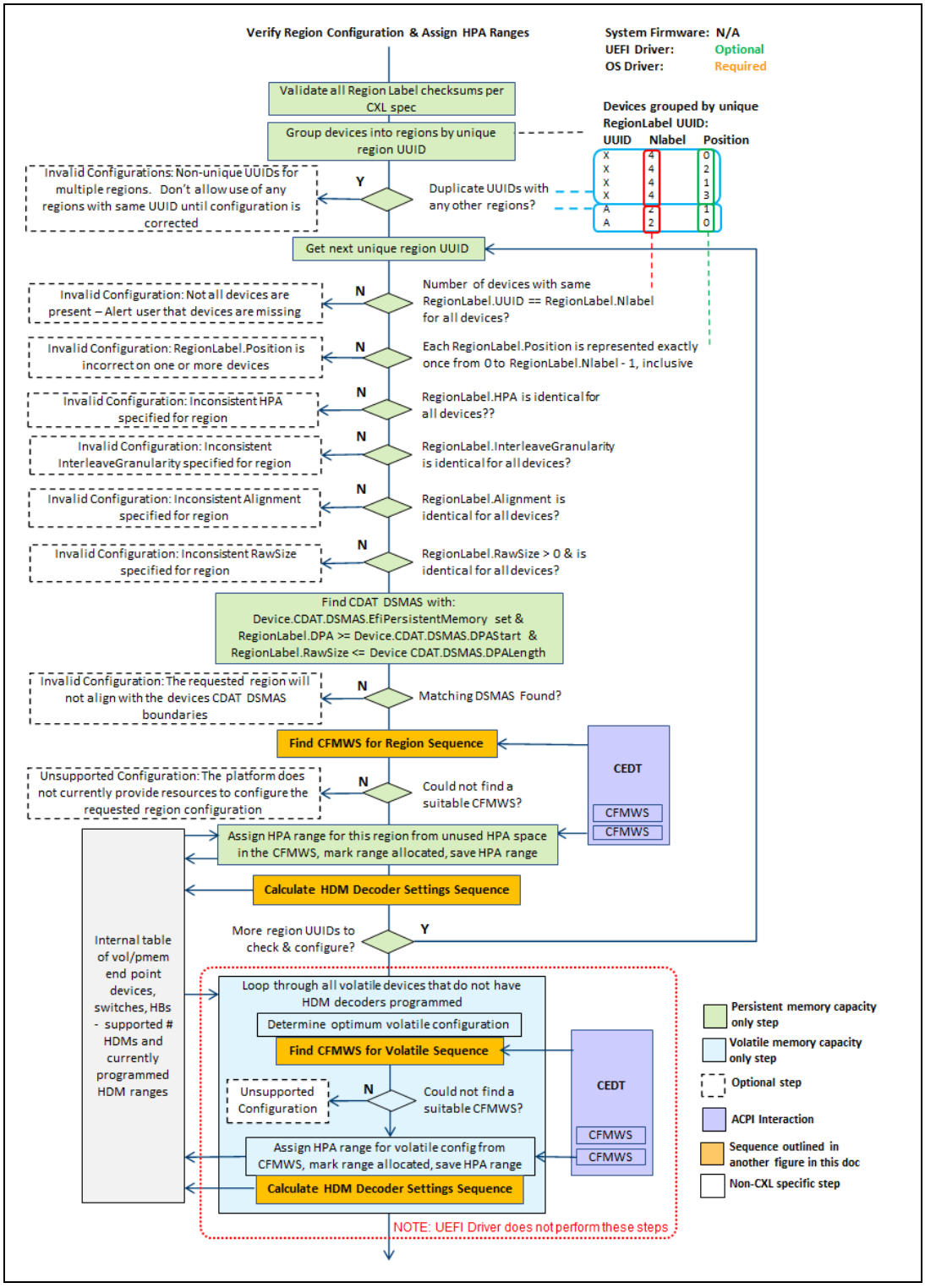


2.13.11 Verifying Region Configuration and Assigning HPA Ranges Sequence

The following sequence outlines the basic steps the OS and UEFI drivers perform when verifying the persistent memory region configuration and assigning HPA ranges from the CEDT. The sequence is performed anytime a persistent memory device is enumerated by the OS and UEFI drivers on every system boot, and by the OS drivers when handling a hot add of a device.

Note that intermediate CXL switches between CXL memory device and the CXL host bridge do not play a part in determining proper region configuration. If the connected devices in the region span multiple CXL host bridges, the OS and UEFI drivers must verify proper device ordering across host bridges. If the correct devices are connected to each host bridge, the order the devices are connected within the host bridge does not matter, as shown in the out of order memory region examples.

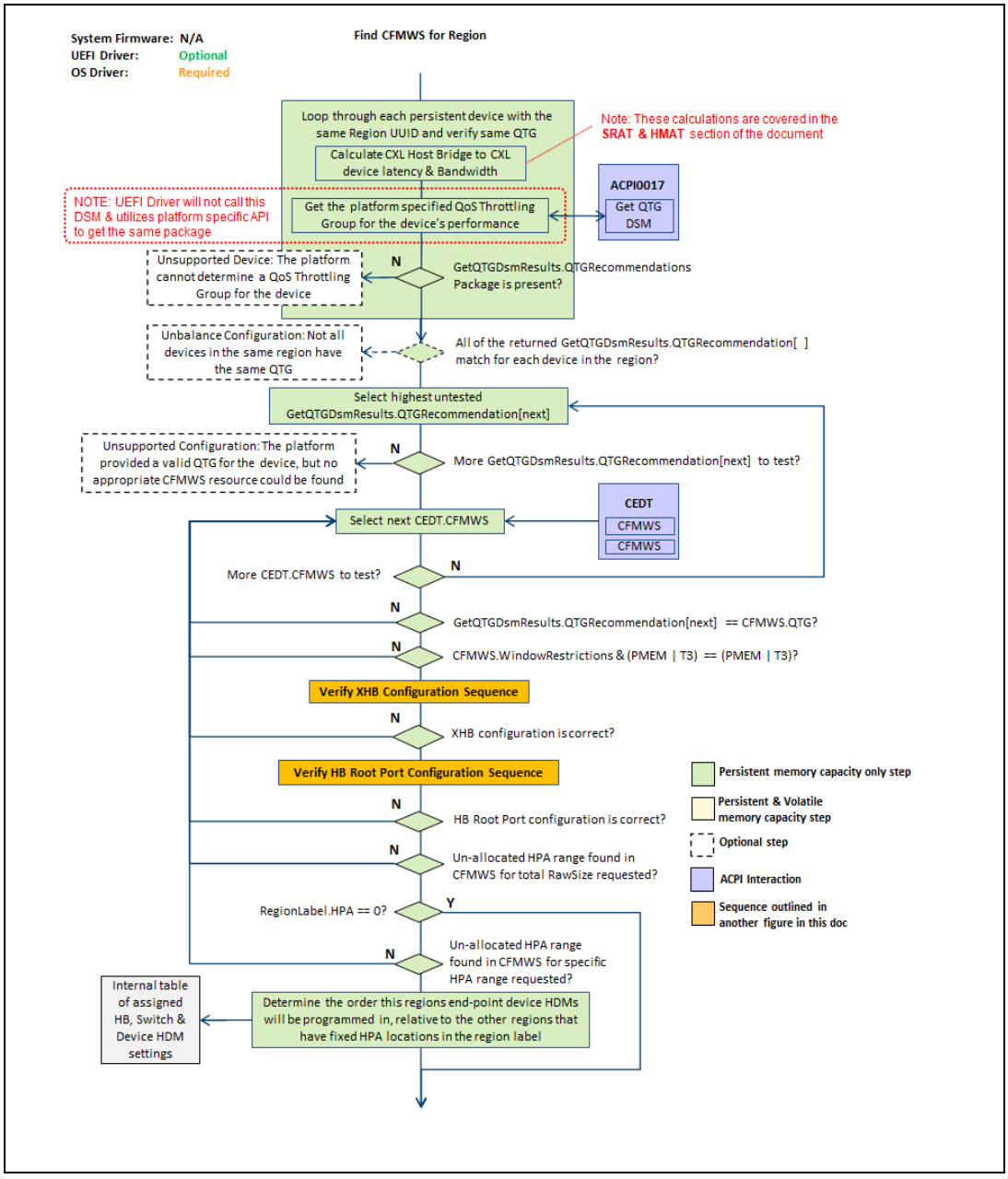
Figure 2-44 High-level Sequence: Verifying Region Configuration



2.13.12 Find CFMWS for Region Sequence

The following figure outlines the basic logic to find a suitable CFMWS for each region being configured. The sequence is performed anytime a persistent memory device is enumerated by the OS and UEFI drivers on every system boot, and by the OS drivers when handling a hot add of a device.

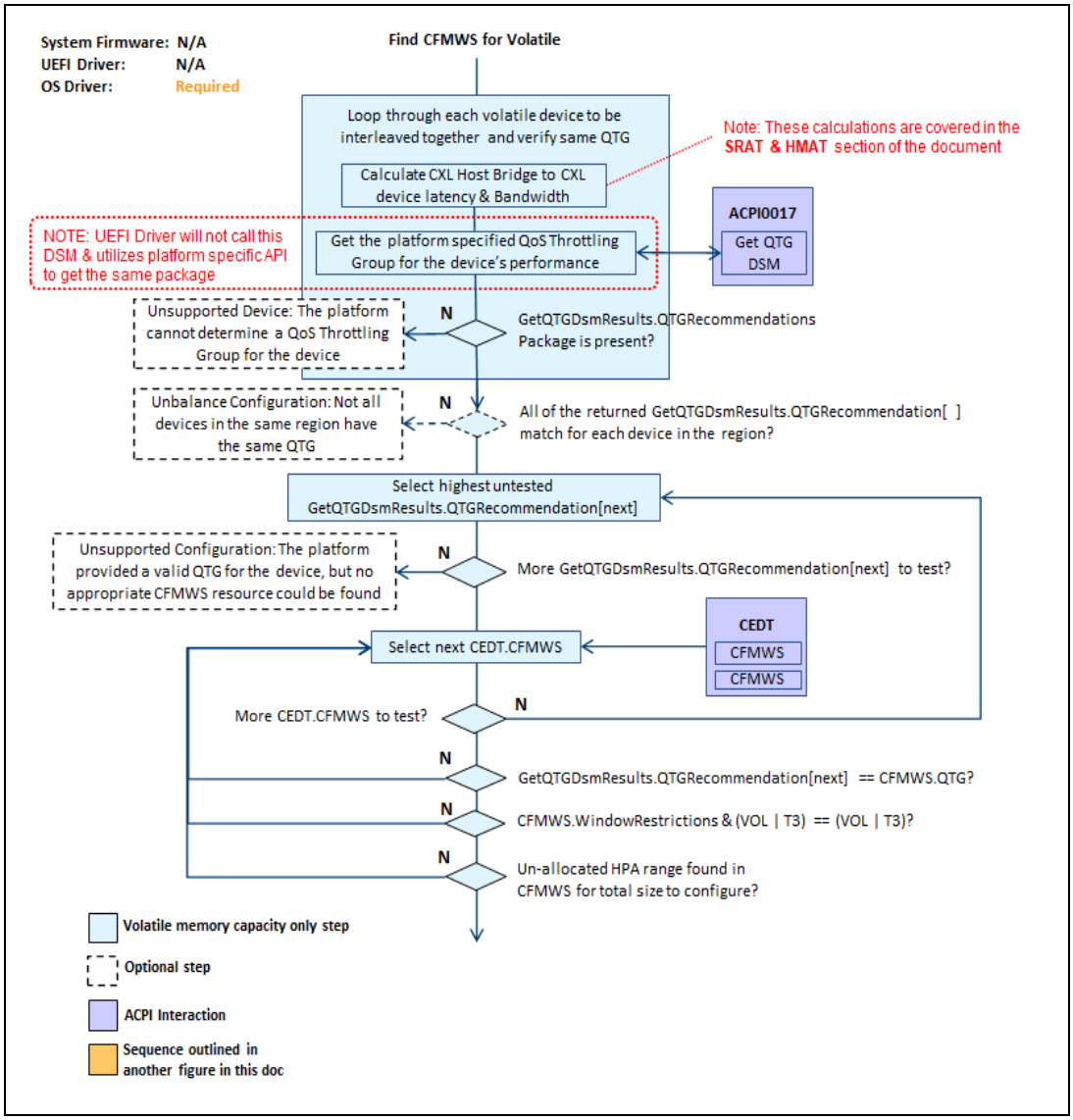
Figure 2-45 High-level Sequence: Finding CFMWS for Region



2.13.13 Find CFMWS for Volatile Sequence

The following figure outlines the basic logic to find a suitable CFMWS for volatile memory being configured by the OS. The sequence is performed anytime a volatile memory device is enumerated by the OS drivers on every system boot, and by the OS drivers when handling a hot add of a device.

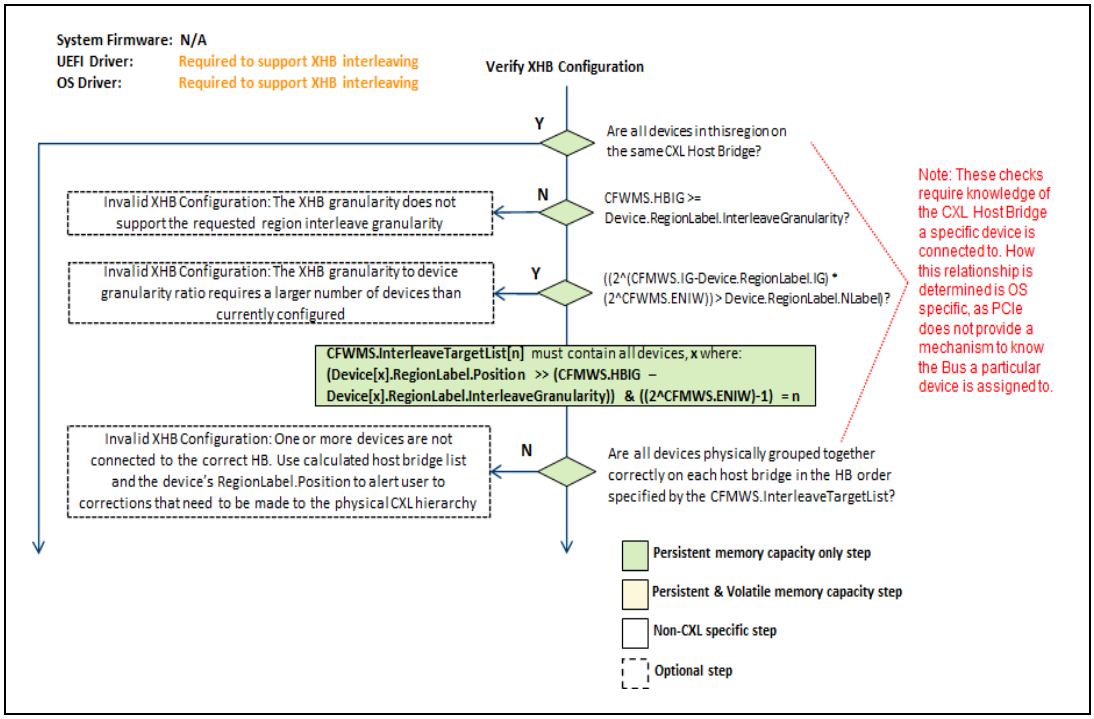
Figure 2-46 High-level Sequence: Finding CFMWS for Volatile



2.13.14 Verify XHB Configuration Sequence

For platforms that support XHB regions, the following sequence outlines the basic steps the OS and UEFI drivers perform when verifying the XHB persistent memory region configuration. The sequence is performed anytime a persistent memory device is enumerated by the OS and UEFI drivers on every system boot, and by the OS drivers when handling a hot add of a device.

Figure 2-47 High-level Sequence: Verify XHB Configuration

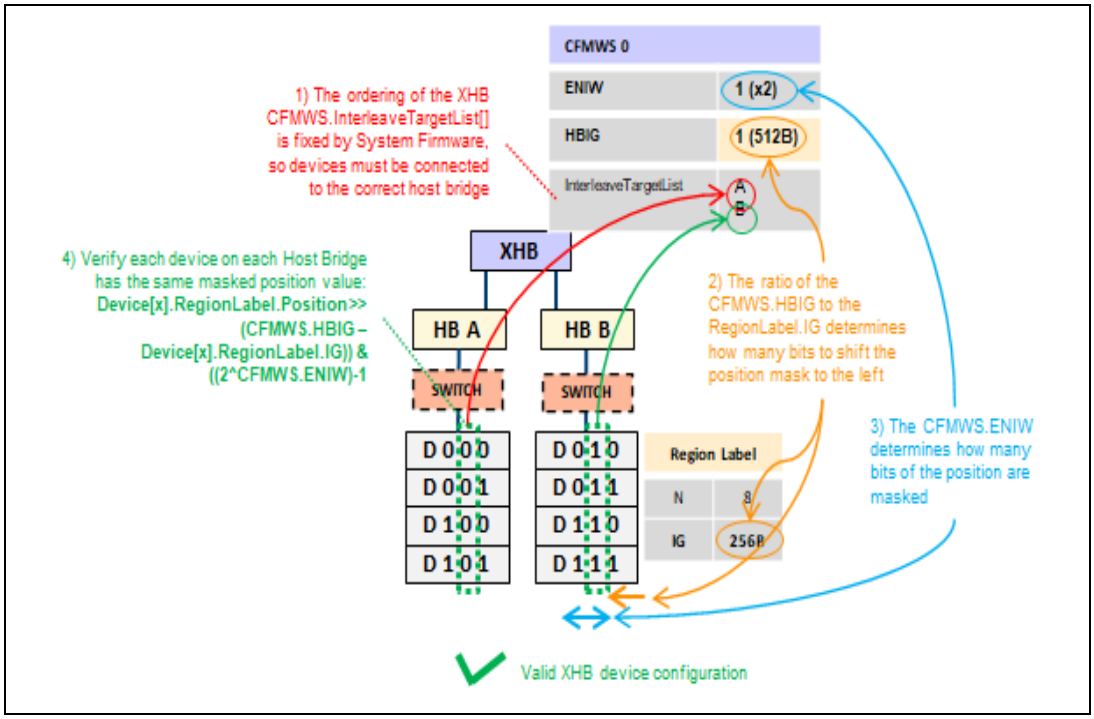


Here are some example region configurations and how this algorithm allows SW to determine correct device placement on each host bridge. This XHB algorithm does not need to check device ordering within a host bridge or intermediate switch that might be present. Those are covered in the following section for CXL host bridge root port ordering checks.

2.13.14.1 x2 XHB Example Configuration Checks

Here is an example valid x2 XHB configuration and illustrated steps this algorithm executes:

Figure 2-48 Example Valid x2 XHB Configuration Execution Steps



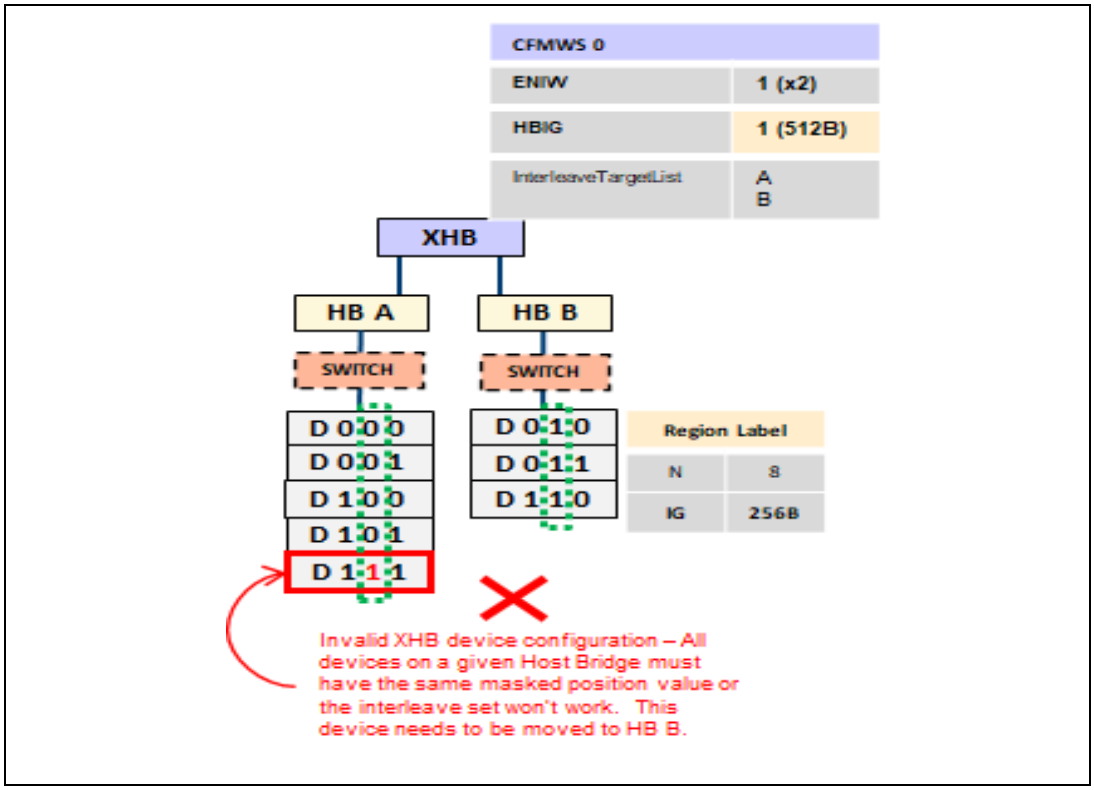
And the resulting calculations used to verify the configuration:

Table 2-7 Valid x2 XHB Configuration

Configuration	XHB CFMWS	Device RegionLabel	RegionLabel. Position	CFMWS.HBIG – Dev.RegionLabel.IG	((2^CFMWS.ENIW) -1))	n
-x2 XHB w x8 device region -4 devices on each HB -Different host bridge and device Interleave Granularity	CFMWS.ENIW = 1 (x2) CFMWS.HBIG = 1 (512B)	NLabel = 8 Interleave Granularity = 0 (256B)	0000	1-0=1	2^1-1=1	0
			0001			0
			0010			1
			0011			1
			0100			0
			0101			0
			0110			1
			0111			1

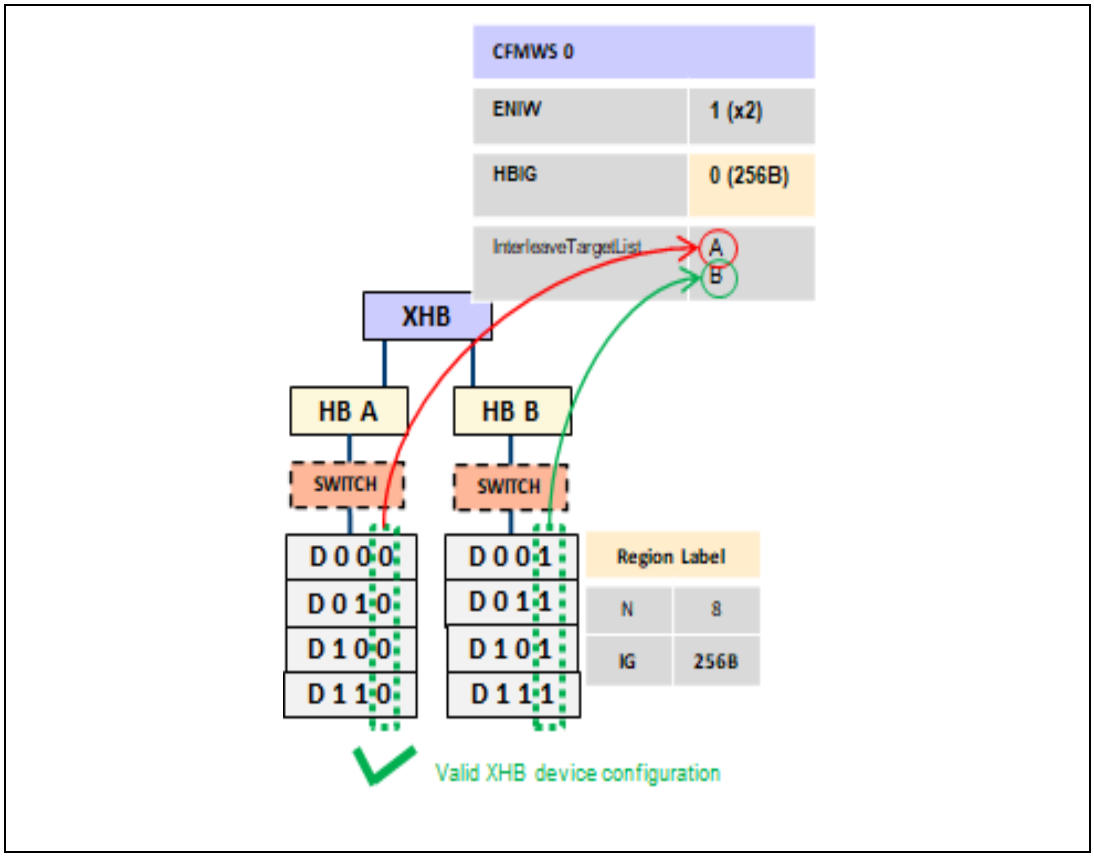
Example of an invalid x2 XHB configuration that this check would catch:

Figure 2-49 Example Invalid x2 XHB Configuration



Here is another example valid x2 XHB configuration:

Figure 2-50 Example Valid x2 XHB Configuration



And the resulting calculations used to verify the configuration:

Table 2-8 Valid x2 XHB Configuration 2

Configuration	XHB CFMWS	Device RegionLabel	RegionLabel. Position	CFMWS.HBIG – Dev.RegionLabel.IG	((2^CFMWS.ENIW) -1))	n
-x2 XHB w x8 device region -4 devices on each HB -Same host bridge and device Interleave Granularity	CFMWS.ENIW = 1 (x2) CFMWS.HBIG = 0 (256B)	NLabel = 8 Interleave Granularity = 0 (256B)	0000	0-0=0	2^1-1=1	0
			0001			1
			0010			0
			0011			1
			0100			0
			0101			1
			0110			0
			0111			1

2.13.14.2 x4 XHB Example Configuration Checks

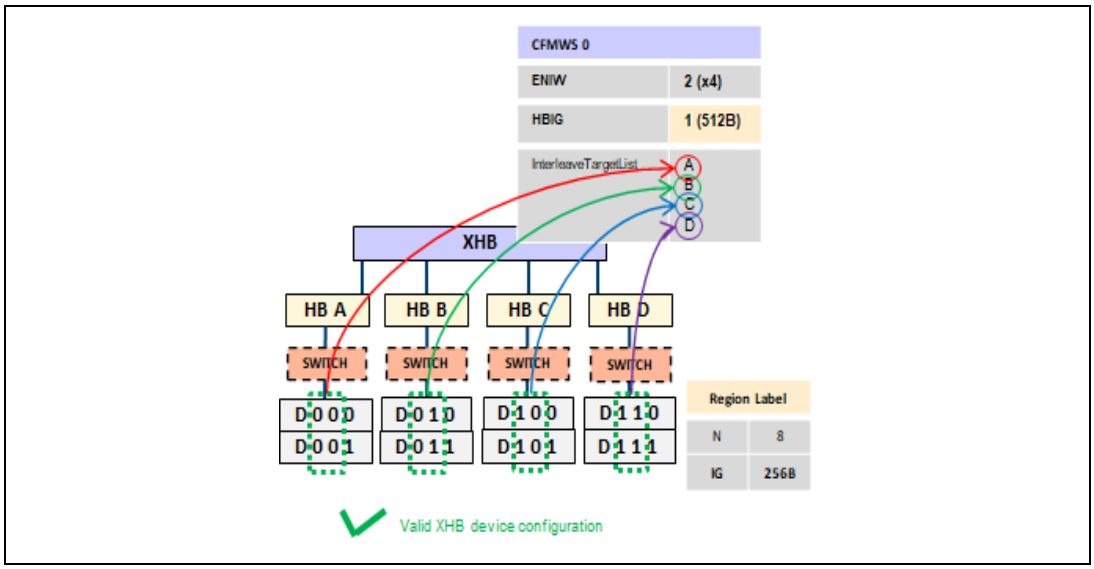
Here is an invalid x4 XHB configuration example that would require more devices than are connected:

Table 2-9 Invalid x4 xHB Configuration

Configuration	XHB CFMWS	Device RegionLabel	RegionLabel. Position	CFMWS.HBIG – Dev.RegionLabel.IG	((2^CFMWS.ENIW) -1))	n
-x4 XHB w x8 device region -2 devices on each HB -Different host bridge and device Interleave Granularity	CFMWS.ENIW = 2 (x4) CFMWS.HBIG = 2 (1K)	NLabel = 8 Interleave Granularity = 0 (256B)	Invalid configuration: The CFMWS.IG will send 1K down each HB so with 256 for device IG there would need to be 1K/256B = 4 devices on each HB which is 16 devices and is > Dev.RegionLabel.NLabel. This is caught by the following test in the previous flow: If ((2^(CFMWS.IG-Dev.RegionLabel.IG) * (2^CFWMS.ENIW)) > Dev.RegionLabel.NLabel) //invalid configuration			

Here is an example valid x4 XHB configuration:

Figure 2-51 Example Valid x4 XHB Configuration



And the resulting calculations used to verify the configuration:

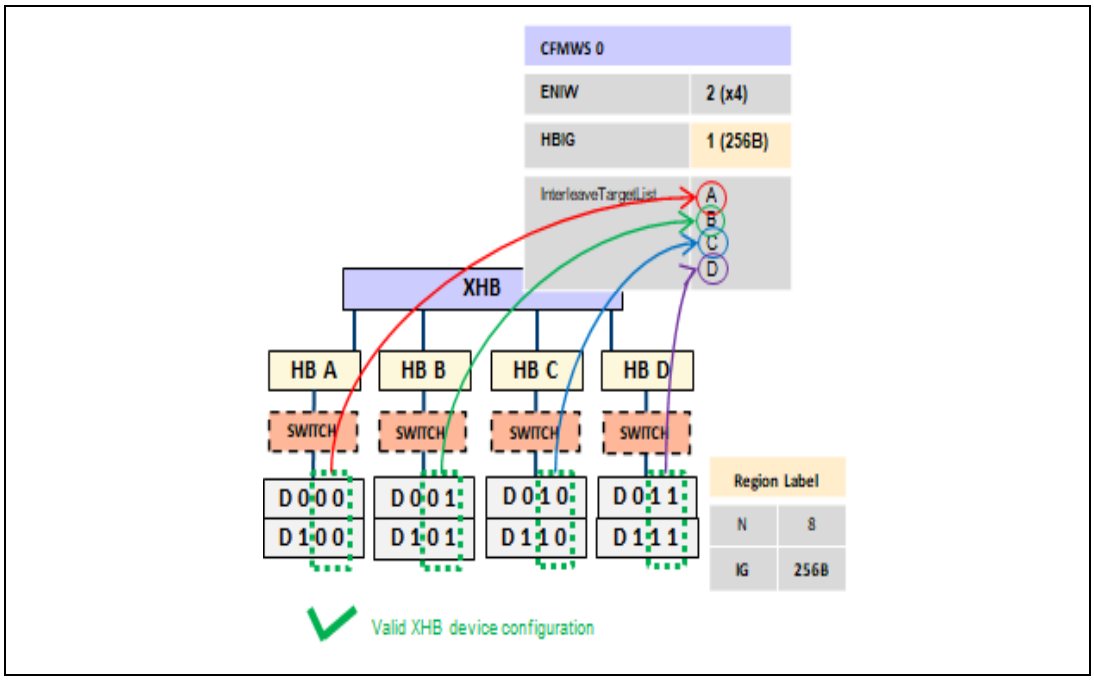
Table 2-10 Valid x4 XHB Configuration

Configuration	XHB CFMWS	Device RegionLabel	RegionLabel. Position	CFMWS.HBIG – Dev.RegionLabel.IG	((2^CFMWS.ENIW) -1))	n
-x4 XHB w x8 device region	CFMWS.ENIW = 2 (x4) CFMWS.HBIG = 1 (512B)	NLabel = 8 Interleave Granularity = 0 (256B)	0000	1-0=1	2^2-1=11b	00
			0001			00
			0010			01
			0011			01
			0100			10
			0101			10

Configuration	XHB CFMWS	Device RegionLabel	RegionLabel. Position	CFMWS.HBIG – Dev.RegionLabel.IG	((2^CFMWS.ENIW) -1))	n
-2 devices on each HB -Different host bridge and device Interleave Granularity			0110			11
			0111			11

Here is another example valid x4 XHB configuration:

Figure 2-52 Example Valid x4 XHB Configuration



And the resulting calculations used to verify the configuration:

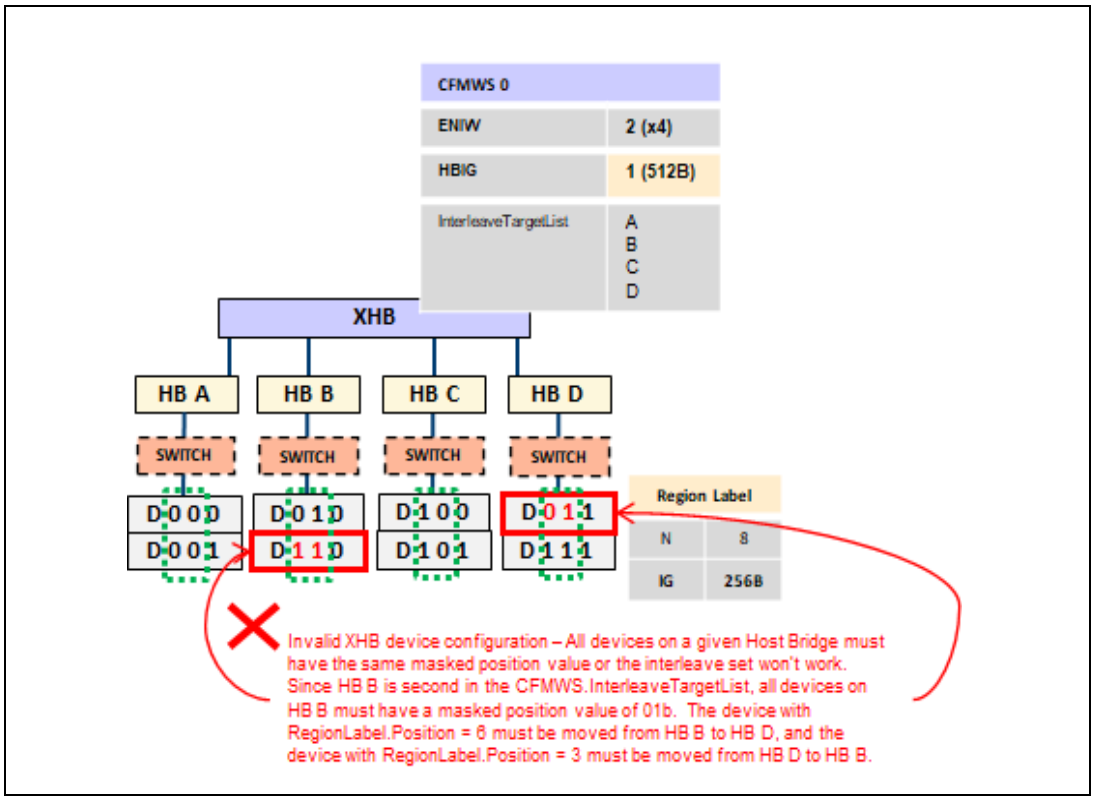
Table 2-11 Valid x4 XHB Configuration 2

Configuration	XHB CFMWS	Device RegionLabel	RegionLabel. Position	CFMWS.HBIG – Dev.RegionLabel.IG	((2^CFMWS.ENIW) -1))	n
-x4 XHB w x8 device region -2 devices on each HB -Same host bridge and device Interleave Granularity	CFMWS.ENIW = 2 (x4) CFMWS.HBIG = 0 (256B)	NLabel = 8 Interleave Granularity = 0 (256B)	0000	0-0=0	2^2-1=11b	00
			0001			01
			0010			10
			0011			11
			0100			00
			0101			01
			0110			10

Configuration	XHB CFMWS	Device RegionLabel	RegionLabel. Position	CFMWS.HBIG – Dev.RegionLabel.IG	((2^CFMWS.ENIW)-1))	n
			0111			11

Example of an invalid x4 XHB configuration that this check would catch:

Figure 2-53 Example Invalid x4 XHB Configuration



2.13.14.3 x8 XHB Example Configuration Checks

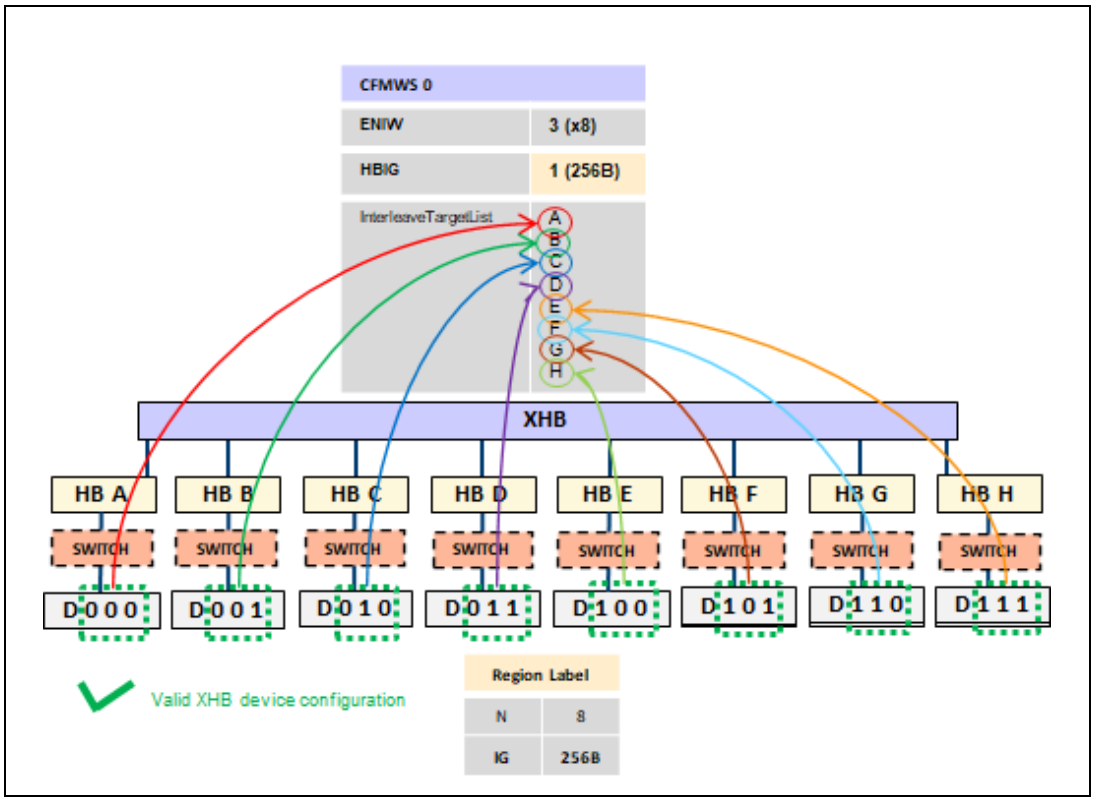
Here is an invalid x8 XHB configuration example that would require more devices than are connected:

Table 2-12 Invalid x8 xHB Configuration

Configuration	XHB CFMWS	Device RegionLabel	RegionLabel. Position	CFMWS.HBIG – Dev.RegionLabel.IG	((2^CFMWS.ENIW) -1))	n
-x8 XHB w x8 device region -1 devices on each HB -Different host bridge and device Interleave Granularity	CFMWS.ENIW = 3 (x8) CFMWS.HBIG = 2 (1K)	NLabel = 8 Interleave Granularity = 0 (256B)	Invalid configuration: The CFMWS.IG will send 512B down each HB so with 256 for device IG there would need to be 1K/256B = 4 devices on each HB which is 16 devices and is > Dev.RegionLabel.NLabel. This is caught by the following test in the previous flow: If ((2^(CFMWS.IG-Dev.RegionLabel.IG) * (2^CFWMS.ENIW)) > Dev.RegionLabel.NLabel) //invalid configuration			

Here is an example valid x8 XHB configuration:

Figure 2-54 Example Valid x8 XHB Configuration



And the resulting calculations used to verify the configuration:

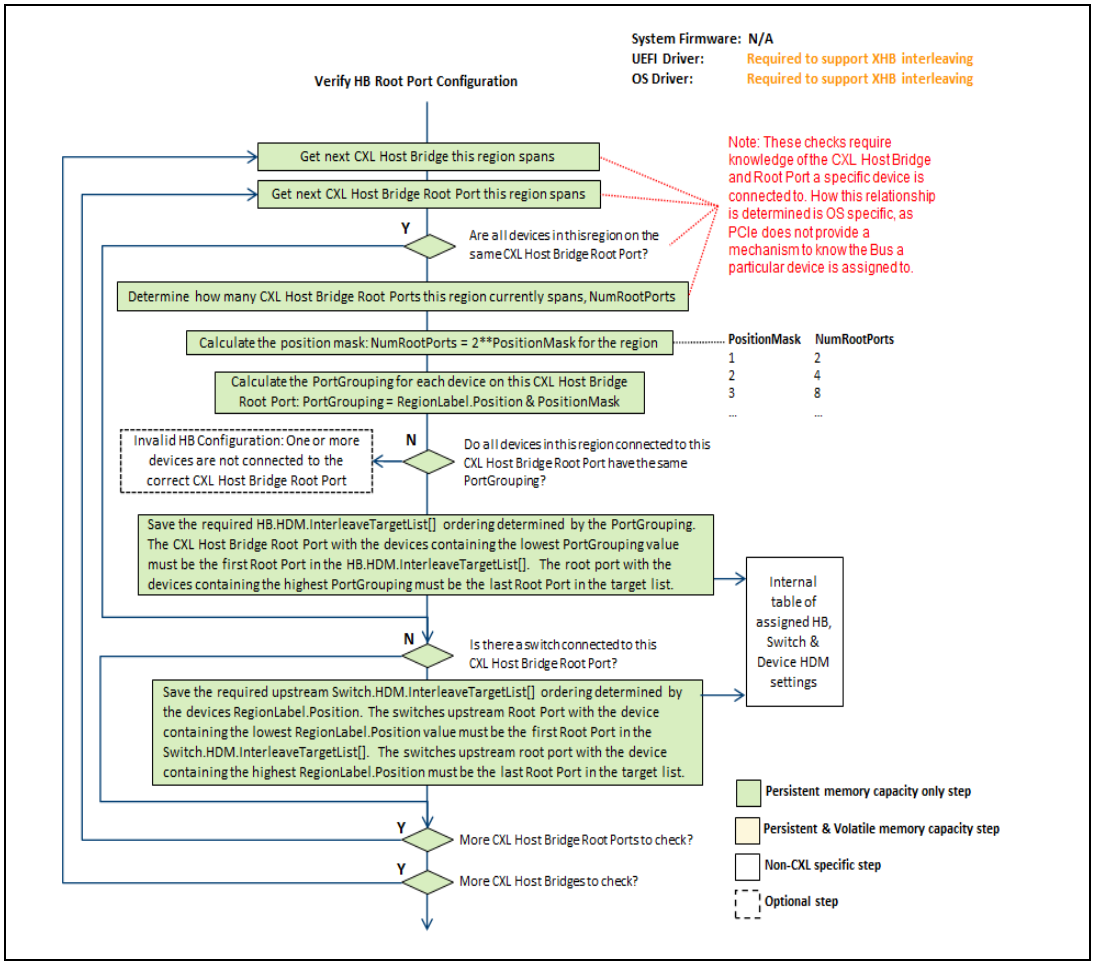
Table 2-13 Valid x8 XHB Configuration

Configuration	XHB CFMWS	Device RegionLabel	RegionLabel. Position	CFMWS.HBIG – Dev.RegionLabel.IG	((2^CFMWS.ENIW) - 1))	n
-x8 XHB w x8 device region -1 device on each HB -Same host bridge and device Interleave Granularity	CFMWS.ENIW = 3 (x8) CFMWS.HBIG = 0 (256B)	NLabel = 8 Interleave Granularity = 0 (256B)	0000	0-0=0	2^3-1=111b	000
			0001			001
			0010			010
			0011			011
			0100			100
			0101			101
			0110			110
			0111			111

2.13.15 Verify HB Root Port Configuration Sequence

The following sequence outlines the basic steps the OS and UEFI drivers perform when verifying that each device in each region is grouped correctly on each CXL host bridge root port of the requested region configuration. The sequence is performed anytime a persistent memory device is enumerated by the OS and UEFI drivers on every system boot, and by the OS drivers when handling a hot add of a device.

Figure 2-55 High-level Sequence: Verify HB Root Port Configuration

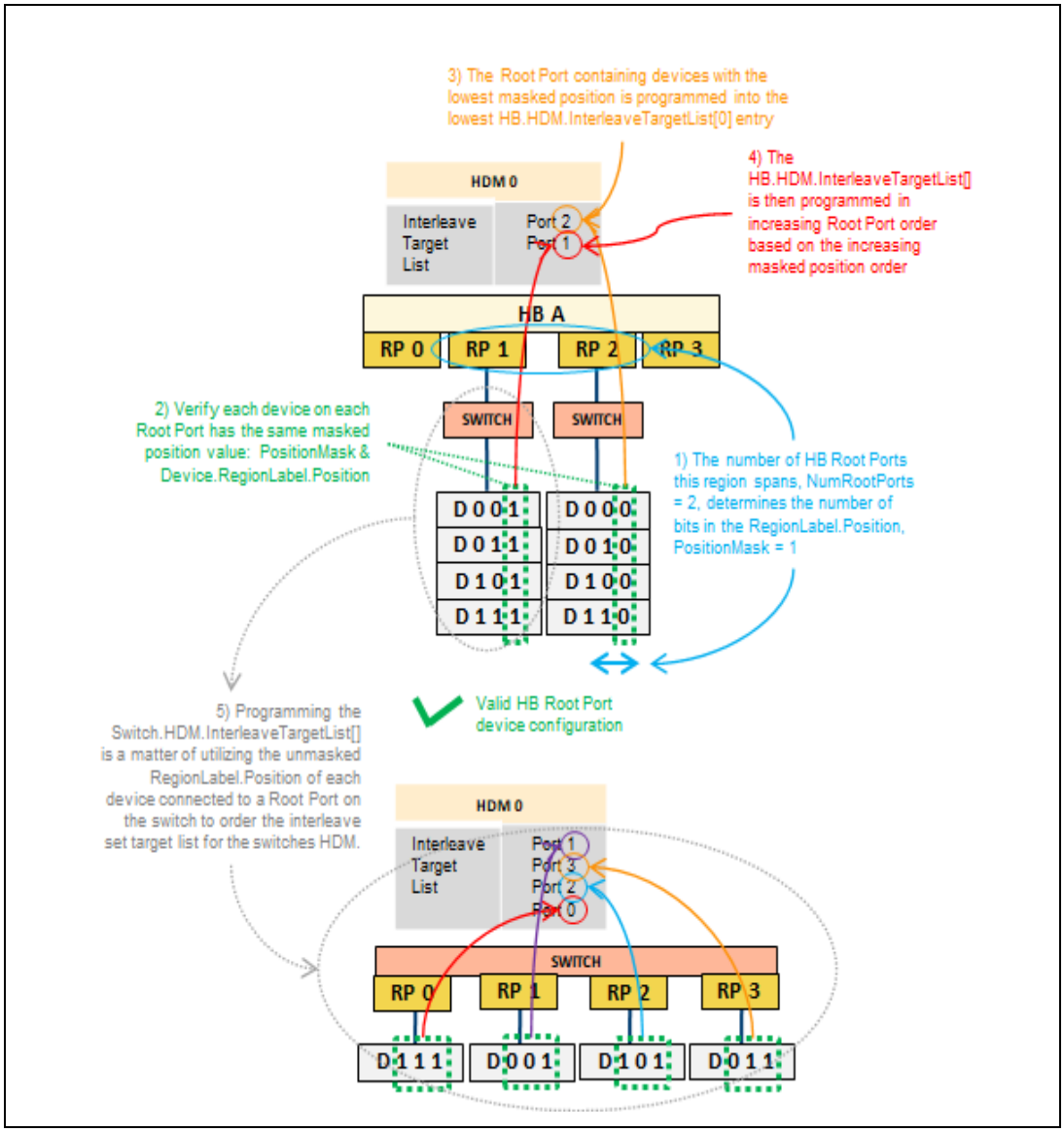


The following sections outline examples of the previous CXL host bridge root port configuration checks required for persistent memory regions.

2.13.15.1 Region Spanning 2 HB Root Ports Example Configuration Checks

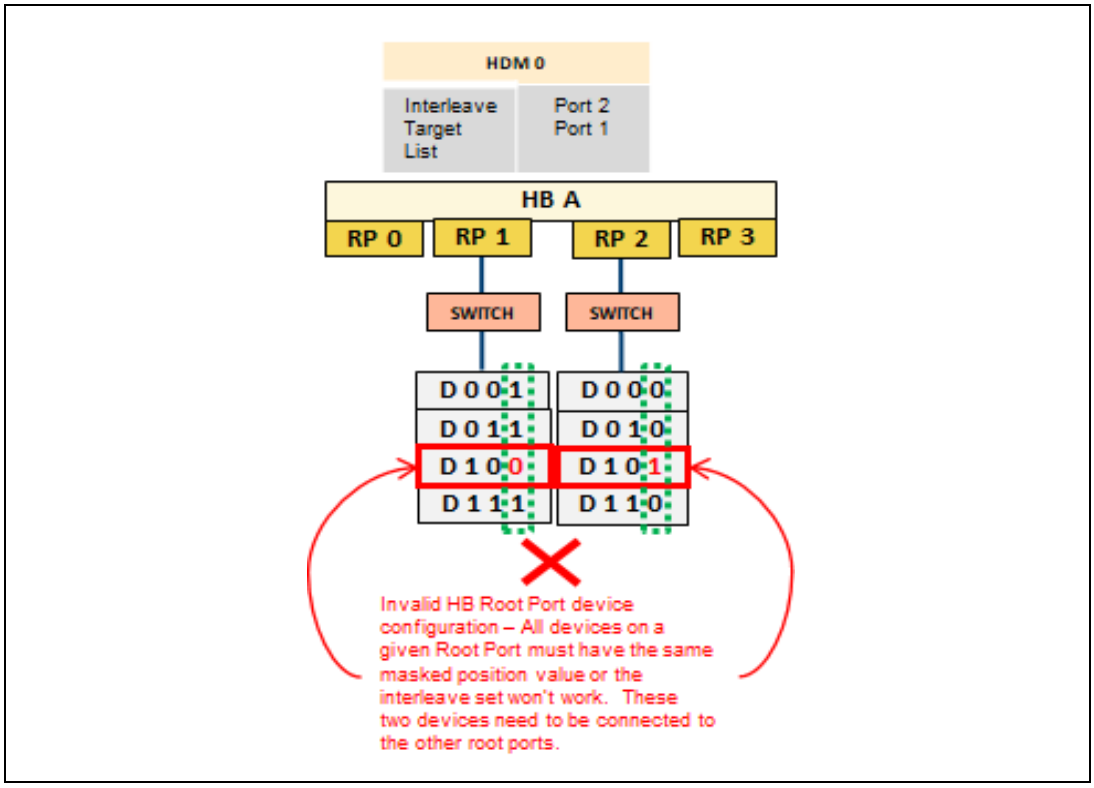
Here is an example valid region spanning 2 HB root ports and illustrated steps this algorithm executes:

Figure 2-56 Example Valid Region Spanning 2 HB Root Ports



Example of an invalid region spanning 2 HB root ports that this check would catch:

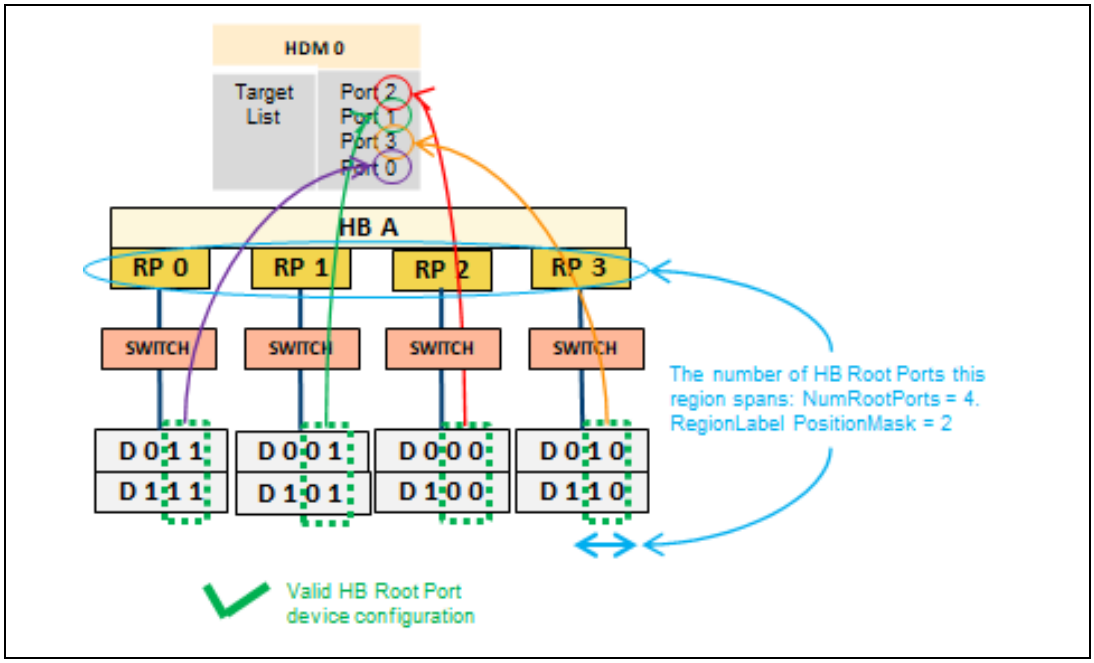
Figure 2-57 Example Invalid Region Spanning 2 HB Root Ports



2.13.15.2 Region Spanning 4 Root Ports Example Configuration Checks

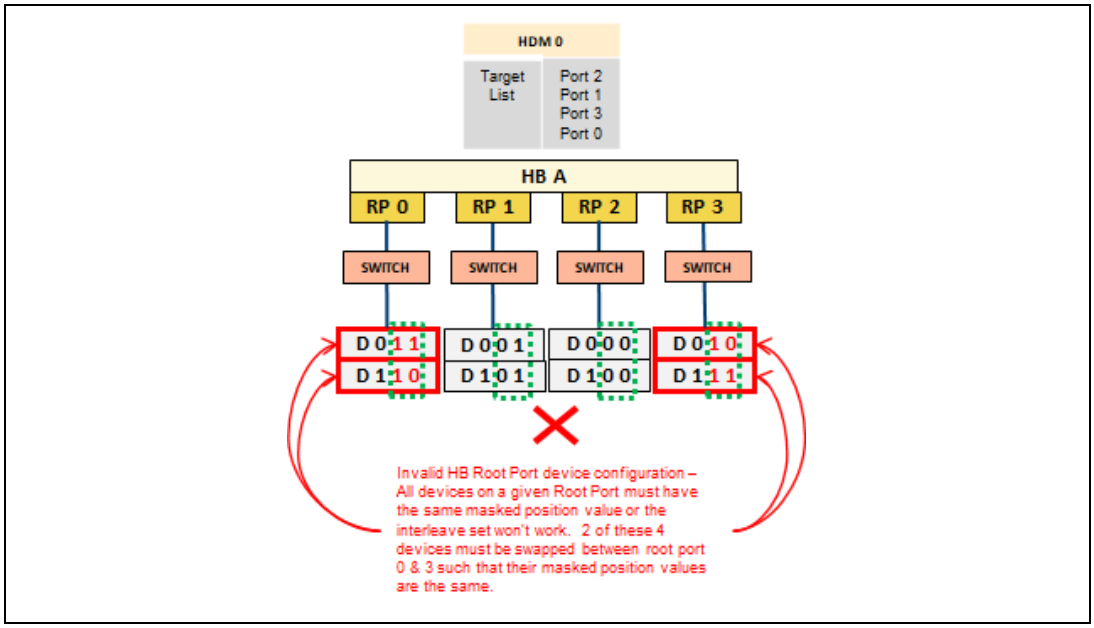
Here is an example valid region spanning 4 HB root ports:

Figure 2-58 Example Valid Region Spanning 4 HB Root Ports



Example of an invalid region spanning 2 HB root ports that this check would catch:

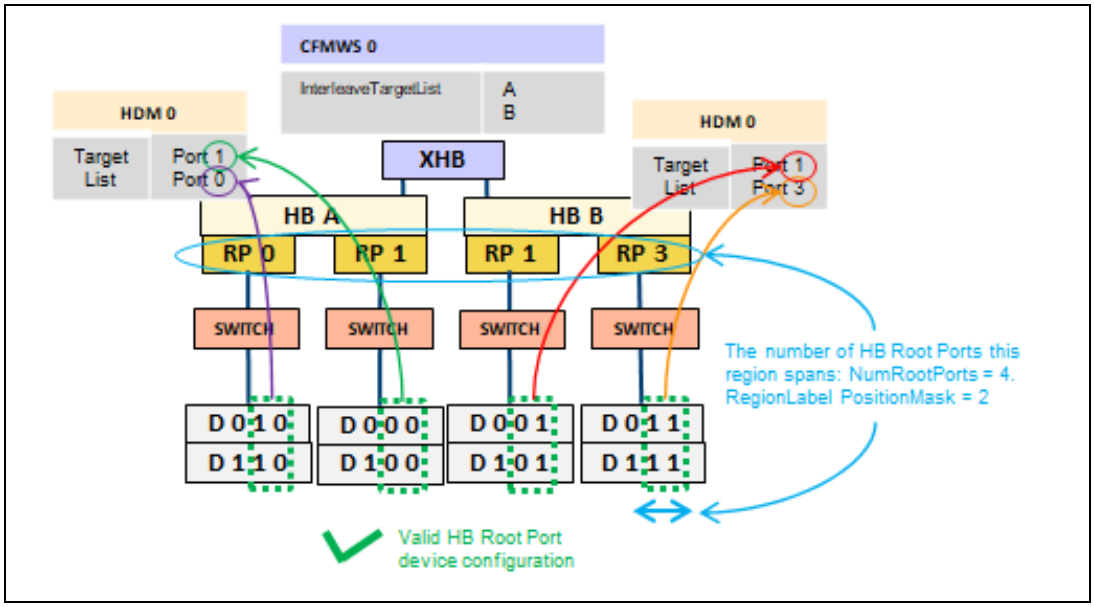
Figure 2-59 Example Invalid Region Spanning 4 HB Root Ports



2.13.15.3 Region Spanning 4 HB Root Ports and x2 XHB Example Configuration Checks

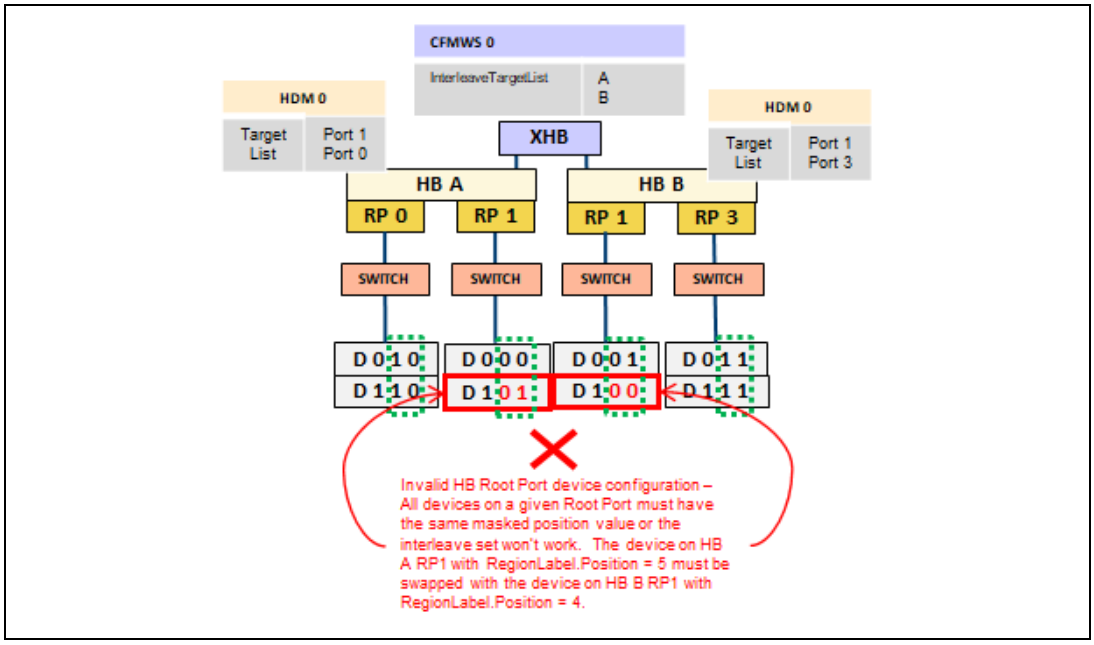
Here is an example valid region spanning 4 HB root ports on a x2 XHB:

Figure 2-60 Example Valid Region Spanning 4 HB Root Ports on a x2 XHB



Example of an invalid region spanning 2 HB root ports on a x2 XHB that this check would catch:

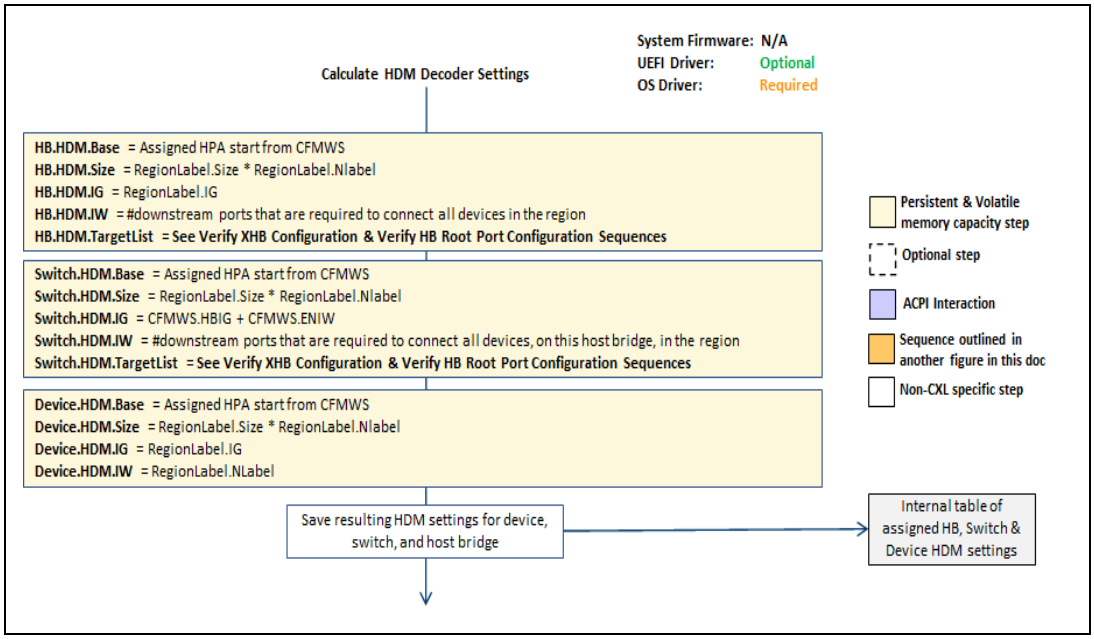
Figure 2-61 Example Invalid Region Spanning 4 HB Root Ports on a x2 XHB



2.13.16 Calculate HDM Decoder Settings Sequence

The following figure outlines the basic high-level sequence the UEFI and OS drivers will execute in preparation for programming the HDM decoders.

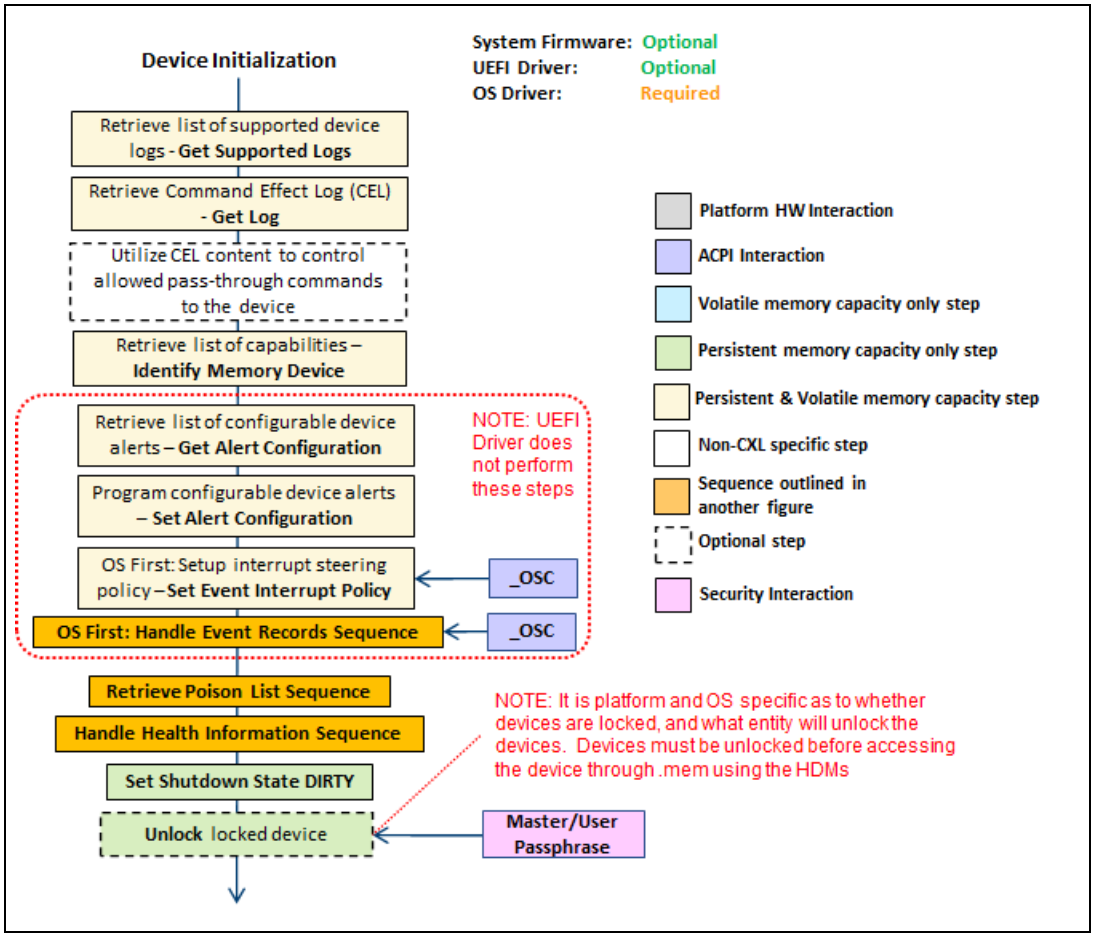
Figure 2-62 High-level Sequence: Calculate HDM Decoder Settings



2.13.17 Device Initialization Sequence

The following figure outlines the basic high-level sequence the platform UEFI and OS drivers will execute to handle the memory device initialization.

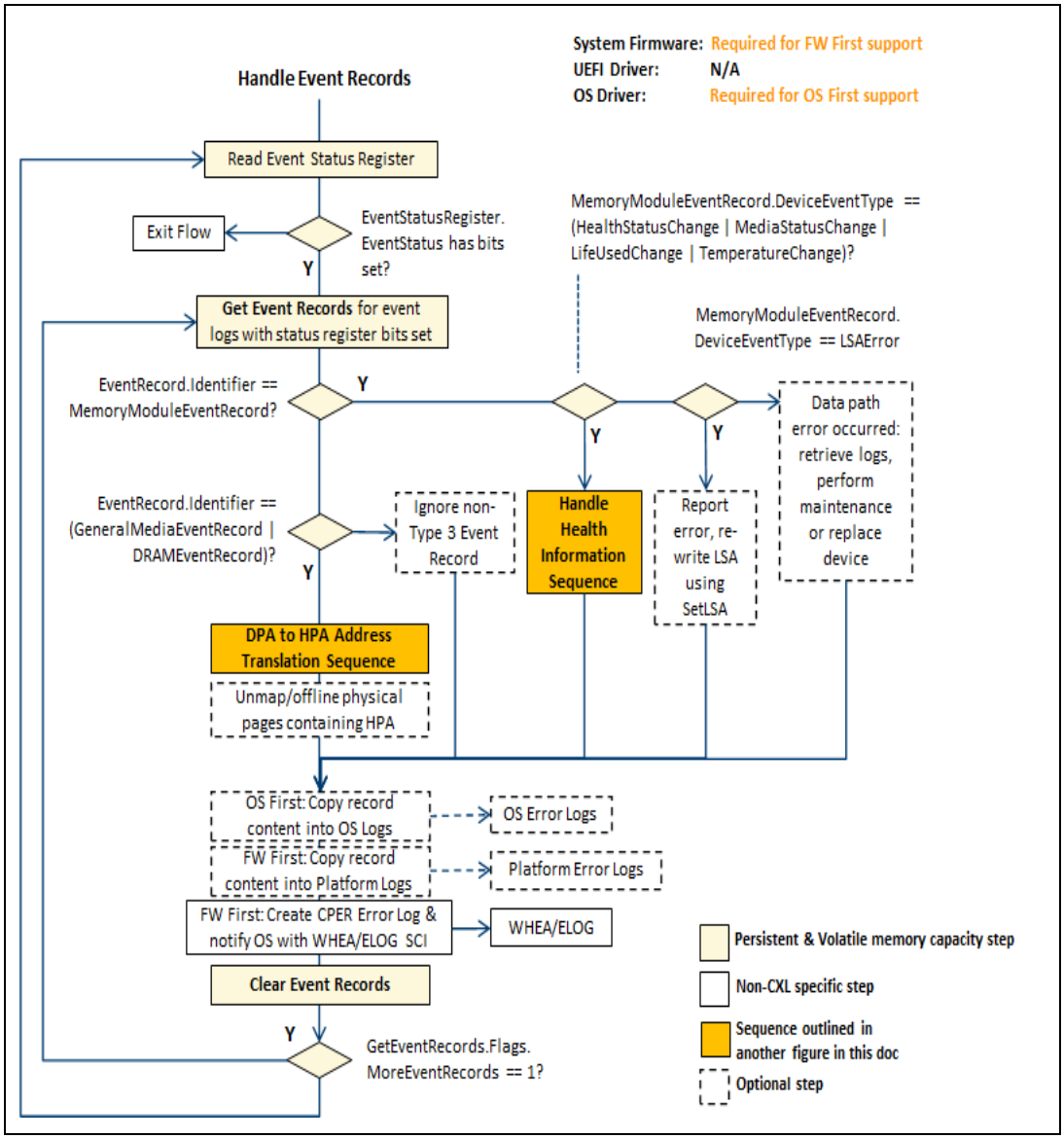
Figure 2-63 High-level Sequence: Device Initialization



2.13.18 Handle Event Records Sequence

The following figure outlines the basic high-level flow that the System Firmware and OS drivers will need to execute to retrieve outstanding event records from each of the device's event logs.

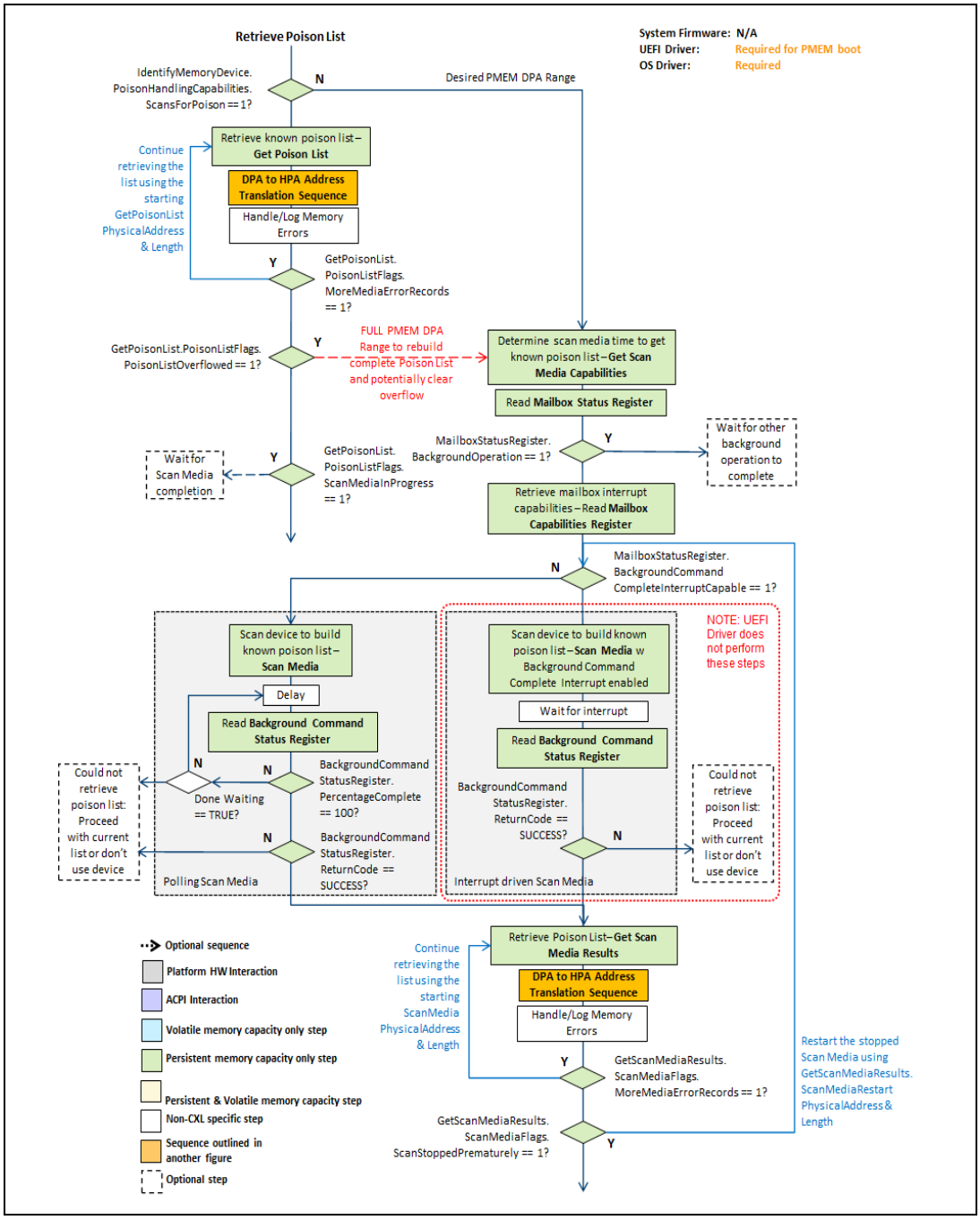
Figure 2-64 High-level Sequence: Handle Event Records



2.13.19 Retrieve Poison List Sequence

The following figure outlines the basic high-level sequence the UEFI and OS drivers will execute to retrieve the list of poisoned address locations the CXL end point device is tracking.

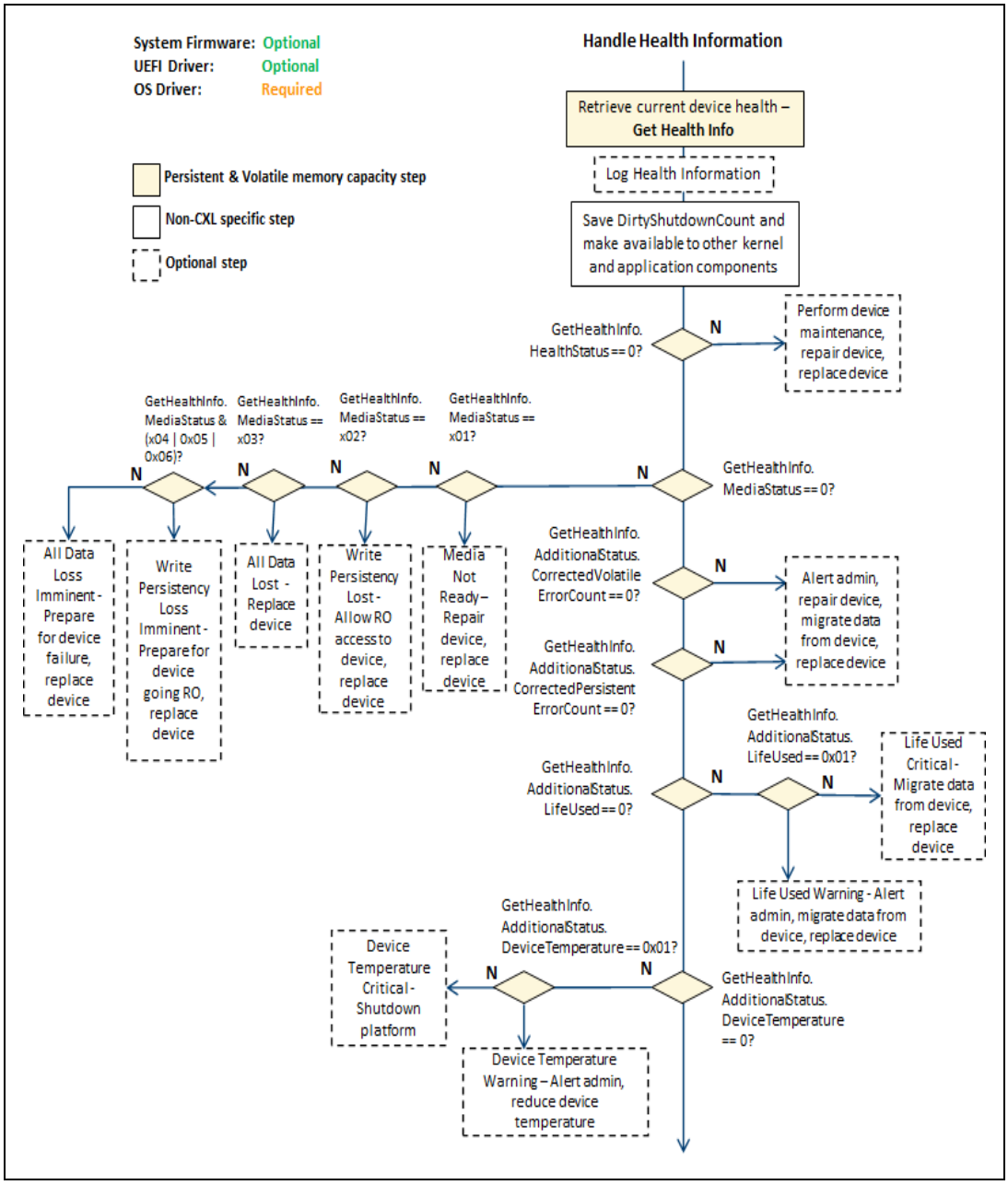
Figure 2-65 High-level Sequence: Retrieve Poison List



2.13.20 Handle Health Information Sequence

The following figure outlines the basic high-level sequence the system firmware, UEFI and OS drivers will execute to handle the memory device's initial health information.

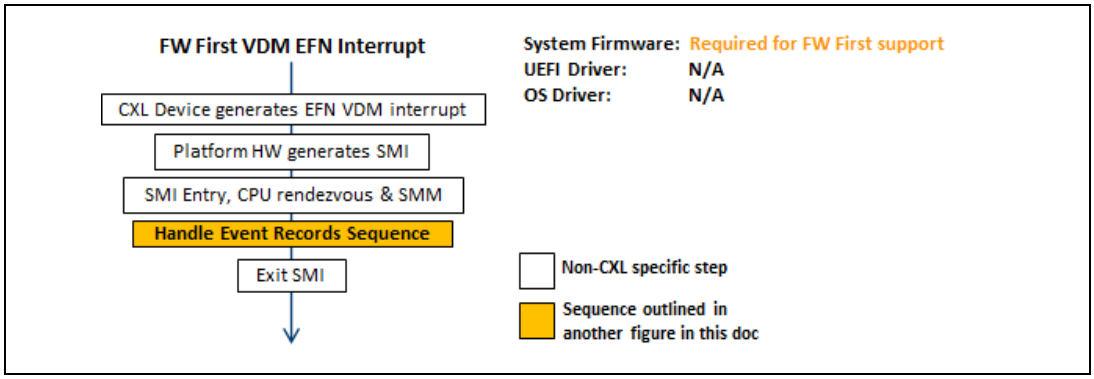
Figure 2-66 High-level Sequence: Handle Health Information



2.13.21 FW First Event Interrupt Sequence

The following figure outlines the basic high-level FW first asynchronous interrupt sequence.

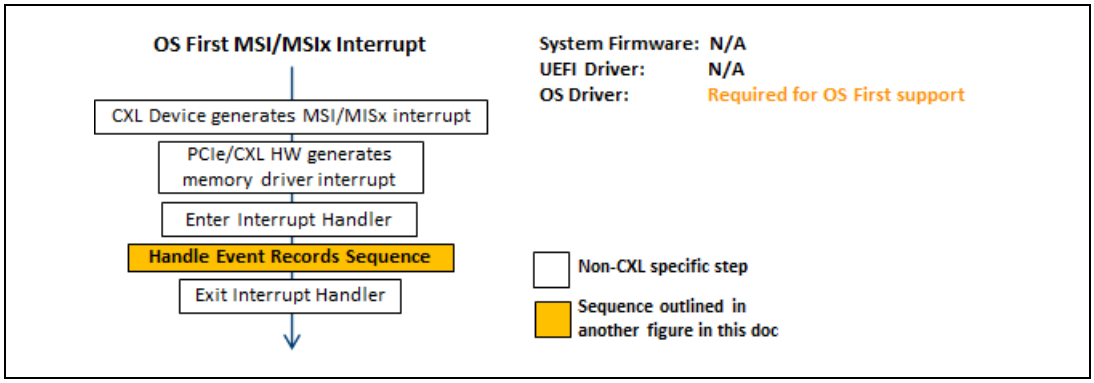
Figure 2-67 High-level Sequence: FW First Event Interrupt



2.13.22 OS First Event Interrupt Sequence

The following figure outlines the basic high-level OS first asynchronous interrupt sequence.

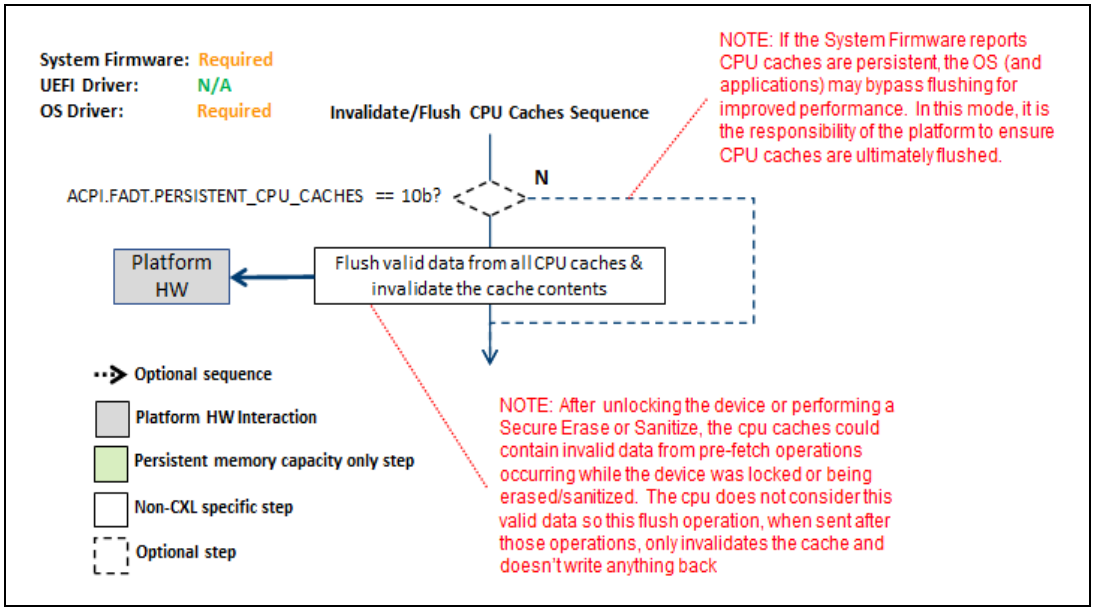
Figure 2-68 High-level Sequence: OS First Event Interrupt



2.13.23 Invalidating/Flushing CPU Caches Sequence

The following figure outlines the basic high-level flow that the system firmware and OS drivers will need to execute to invalidate or flush CPU caches to persistent memory.

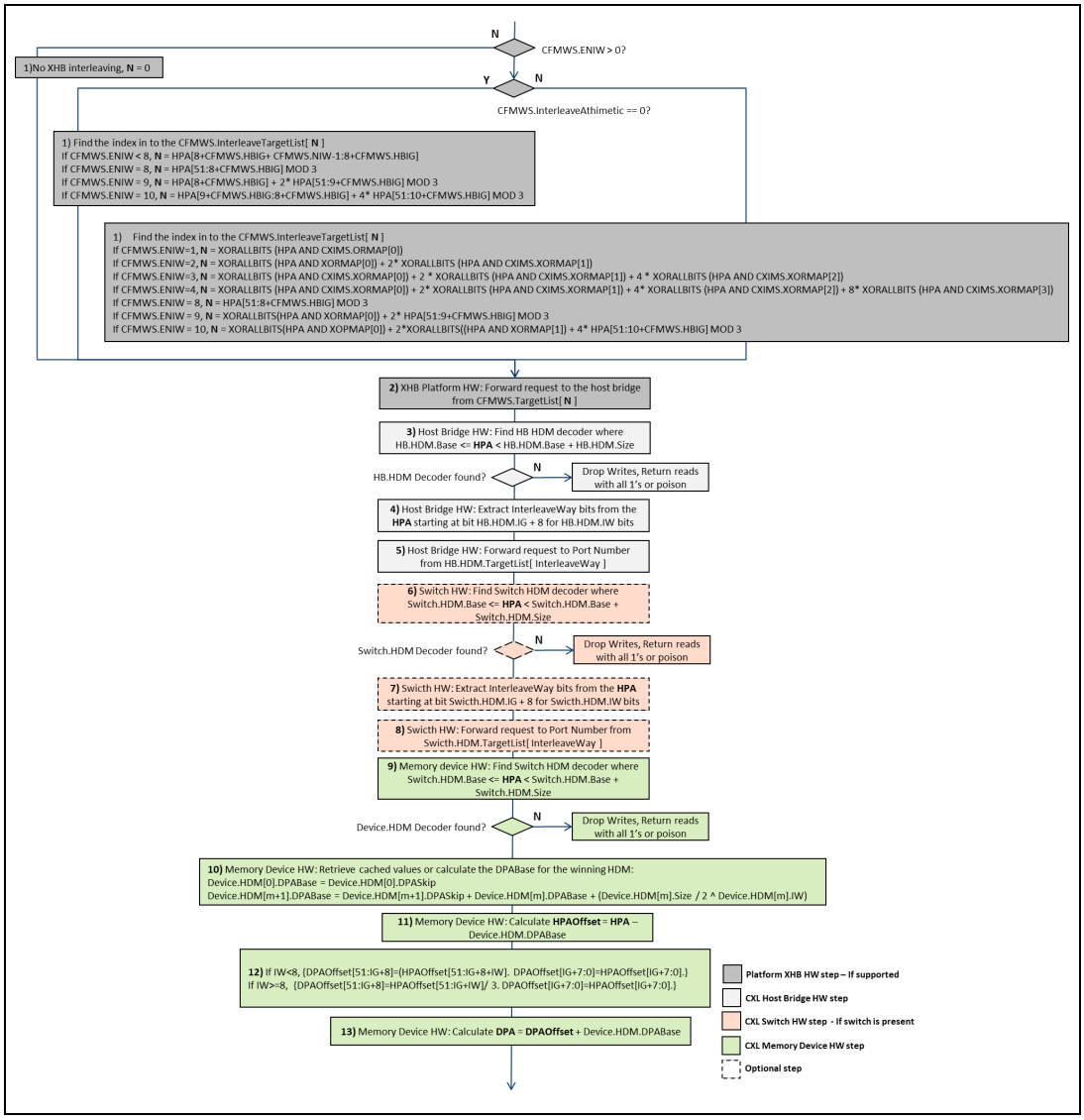
Figure 2-69 High-level Sequence: Invalidating/Flushing CPU Caches



2.13.24 HPA to DPA Translation Sequence

The following figure outlines the host physical address to device physical address translation that occurs in the Platform XHB HW and HDM decoder HW for the CXL host bridge, CXL switch, and CXL memory device. This is the logic outlined in the *CXL Specification*.

Figure 2-70 High-level Sequence: HPA to DPA Translation



Here are some example HPA to DPA translations referencing the steps in the previous flow.

Table 2-14 HPA to DPA Translation 1

HPA to DPA Translation	Example 1	Example 2
Configuration description	See Memory Region Example – Region interleaved across 2 CXL switches for configuration -1 HB -2 Switch -4 devices on each HB -x8 pmem interleave set	See Memory Region Example – Region interleaved across 2 HBs for configuration -x2 XHB interleave -4 devices on each HB -x8 pmem interleave set

HPA to DPA Translation	Example 1	Example 2
Platform HW XHB configuration	N/A	CFMWS.Base 0 CFMWS.WindowsSize 1GB CFMWS.ENIW 1 (x2) CFMWS.HBIG 2 (1K) Target List: HB A, HB B Standard Modulo arithmetic
HB HDM Configuration	Base 0 Size 1GB DPA Skip 0 IG 2 (1K) IW 1 (x2) TargetList: Port0, Port1	Base 0 Size 1GB DPA Skip 0 IG 3 (2K) IW 2 (x4) TargetList: Port0, Port1, Port2 , Port3
Switch HDM configuration	Base 0 Size 1GB DPA Skip 0 IG 3 (2K) IW 2 (x4) TargetList: Port0, Port1, Port 2, Port3	N/A
Device HDM configuration	Base 0 Size 1GB DPA Skip 0 IG 2 (1K) IW 3 (x8)	Base 0 Size 1GB DPA Skip 0 IG 2 (1K) IW 3 (x8)
Example HPA - Host Physical Address	HPA 0x3400	HPA 0x3400
Step 1: XHB Platform HW extract InterleaveWays	N/A	Platform HW: Extract 1 bit (CFMWS.ENIW) from HPA starting at bit 10 (CFMWS.HBIG + 8) = 1b
Step 2: XHB Platform HW forward request to host bridge	N/A	Platform HW: Forward request to CFMWS.TargetList[1] = HB B
Step 3: HB find decoder	HB A: Decoder 0	HB B: Decoder 0
Step 4: HB extract InterleaveWays	HB A: Extract 1 bit (IW) from HPA starting at bit 10 (IG+8) = 1b	HB B: Extract 2 bit (IW) from HPA starting at bit 11 (IG+8) = 10b
Step 5: HB forward request to port	HB A: Forward request to TargetList[1] = Port1 (Switch C)	HB B: Forward request to TargetList[2] = Port2 (Device 6)
Step 6: Switch find decoder	Switch C: Decoder 0	N/A
Step 7: Switch extract InterleaveWays	Switch C: Extract 2 bits (IW) from HPA starting at bit 11 (IG+8) = 10b	N/A
Step 8: Switch forward request to port	Switch C: Forward request to TargetList[2] = Port2 (Device 6)	N/A

HPA to DPA Translation	Example 1	Example 2
Step 9: Device find decoder	Device 6: Decoder 0	Device 6: Decoder 0
Step 10: Device calculate DBA Base	Device 6: Decoder0.DPABase = 0	Device 6: Decoder0.DPABase = 0
Step 11: Device calculate HPAOffset	Device 6: 0x1400	Device 6: 0x1400
Step 12: Device calculate DPAOffset	Device 6: Remove 3 bits (Device.IW) from HPAOffset starting at bit 10 (Device.IG + 8), shift upper address bits right 3 bits = 0x400	Device 6: Remove 3 bits (Device.IW) from HPAOffset starting at bit 10 (Device.IG + 8), shift upper address bits right 3 bits = 0x400
Step 13: Device calculate DPA	Device 6: 0x400	Device 6: 0x400
Example final translated DPA	DPA 0x400 on Device 6	DPA 0x400 on Device 6

Table 2-15 HPA to DPA Translation 2

HPA to DPA Translation	Example 3	Example 4
Configuration description	Memory Region Example – Region interleaved across 3 HBs for configuration -x3 XHB interleave -2 devices on each HB -x6 volatile mem interleave set (Device 0, Device 1, .., Device 5)	Memory Region Example – Region interleaved across 6 HBs for configuration -x6 XHB interleave, XOR math -2 devices on each HB -x12 volatile mem interleave set (Device 0, Device 1, .., Device 11)
Platform HW XHB configuration	CFMWS.Base 0 CFMWS.WindowsSize 1GB CFMWS.ENIW 8 (x3) CFMWS.HBIG 2 (1K) Standard Modulo arithmetic Target List: HB A, HB B, HB C	CFMWS.Base 0 CFMWS.WindowsSize 1GB CFMWS.ENIW 9 (x6) CFMWS.HBIG 1 (512B) Modulo Arithmetic with XOR CXIMS = 0x2200 (XOR A9 with A13) Target List: HB A, HB B, HB C, HB D, Not a CXL HB, Not a CXL HB This represents a heterogeneous interleaving configuration since some of the targets are not CXL.
HB HDM Configuration	Base 0 Size 1GB DPA Skip 0 IG 1 (512B) IW 1 (x2) TargetList: Port0, Port1	Base 0 Size 1GB DPA Skip 0 IG 1 (256B) IW 1 (x2) TargetList: Port0, Port1
Switch HDM configuration	N/A	N/A
Device HDM configuration	Base 0 Size 1GB	Base 0 Size 1GB

HPA to DPA Translation	Example 3	Example 4
	DPA Skip 0 IG 1 (512B) IW 9 (x6)	DPA Skip 0 IG 0 (256B) IW 10 (x12)
Example HPA - Host Physical Address	HPA 0x3400	HPA 0x3400
Step 1: XHB Platform HW extract InterleaveWays	Platform HW: Calculate MOD3 of HPA[51:10] because CFMWS.HBIG + 8=10. 0xD MOD3 = 1	Platform HW: Bitwise AND of 0x3400, 0x2200 is 0x2000. XORing all bits in 0x2000 yields 1. Calculate MOD3 of HPA[51:10] because CFMWS.HBIG + 9=10. 0xD MOD3 = 1. Multiply it by 2 and add to Step a output. (2*1+1=3)
Step 2: XHB Platform HW forward request to host bridge	Platform HW: Forward request to CFMWS.TargetList[1] = HB B	Platform HW: Forward request to CFMWS.TargetList[3] = HB D (With Standard Modulo arithmetic, this would have been CFMWS.TargetList[1] i.e. HB B)
Step 3: HB find decoder	HB A: Decoder 0	HB D: Decoder 0
Step 4: HB extract InterleaveWays	HB B: Extract 1 bit (IW) from HPA starting at bit 9 (IG+1) =0	HB D: Extract 1 bit (IW) from HPA starting at bit 8 (IG) =0
Step 5: HB forward request to port	HB B: Forward request to TargetList[0] = Port0 (Device 3)	HB D: Forward request to TargetList[0] = Port0 (Device 6)
Step 6: Switch find decoder	N/A	N/A
Step 7: Switch extract InterleaveWays	N/A	N/A
Step 8: Switch forward request to port	N/A	N/A
Step 9: Device find decoder	Device 3: Decoder 0	Device 6: Decoder 0
Step 10: Device calculate DPA Base	Device 3: Decoder0.DPABase = 0	Device 6: Decoder0.DPABase = 0
Step 11: Device calculate HPAOffset	Device 3: 0x3400	Device 6: 0x3400
Step 12: Device calculate DPAOffset	Device 3: Divide HPAOffset[51:10] by 3 to get 4. DPAOffset[51:9]=4. DPAOffset[8:0]= HPAOffset[8:0].	Device 6: Divide HPAOffset[51:10] by 3 to get 4. DPAOffset[51:8]=4. DPAOffset[7:0]= HPAOffset[7:0].
Step 13: Device calculate DPA	Device 3: 0x800	Device 6: 0x400
Example final translated DPA	DPA 0x800 on Device 3	DPA 0x400 on Device 6

2.13.24.1 Implications of Exclusively-OR (XOR) Math on HPA to DPA Translation

When modulo arithmetic combined with XOR is used, some middle order address bits below HPA[28] are XORed with the traditional interleave selector bits during xHB decode phase to select the target HB. These middle order address bits are communicated via the XORMAP array elements in the CXIMS structure. The set bits across all the XORMAP elements must be disjoint i.e. no two elements are permitted to have the same bit set.

CEDT may contain multiple CFMWS entries with XOR Modulo arithmetic. In such a case, the number of CXIMS entries must match the number of CFMWS entries with XOR Modulo arithmetic and they must be in the same order. The CXIMS entry number N is associated with the N'th CFMWS entry with XOR Modulo arithmetic. For example, a CEDT instance may include 4 CHBS entries, 3 CFMWS entries and 2 CXIMS entries as shown below.

{ CHBS0, CHBS1, CHBS2, CHBS3, CFMWS0, CFMWS1, CFMWS2, CXIMS0, CXIMS1, ..}

Assume that CFMWS0 and CFMWS2 have the XOR Modulo Arithmetic flag set.

In this case, CXIMS0 describes the XOR calculation associated with CFMWS0 range and CXIMS1 describes the XOR calculation associated with CFMWS2. Since CFMWS1 uses standard modulo arithmetic, it is not associated with any CXIMS entry.

Here is an example that illustrates the difference between Standard and XOR Arithmetic: With Standard Modulo arithmetic, 4-way interleaving at 256B granularity may use HPA[9:8] as the interleave selector. HPA[9] is the selector MSb and HPA[8] is the selector LSb. With a host that implements Modulo arithmetic combined with XOR, Interleave selector LSb may be computed by XORing HPA[8] with HPA[11], HPA[17] and HPA[25]. HPA[9] may be XORed with HPA[12], HPA[18] and HPA[26] while calculating Interleave Selector MSb.

Figure 2-71 shows a configuration that is constructed out of 4-way xHB interleaving. Interleave selector LSb is XOR of HPA[8] with HPA[11], HPA[17] and HPA[25]. Interleave Selector MSb is XOR of HPA[9], HPA[12], HPA[18] and HPA[26]. This is represented in CEDT as follows:

CFMWS.Base = 0

CFMWS.WindowsSize = 4 GB

CFMWS.Interleave Arithmetic = 01 (Modulo arithmetic combined with XOR)

CFMWS.ENIW 2 (x4)

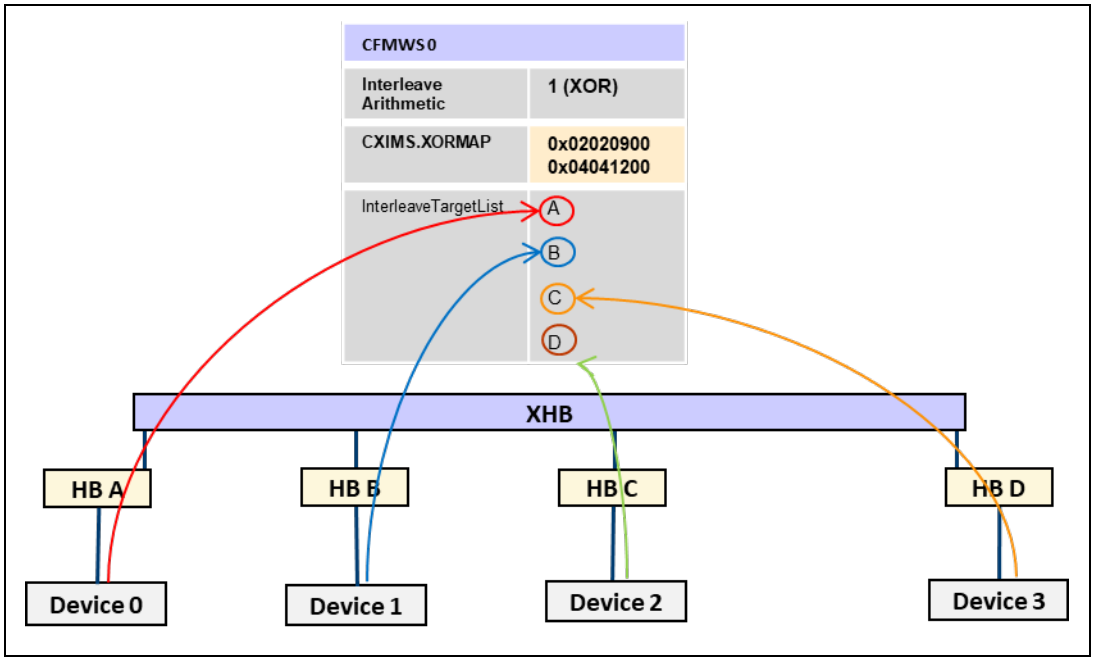
CFMWS.HBIG 0 (256B)

CXIMS.XORMAP[0] = 0x02020900 (indicates XOR of A8,A11,A17,A25)

CXIMS.XORMAP[1] = 0x04041200 (Indicates XOR of A9,A12,A18,A26)

Target List: HB A, HB B, HB C, HB D

Figure 2-71 4-way xHB Interleave with XOR



The following table shows HPA to DPA conversion for this 4-way interleaving configuration. It may be used to cross-check software implementations. Note the ISP corresponding to HPA=2080 is different due to XOR. Without XOR, it would have been 0.

All the numbers are in decimal format.

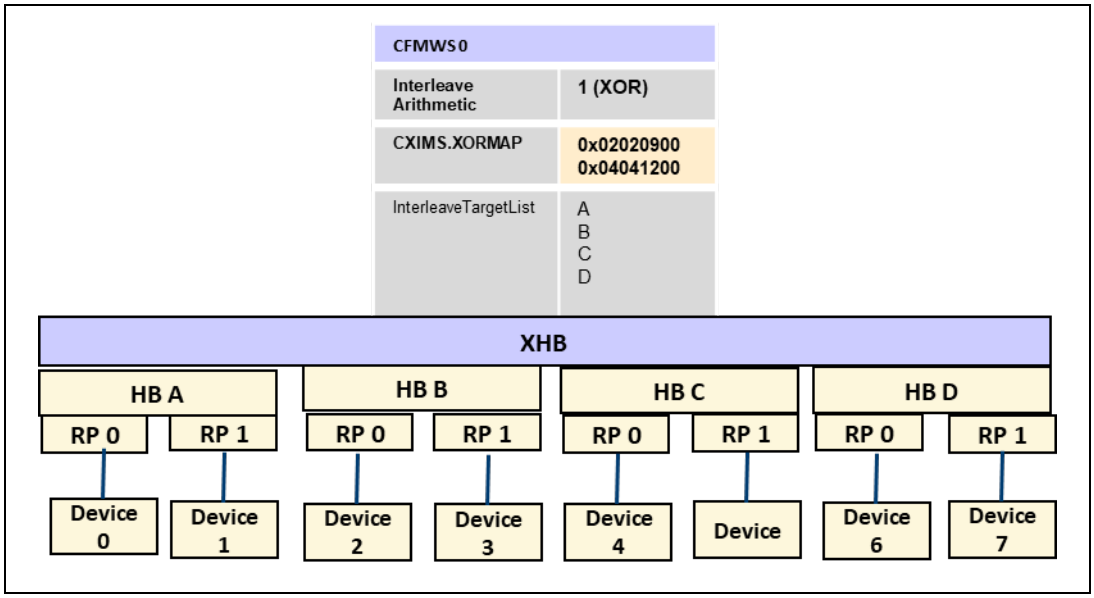
Table 2-16 HPA to DPA Translation – 4-way XOR

HPA	HB	ISP (Device number)	DPA
248	0	0	248
272	1	1	16
528	2	2	16
800	3	3	32
1056	0	0	288
1312	1	1	288
1568	2	2	288
1824	3	3	288
2080	1	1	544
2336	0	0	544
2592	3	3	544
4112	2	2	1040
6176	3	3	1568

HPA	HB	ISP (Device number)	DPA
131088	1	1	32784
262160	2	2	65552
393248	3	3	98336
393496	2	2	98328
393520	2	2	98352
1775813747	0	0	443953523

Here is another example of XOR configuration.

Figure 2-72 8-way Interleave with XOR



The 8-way Interleave is constructed by combining 4-way xHB interleaving with 2-way interleaving at each HB.

- 4-way xHB interleaving. Interleave selector LSb is XOR of HPA[8] with HPA[11], HPA[17] and HPA[25]. Interleave Selector MSb is XOR of HPA[9], HPA[12], HPA[18] and HPA[26]. This is represented in CEDT as follows:
  - CFMWS.Base = 0
  - CFMWS.WindowsSize = 4 GB
  - CFMWS.ENIW 2 (x4)
  - CFMWS.Interleave Arithmetic = 01 (Modulo arithmetic combined with XOR)
  - CFMWS.HBIG 0 (256B)
  - CXIMS.XORMAP[0] = 0x02020900 (indicates XOR of A8,A11,A17,A25)
  - CXIMS.XORMAP[1] = 0x04041200 (Indicates XOR of A9,A12,A18,A26)
  - Target List: HB A, HB B, HB C, HB D

- Each Host Bridge selects the target Root Port is based on HPA[10]. This information is available via the HB HDM Decoders.

The following table shows HPA to DPA conversion for this 8-way interleaving configuration. It may be used to cross-check software implementations. All the numbers are in decimal format.

Table 2-17 HPA to DPA Translation – 8-way XOR

HPA	HB	RP	ISP (Device number)	DPA
248	0	0	0	248
272	1	0	2	16
528	2	0	4	16
800	3	0	6	32
2064	1	0	2	272
4112	2	0	4	528
6176	3	0	6	800
131088	1	0	2	16400
262160	2	0	4	32784
393248	3	0	6	49184
393496	2	0	4	49176
393520	2	0	4	49200
932162229	3	0	6	116520373
1957525275	1	1	3	244690459
2342899463	1	1	3	292862215
245769350	0	1	1	30721158
1971096847	0	1	1	246386959
581610561	1	1	3	72701249
4235060509	1	1	3	529382429
1529057420	2	0	4	191132300
148712089	1	0	2	18589081
2754367011	3	1	7	344295715

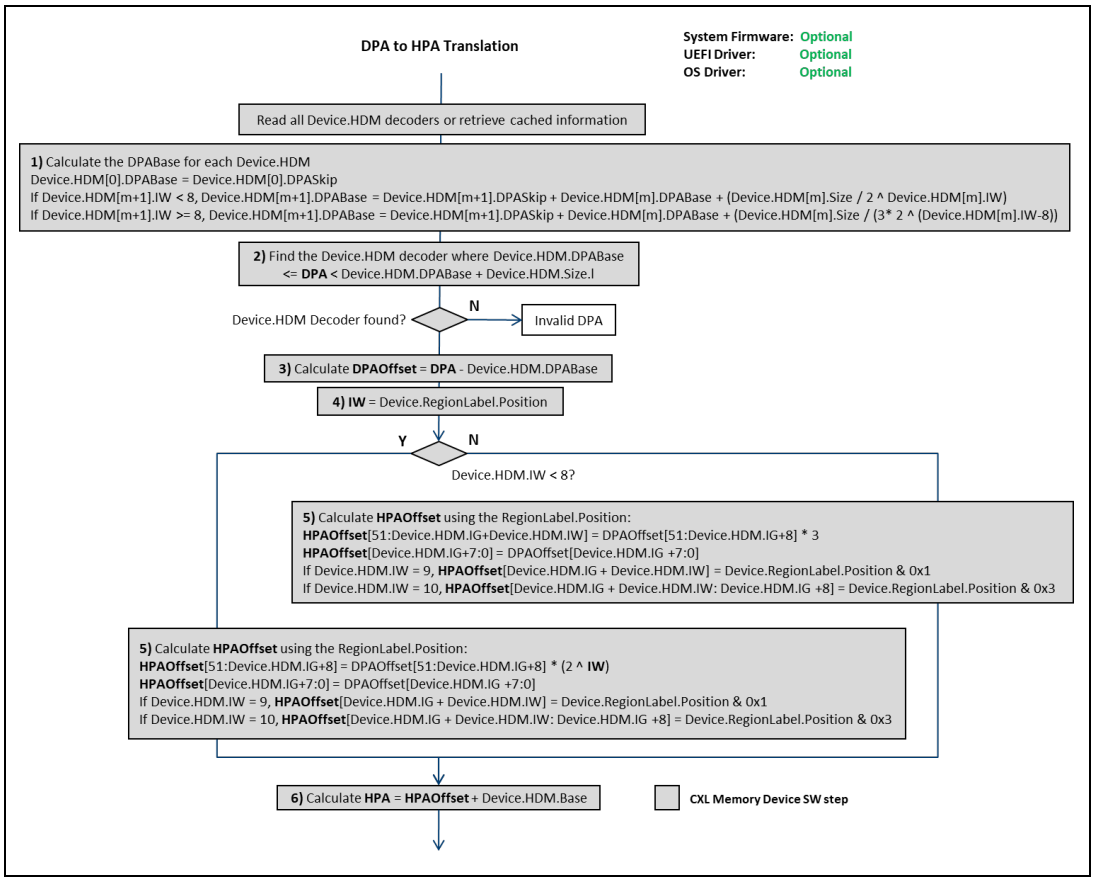
2.13.25 DPA to HPA Translation Sequence

The following figure outlines the device physical address to host physical address translation that system firmware (FW First), UEFI and OS drivers may need to implement if consuming general media event records that report a DPA from the device.

Note that when the IW bits are added back into the address, software needs to insert the correct number for that device which is equivalent to that device's RegionLabel.Position or associated ISP. If the device does not implement LSA,

which is typical for a volatile memory device, Software can calculate the Interleave Set position of the device (ISP) for each CFMWS entry by walking the CXL tree depth-first. For example, in Example 4 from Section 2.13.24, the device below HB D and Port 1 is associated with ISP=3 *2+1 because HB D is CFMWS Target list[3] and Port 1 is listed as HB D's Target List[1]. Figure 2-73 uses the term RegionLabel.Position, but it can be substituted by ISP if LSA is not present.

Figure 2-73 High-level Sequence: DPA to HPA Translation (Standard Modulo Arithmetic)



Here are some example DPA to HPA translations referencing the steps in the previous flow.

Table 2-18 DPA to HPA Translation

DPA to HPA Translation	Example 1	Example 2
Configuration description	See Memory Region Example – Region interleaved across 2 CXL switches for configuration -1 HB -2 Switch -4 devices on each HB -x8 pmem interleave set	See Memory Region Example – Region interleaved across 2 HBs for configuration -x2 XHB interleave -4 devices on each HB -x8 pmem interleave set Standard Modulo arithmetic

DPA to HPA Translation	Example 1	Example 2
	Standard Modulo arithmetic	
Device HDM configuration	Base 0 Size 1GB DPA Skip 0 IG 2 (1K) IW 3 (x8)	Base 0 Size 1GB DPA Skip 0 IG 2 (1K) IW 3 (x8)
Example DPA - Host Physical Address	DPA 0x400 on Device 6	DPA 0x400 on Device 6
Step 1: Calculate DBA Base for each device HDM	Device 6: Decoder0.DPABase = 0	Device 6: Decoder0.DPABase = 0
Step 2: Find device decoder	Device 6: Decoder 0	Device 6: Decoder 0
Step 3: Calculate DPAOffset for decoder	Device 6: 0x400	Device 6: 0x400
Step 4: Use Device.RegionLabel.Position as the IW	IW = RegionLabel.Position = 5	IW = RegionLabel.Position = 5
Step 5: Calculate HPAOffset	Device 6: Insert 3 bits (Device.IW) into DPAOffset starting at bit 10 (Device.IG + 8), shift upper address bits left 3 bits = 0x3400	Device 6: Insert 3 bits (Device.IW) from HPAOffset starting at bit 10 (Device.IG + 8), shift upper address bits left 3 bits = 0x3400
Step 6: Calculate HPA from HPAOffset	HPA 0x3400	HPA 0x3400

When dealing with a multiple of 3 interleaving scheme, bit-shift alone is not sufficient and a multiplication operation is necessary. For example, in the case of a 12 way interleave at 256B granularity, HPA can be computed as

HPA = DPA[7:0] + 12 * 256 * DPA[51:8] + ISP * 256,

Where ISP is the position of the device in the interleave set and it is calculated by walking the CXL tree depth-first.

More generally, when IW>=8 for example, for a multiple of 3 interleaving

HPA = DPA[7+IG:0] + (3 * 2**(IW-8)) * (2**(IG+8)) * DPA[51:IG+8] + ISP * (2**(IG+8))

2.13.25.1 Implications of XOR Math on DPA to HPA Translation

As described in the previous section, HPA to DPA conversion involves removal of certain HPA bits to deal with power-of-2 interleaving. DPA to HPA conversion therefore requires recovering these HPA bits. With standard modulo arithmetic, these bits can be easily extracted from the interleave set position (or region label.position) of the device.

When Modulo arithmetic combined with XOR, the extraction of these HPA bits involves two steps

Step 1: Construct an interim HPA called HPA1 using the algorithm used when Standard Modulo arithmetic is employed. See Section 2.13.25 for details.

Step 2: XOR certain middle order bits in HPA1 with the Interleave Selector bits in HPA1 to undo the XOR operation that occurred during HPA to DPA conversion. If 0<IW<8, the interleave selector bits are HPA1[IG+IW+7:IG+8]. If 8<IW<0Bh, the interleave selector bits are HPA1[IG+IW-1:IG+8]. There shall be one XORMAP array element in the CXIMS corresponding to each HPA bit that needs to be recovered using this XOR operation.

Algorithm:

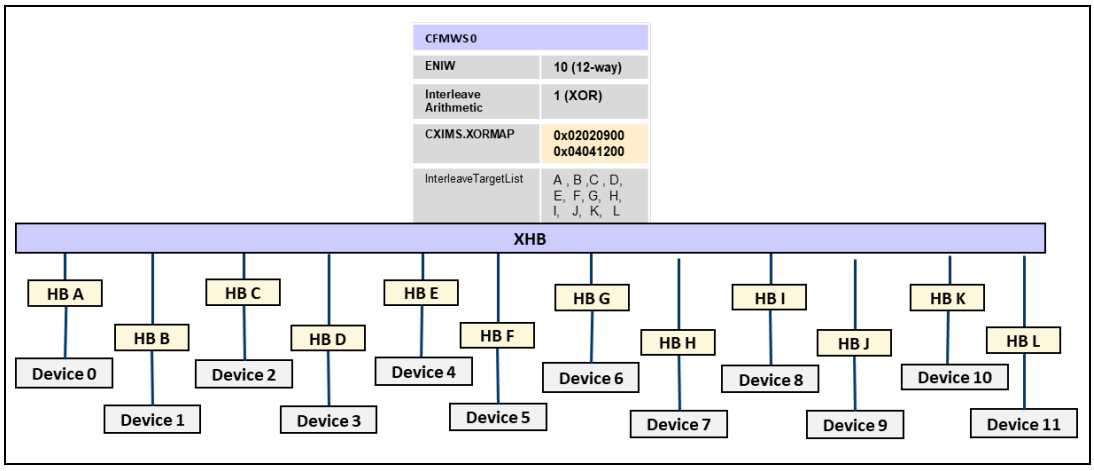
Step 1: Calculate HPA1 using the algorithm used when Standard Modulo arithmetic is employed.

Step 2: Perform the XOR adjustment.

1. Locate the CFMWS entry associated with HPA1. Locate the CXIMS entry associated with this CFMWS entry. Section 2.13.24.1 describes how to associate CFMWS and CXIMS entries.
2. Walk through all the elements of the XORMAP list array in the CXIMS. NIB field in the CXMIS represents the number of array elements, each element being 8 bytes in size.
3. For each XORMAP element XORMAP[m], where 0 <= m < NIB
   a. Store it in a temporary variable XORMAP1=XORMAP[m]
   b. Identify Xm, position of the lowest set bit in XORMAP1
   c. Logically AND XORMAP1 and HPA1. XOR together all the bits in the result to get HPA[Xm]. (If the logical AND of XORMAP1 and HPA1 has even number of 1's, HPA[Xm]=0, otherwise HPA[Xm]=1.)

Consider the configuration shown in Figure 2-74.

Figure 2-74 12-way xHB Interleave with XOR



- 12-way xHB interleaving at 256B interleave granularity
  — HPA[51:10] MOD 3 is used to calculate the 3-way HB selector.

- The 3-way selection must be combined with a 4-way selector to get 12-way interleaving since 12=3 * 4. For the remaining 4-way selection, Host Bridge Selector LSb is XOR of HPA[8], HPA[11], HPA[17] and HPA[25] and Host Bridge Selector MSb is XOR of HPA[9], HPA[12], HPA[18] and HPA[26]. This information is encoded in two elements of XORMAP array in CXIMS.

- One device attached to each HB and HBs are listed in the CFMWS Target List in order. Since one CXL device is attached directly to each HB, ISP = HB.

This information is encoded in CFMWS as follows

CFMWS.Base = 0

CFMWS.WindowsSize = 6 GB

CFMWS.ENIW 10 (x12 interleave)

CFMWS.Interleave Arithmetic = 01 (Modulo arithmetic combined with XOR)

CFMWS.HBIG 0 (256B)

Interleave Target List: HB A, HB B, HB C, HB D, HB E, HB F, HB G, HB H, HB I, HB J, HB K, HB L

Per the specification, each CXIMS entry is variable size. XORMAP[0] is the first entry. Since 12=4*3, there must be 2 XORMAP entries in the CXIMS associated with this CFMWS entry, one for each x2 interleave.

- CXIMS.XORMAP[0] = 0x02020900 (indicates XOR of A8,A11,A17,A25)
- CXIMS.XORMAP[1] = 0x04041200 (Indicates XOR of A9,A12,A18,A26)

For this example, the first iteration in step 2.c is expanded below

For the first XORMAP element XORMAP[0],

- XORMAP1=0x02020900
- Bit 8 is the lowest set bit in XORMAP1
- HPA[8]=HPA1[8] XOR HPA1[11] XOR HPA1[17] XOR HPA1[25]

The following table shows HPA to DPA and DPA to HPA transformation for this configuration. All numbers in the below are represented in decimal.

Table 2-19 HPA to DPA and DPA to HPA Translation - XOR

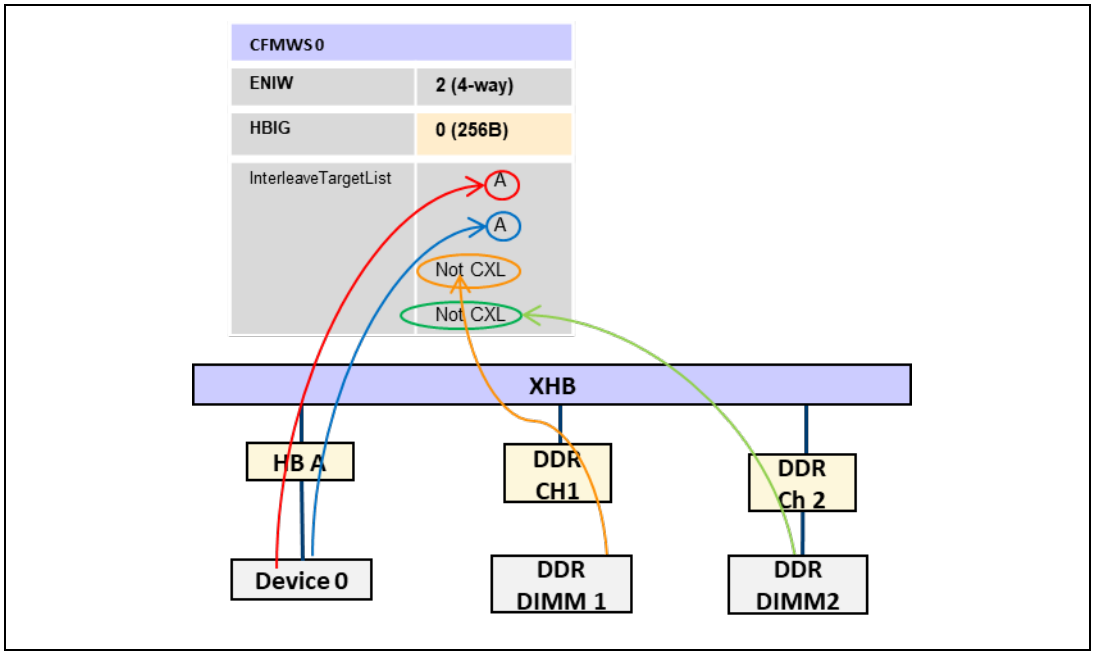
HPA	HB	ISP	DPA	HPA	HPA1	XOR Adjustment
224	0	0	224	224	224	0
272	1	1	16	272	272	0
528	2	2	16	528	528	0
800	3	3	32	800	800	0
1056	4	4	32	1056	1056	0

HPA	HB	ISP	DPA	HPA	HPA1	XOR Adjustment
1312	5	5	32	1312	1312	0
1568	6	6	32	1568	1568	0
1824	7	7	32	1824	1824	0
2080	9	9	32	2080	2336	-256
2336	8	8	32	2336	2080	256
2592	11	11	32	2592	2848	-256
2848	10	10	32	2848	2592	256
3104	0	0	288	3104	3360	-256
3588205439	7	7	299017087	3588205439	3588205439	0
3950524995	0	0	329210435	3950524995	3950524483	512
1812608653	11	11	151050637	1812608653	1812608909	-256
1742030654	2	2	145169214	1742030654	1742030398	256
3947985996	10	10	328998732	3947985996	3947986508	-512
1911027415	3	3	159252439	1911027415	1911027671	256
4105186020	5	5	342098916	4105186020	4105185764	-768
1175645096	8	8	97970344	1175645096	1175644328	768
2579948404	8	8	214995572	2579948404	2579947636	768
1215475645	7	7	101289661	1215475645	1215475645	0
485088911	7	7	40424079	485088911	485089167	-256
2777503900	7	7	231458716	2777503900	2777504668	-768

2.13.25.2 Duplication of HB Instance in CFMWS Target List

It is legal for a single HB instance to appear twice in the CFMWS target list. This situation can occur in heterogeneous interleaving configuration such as the one below where the Interleave set is made up of CXL devices and DDR DIMMs.

Figure 2-75 Duplication of HB Instances - Example



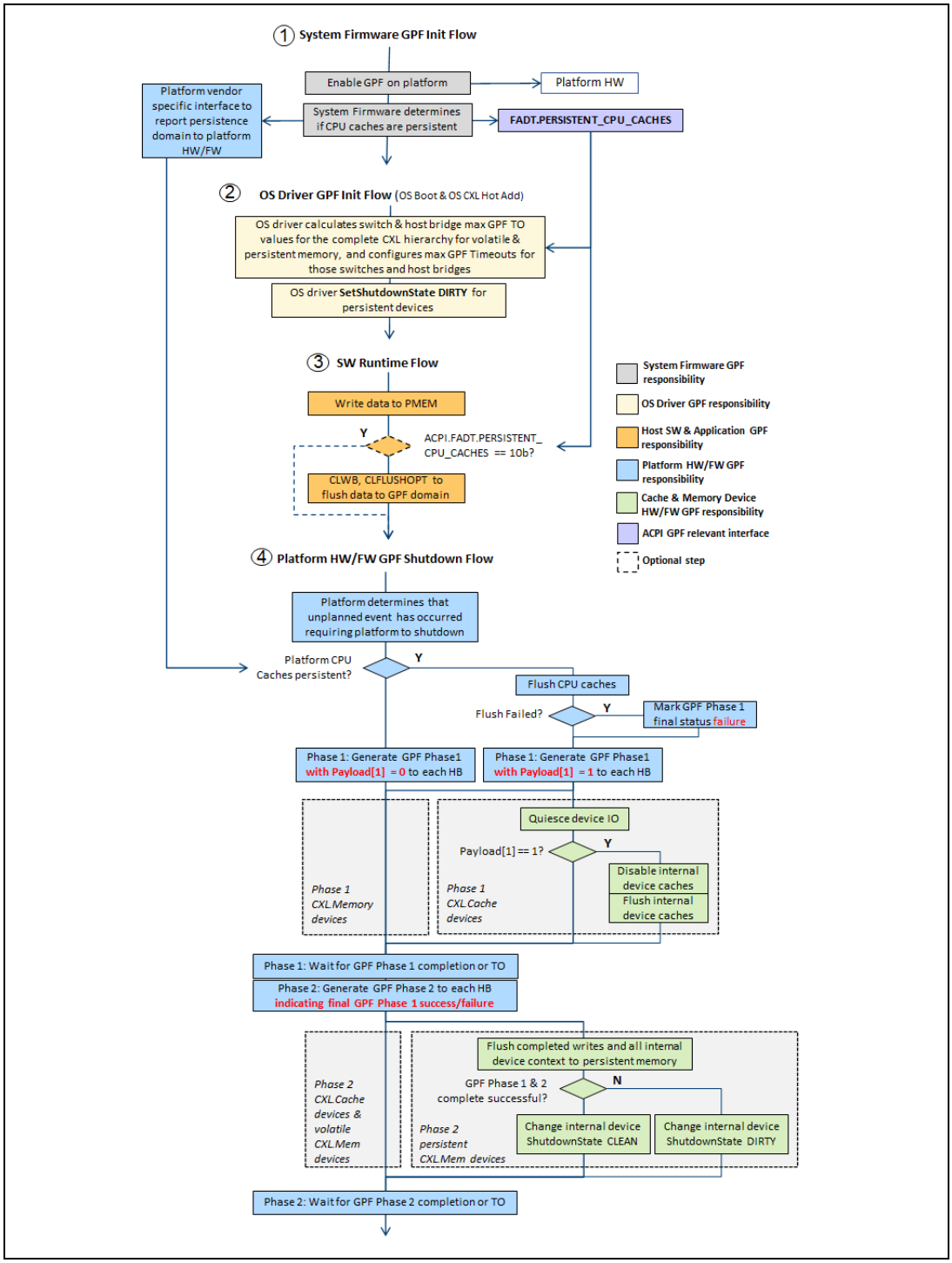
If CXL Host Bridge appears twice in the list in consecutive position, that means the hardware will send 2*256 consecutive bytes out of every 4*256 bytes to that HB and the device below it.

From the device perspective, that essentially means a 2 way interleave (IW=1) at 512B granularity (IG=1). During the DPA to HPA translation, the interleaving scheme as seen by the device along with the position of the device in that Interleave set shall be used.

2.13.26 GPF Sequence

The following figure outlines the high-level sequence for GPF. The System Firmware, UEFI and OS Drivers, and the platform HW/FW all play apart in the GPF sequence.

Figure 2-76 High-level Sequence: GPF



A EFI Volatile Memory Type and Attributes For Linux

This section briefly explains the CXL Consortium's recommendations for correct usage of the memory type and attributes as currently supported by EFI. The following memory types are of interest:

Table 2-20 EFI Memory Types

EFI Type	Notes
EFI_CONVENTIONAL_MEMORY	Conventional and CXL memory
EFI_RESERVED_TYPE	Reserved memory (for example, on a CXL type 2 accelerator / offload device)

The EFI "specific purpose memory" attribute was added to help Linux handle performance-differentiated memory. System firmware/bios should apply this attribute to all volatile CXL memory.

Table 2-21 EFI Memory Attribute

EFI Attribute	Notes
EFI_MEMORY_SP	Specific Purpose Memory (SPM)

When Linux sees the EFI_MEMORY_SP attribute, it avoids using the subject memory during boot, and never uses it for kernel data structures. Instead, the memory becomes available as one or more devdax devices (for example, /dev/dax0.0, which can be memory mapped by applications to access the memory).

Starting in Linux 6.3, the default memory policy is to wait until after boot and then online the CXL memory as system-ram. This policy can be overridden via the "memhp_default_state=offline" kernel command line argument, or with build-time kernel configuration options. Thus the default behavior (system-ram) can be overridden to devdax mode by modifying the kernel command line and rebooting once.

The daxctl utility can be used to perform online conversion between system-ram and device-dax modes – with the caveat that conversion from system-ram to devdax mode cannot always be guaranteed to work (because it is not always possible to relocate all contents of the memory). It is for this reason that Linux needs the EFI_MEMORY_SP attribute to prevent onlining CXL memory as system-ram during boot.

The attribute has no effect with operating systems that ignore the SPM attribute. Operating systems that ignore EFI_MEMORY_SP will see the memory as conventional.

Note that the CDAT DSEMTS can override the type to EFI_RESERVED_TYPE (which must be respected by firmware/bios), but the CDAT DSEMTS should not

be followed if it omits the EFI_MEMORY_SP attribute; the EFI_MEMORY_SP attribute should always be applied unless the CDAT DSEMTS overrides the type to EFI_RESERVED_TYPE.

The only cases where it is recommended to omit the EFI_MEMORY_SP attribute are those where the system contains exclusively CXL memory, or where CXL memory is interleaved with DDR memory or where the CXL memory is required to boot.